

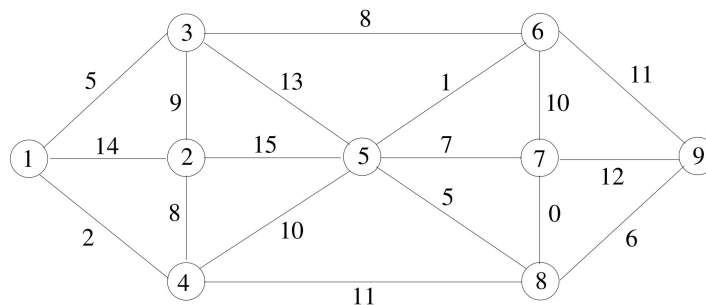
Take-home exam: December 19, 2012.

Course name: Network Optimization

Course no: 42115

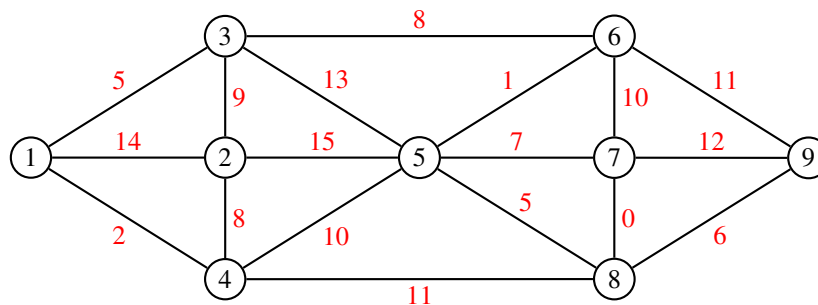
Minimum spanning tree

Given the following graph $G = (V, E)$



Problem 1: (small) Find the minimum spanning tree of G using Kruskal's algorithm ■

Answer 1



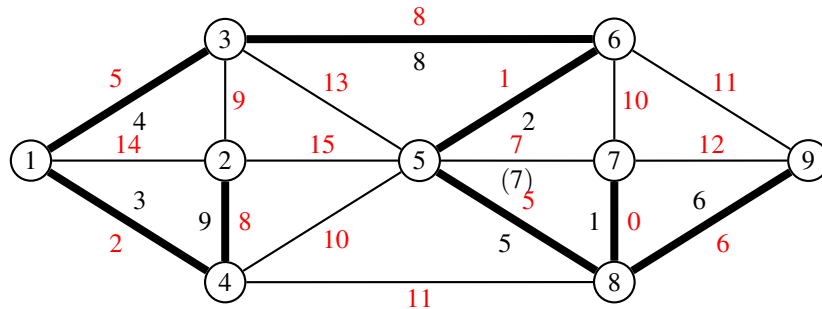
First we sort the edges according to their weight

$e_{78}, e_{56}, e_{14}, e_{13}, e_{58}, e_{89}, e_{57}, e_{36}, e_{24}, e_{23}, e_{45}, e_{67}, e_{48}, e_{69}, e_{79}, e_{35}, e_{12}, e_{25}$

We then add the edges in increasing order, each time checking if a cycle is introduced

$e_{78}, e_{56}, e_{14}, e_{13}, e_{58}, e_{89}, (e_{57}), e_{36}, e_{24}$

The edge marked with paranthesis was skipped since it would introduce a cycle. We stop when $n - 1 = 8$ edges has been added.



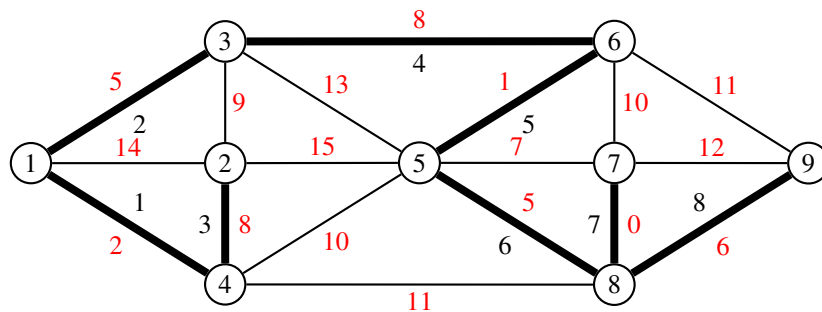
The cost of the MST is $0 + 1 + 2 + 5 + 5 + 6 + 8 + 8 = 35$. ■

Problem 2: (small) Find the minimum spanning tree of G using Prim's algorithm ■

Answer 2 Starting from node 1, we add the following edges:

$$e_{14}, e_{13}, e_{24}, e_{36}, e_{56}, e_{58}, e_{78}, e_{89}$$

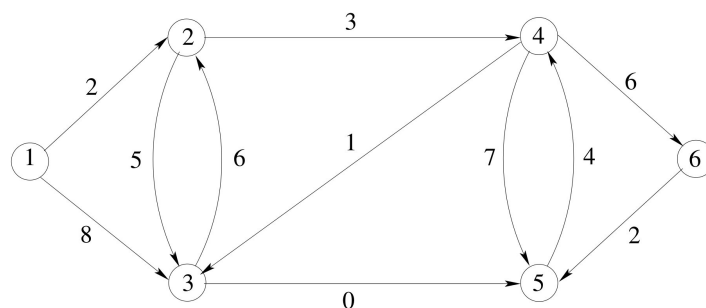
We stop when the last node has been considered.



The cost of the MST is again $0 + 1 + 2 + 5 + 5 + 6 + 8 + 8 = 35$. ■

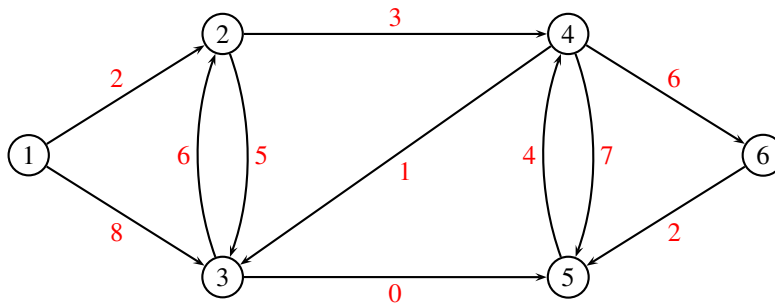
It is sufficient to indicate the chosen edges on the graph and to give each edge a sequence number (in which order they have been considered).

Shortest path

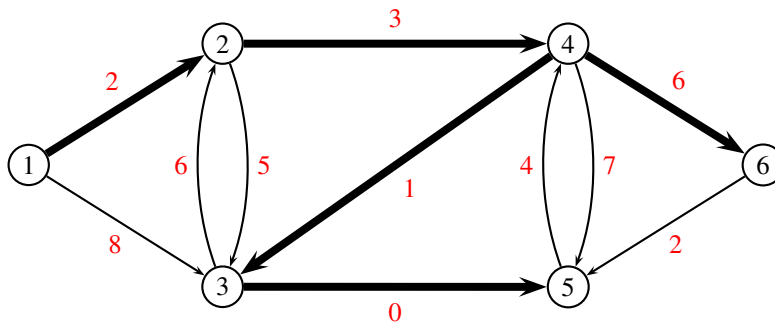


Problem 3: (small) Use Dijkstra's algorithm to solve the above shortest path problem starting from node 1. ■

Answer 3 We have the following



relaxing	potential					
	1	2	3	4	5	6
init	∞	∞	∞	∞	∞	∞
1	0	2	8	∞	∞	∞
2			7	5		
4			6		12	11
3					6	
5						
6						



It is sufficient to indicate the chosen edges in a shortest path tree on the graph and to give each edge a sequence number (in which order they have been considered). Moreover, the distance to each node should be indicated on the node.

Shortest path tree

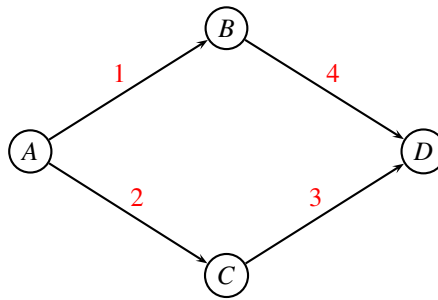
Problem 4: (large) Are the following statements correct? (Prove or disprove each statement. To disprove a statement it is sufficient to show an example.)

- The shortest path tree is unique when all edges have distinct weights.
- In a directed graph with positive edge weights the shortest paths do not change when directed edges are replaced with undirected edges.
- In a shortest path problem, if all edge weights are increased by adding a constant $k > 0$ then the cost of a shortest path is also increased by a multiple of k .

- d) Among all shortest paths in $G = (V, E)$ the Dijkstra algorithm finds the shortest path with fewest edges.

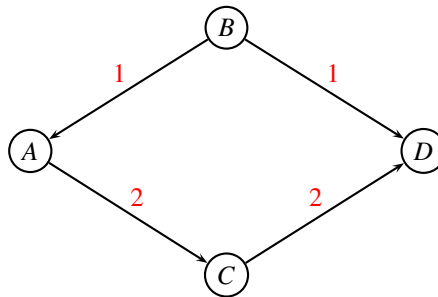
■

Answer 4 a) Consider the following weighted directed graph



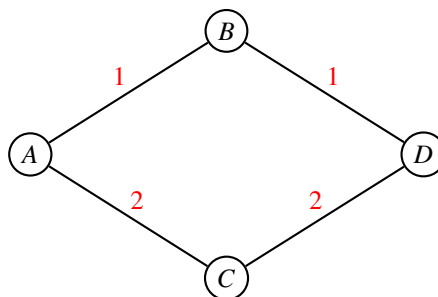
All edges have distinct weights, but there are two paths A to D having the same distance. So, the answer is: **false**.

b) Consider the following weighted directed graph



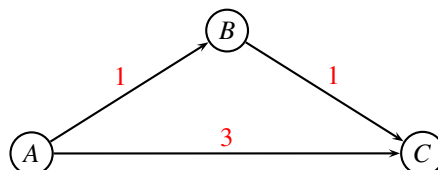
The shortest path from A to D is $A \rightarrow C \rightarrow D$.

Now, consider the same undirected graph



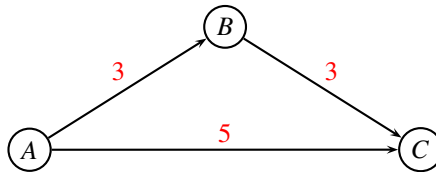
The shortest path from A to D is now $A \rightarrow B \rightarrow D$. So, the answer is: **false**.

c) Consider the following weighted directed graph



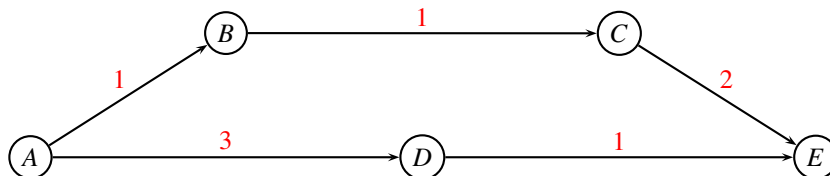
The shortest path from A to C is $A \rightarrow B \rightarrow C$ with cost 2.

Now, increase all weights by a constant $k = 2$



The shortest path from A to C is $A \rightarrow C$ with cost 5. The increase is not a multiple of $k = 2$. So, the answer is: **false**.

d) Consider the following weighted directed graph



Dijkstra will first consider node A , then node B and node C before considering node D . This means that it will find path $A \rightarrow B \rightarrow C \rightarrow E$ as shortest path from A to E . But the shortest path with minimum number of edges from A to E is $A \rightarrow D \rightarrow E$. So, the answer is: **false**.

e) Consider the same weighted directed graph as in d) above. Notice that Bellman-Ford does not define in which order the edges should be considered, we should just consider all edges. So, assume that Bellman-Ford in the first iteration considers the edges in the order

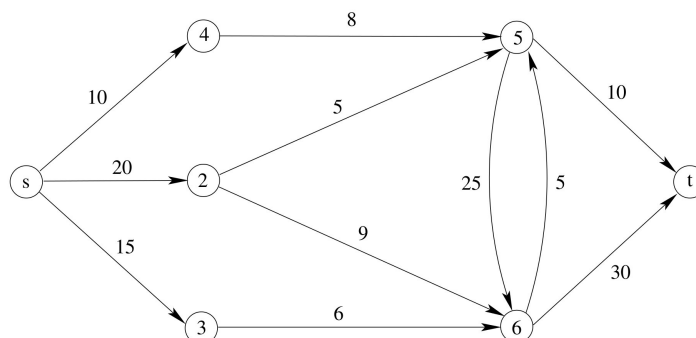
$(A, B), (B, C), (C, E), (A, D), (D, E)$.

This means that the path $A \rightarrow B \rightarrow C \rightarrow E$ is updated before path $A \rightarrow D \rightarrow E$, and hence we will not find the path with fewest edges. So, the answer is: **false**.

■

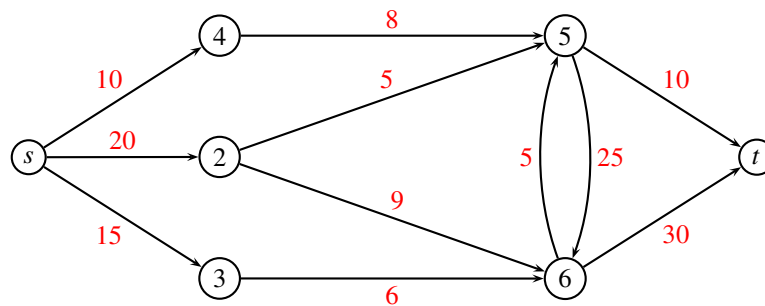
Maximum flow problem

Consider the maximum flow problem with source s and sink t



Problem 5: (medium) Solve the max flow problem using an augmenting path algorithm. In each iteration let the augmenting path be the shortest (measured in number of edges) path from s to t in the residual graph. In each iteration write down which augmenting path was used. ■

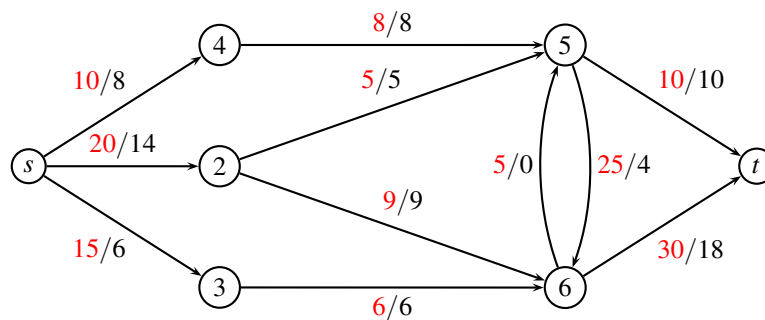
Answer 5 We have



We do the following iterations:

- Augmenting path: $s45t$, capacity 8.
- Augmenting path: $s26t$, capacity 9.
- Augmenting path: $s36t$, capacity 6.
- Augmenting path: $s25t$, capacity 2.
- Augmenting path: $s256t$, capacity 3.

Getting the solution



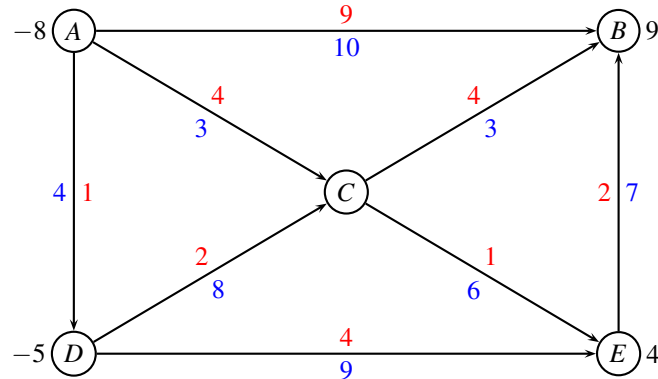
■

Problem 6: (small) Indicate a minimum cut, and report the capacity of the cut and the value of the flow. ■

Answer 6 A cut is $S = \{s, 2, 3, 4\}$ and $T = \{5, 6, t\}$. The capacity of the cut is 28. The flow in the solution is also 28. ■

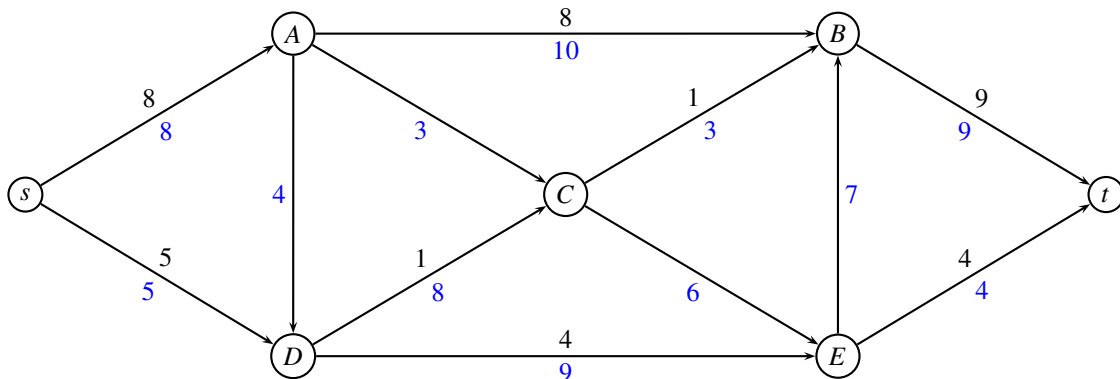
Min cost flow

Consider the following network with edge costs c_{ij} (red) and edge capacities u_{ij} (blue). The demand b_i in every node is positive for a sink, and negative for a source.



Problem 7: (small) Find first a legal start solution by solving an appropriate max-flow problem. ■

Answer 7 We add two extra nodes s, t , and solve a maximum-flow problem as follows: (blue values are capacity, black values are flow)

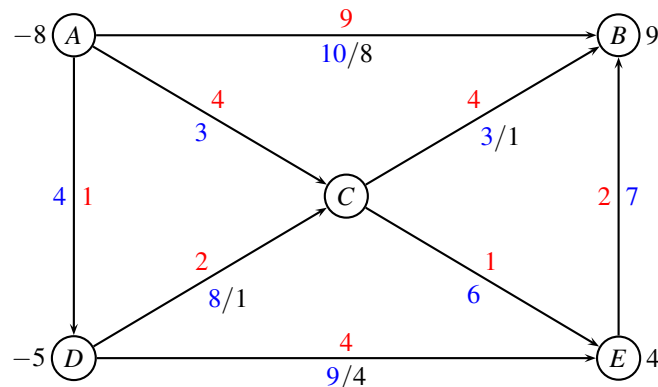


■

It is sufficient to draw the max-flow problem and to show the solution. No details about how the max-flow problem is solved are needed.

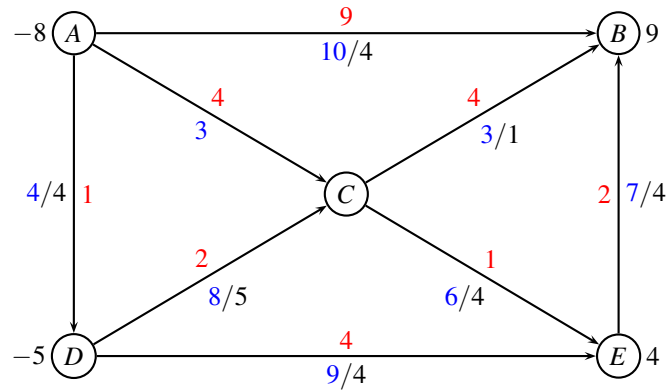
Problem 8: (large) Use the augmenting cycle algorithm on the above network. In each iteration, write down which augmenting cycle was used. ■

Answer 8 We start with:



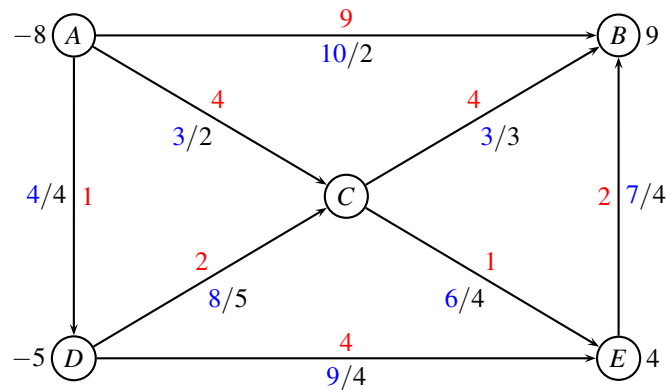
The cost of the initial solution is $72 + 4 + 2 + 16 = 94$.

An augmenting cycle is $ADCEBA$ with cost $1 + 2 + 1 + 2 - 9 = -3$. Maximum capacity is $\min\{4, 7, 6, 7, 8\} = 4$. After sending the 4 units the new graph becomes:



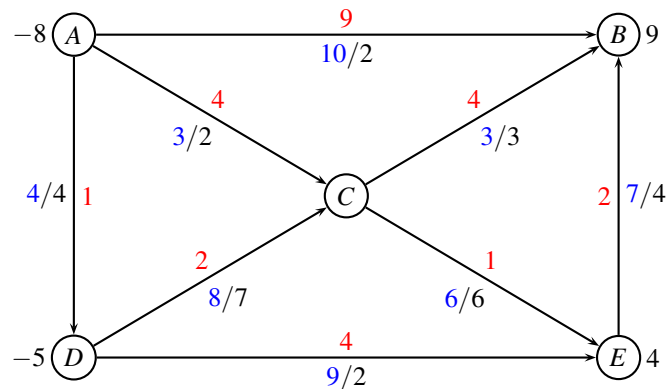
The cost of the solution is $36 + 4 + 4 + 8 + 10 + 4 + 16 = 82$.

An augmenting cycle is ACB with cost $4 + 4 - 9 = -1$. Maximum capacity is $\min\{3, 2, 8\} = 2$. Sending 2 units along the cycle:



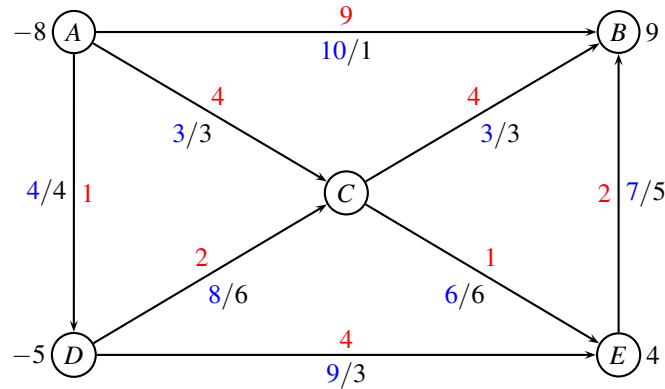
The cost of the solution is $18 + 8 + 12 + 4 + 8 + 10 + 4 + 16 = 80$.

An augmenting cycle is DCE with cost $2 + 1 - 4 = -1$. Maximum capacity is $\min\{3, 2, 4\} = 2$. Sending 2 units along the cycle:



The cost of the solution is $18 + 8 + 12 + 4 + 8 + 14 + 6 + 8 = 78$.

An augmenting cycle is $ACDEBA$ with cost $4 - 2 + 4 + 2 - 9 = -1$. Maximum capacity is $\min\{1, 5, 5, 3, 2\} = 1$. Sending 1 units along the cycle:



The cost of the solution is $9 + 12 + 12 + 4 + 10 + 12 + 6 + 12 = 77$. No more augmenting cycles can be found, so the solution is optimal.

Checking with CPLEX

```

minimize
  9 xAB + 4 xAC + 1 xAD + 4 xCB + 1 xCE + 2 xDC + 4 xDE + 2 xEB

subject to
  A: - xAD - xAB - xAC = -8
  B: xAB + xCB + xEB = 9
  C: xAC + xDC - xCB - xCE = 0
  D: xAD - xDC - xDE = -5
  E: xCE + xDE - xEB = 4

  xAB <= 10
  xAC <= 3
  xAD <= 4
  xCB <= 3
  xCE <= 6
  xDC <= 8
  xDE <= 9
  xEB <= 7

```

end

Dual simplex - Optimal: Objective = 7.7000000000e+01
 Solution time = 0.00 sec. Iterations = 3 (0)

Variable Name	Solution Value
xAB	1.000000
xAC	3.000000
xAD	4.000000
xCB	3.000000
xCE	6.000000
xDC	6.000000
xDE	3.000000
xEB	5.000000

■

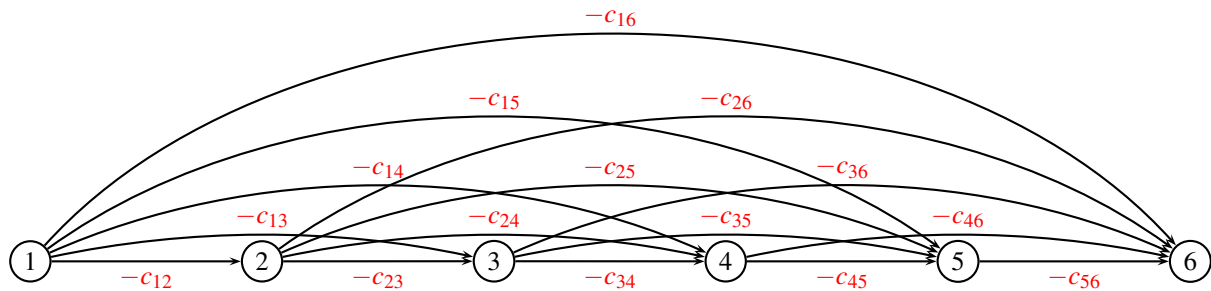
No details about how you found an augmenting cycle are needed.

Text formatting

The well-known document processing program LATEX uses an optimization procedure to decompose a paragraph into several lines so that when lines are left- and right-adjusted, the appearance of the paragraph will be the most attractive. Suppose that a paragraph consists of n words and that each word is assigned a sequence number. Let $c_{ij} > 0$ denote the attractiveness of a line if it begins with the word i and ends with word $j - 1$. Bigger values of c_{ij} are better. The program LATEX uses formulas to calculate the value of each c_{ij} , and we can assume they are given.

Problem 9: (large) Given the c_{ij} values, show how to formulate the problem of decomposing the paragraph into several lines of text in order to maximize the total attractiveness (of all lines) as a shortest path problem. ■

Answer 9 We get the graph:



Problem 10: (small) Does the graph have any negative cycles? ■

Answer 10 The figure is a DAG (directed acyclic graph). This means that all edges are pointing forward, hence no cycles are possible. ■

Problem 11: (medium) Propose an algorithm for solving the above problem. Try to find an algorithm with the best possible running time (in big-Oh notation). ■

Answer 11 Due to the negative edge weights, we cannot use Dijkstra which has the attractive running time $O(m \log n)$, where m is the number of edges and n is the number of nodes.

However, since we have a DAG, we can use dynamic programming, considering the nodes from left to right. In each step we find:

$$y_j = \min_{i:(i,j) \in E} \{y_i + w_{ij}\}$$

Since all y_i values are calculated before we consider y_j , the recursion is valid. The running time is $O(n + m)$, where n is the number of nodes and m is the number of edges. ■

Hotel planning

A hotel has received a number of requests for the following week as shown in the following table

start\end	Wed	Thu	Fri
Mon	2	3	-
Tue	4	3	5
Wed	-	4	3

Each entry in the table gives the number of guests requesting a room from the "start" day to the "end" day. A dash indicates that no requests have been given. A stay may not be split.

The hotel gets 2\$ for a 1-day stay, 5\$ for a 2-day stay, and 8\$ for a 3-day stay. The hotel wants to maximize the revenue. However, only the following number of rooms is available each of the nights:

Mon-Tue	Tue-Wed	Wed-Thu	Thu-Fri
5	6	8	7

Problem 12: (xlarge) Formulate the problem as a minimum cost flow problem by drawing a graph $G = (V, E)$ where each edge (i, j) has a cost c_{ij} , and an upper bound on the flow u_{ij} . Moreover, every node i has a demand b_i which is negative for a source, and positive for a sink. ■

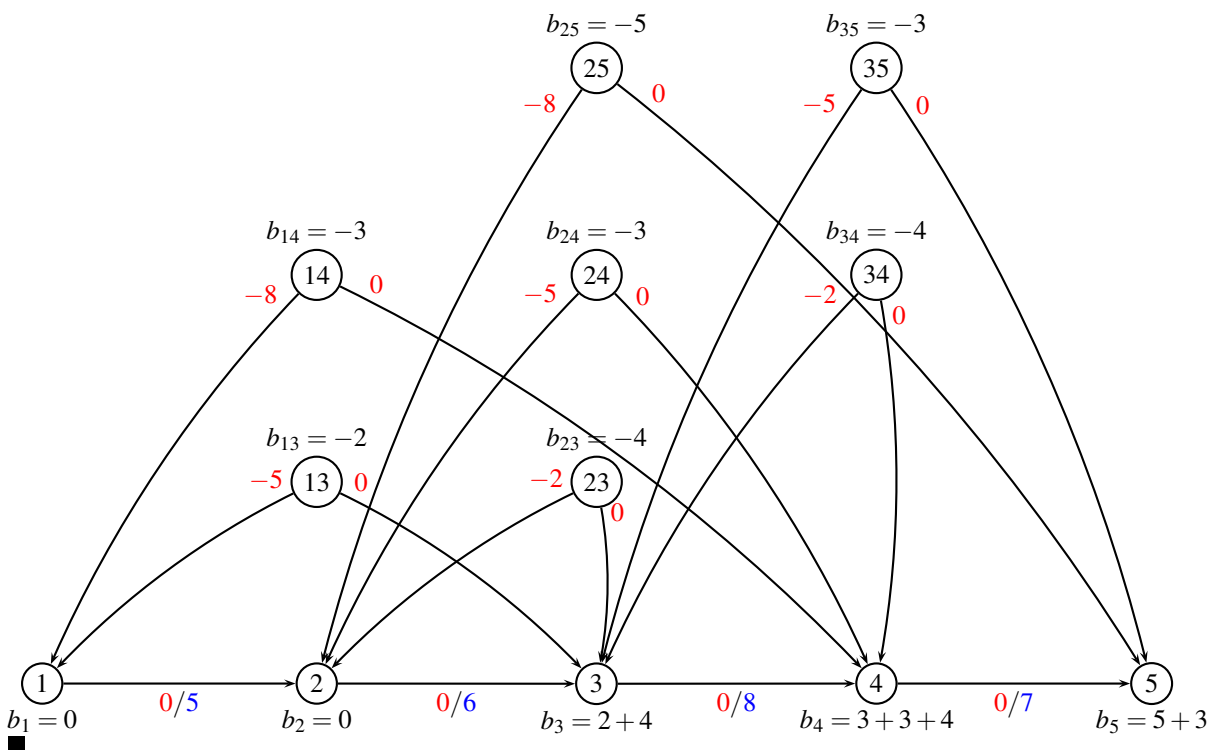
Answer 12 Let nodes 1, 2, 3, 4, 5 correspond to the week days considered. An edge from day i to day $i + 1$ has the capacity corresponding to the number of available rooms.

For every rental period from day i to day j we create a node ij . The node ij is connected to node i with an edge having cost $-p$ where p is the revenue of a stay from day i to day j . The edge has infinite capacity. Moreover, the node ij is connected to node j with an edge having cost 0 and infinite capacity.

Every node ij has a demand b_{ij} corresponding to the minus number of requests for a stay from day i to day j .

Every node i corresponding to a week day, has a demand b_i corresponding to the sum of all requests ending day i . E.g. Wednesday has a demand $b_3 = 2 + 4$, and Thursday has a demand $b_4 = 3 + 3 + 4$.

This leads to the following graph (capacities are blue, costs are red. If no capacity is given, then the capacity is infinite).



Hint: See application 9.4 in Ahuja, Magnanti, Orlin "Network Flows". (Available on campusnet).

Problem 13: (medium) Solve the problem with CPLEX or any other solver. Enclose a copy of the MIP model, and report the optimal solution. ■

Answer 13 We use the minimum cost flow formulation from above:

```

minimize
- 8 x14_1 - 5 x13_1 - 8 x25_2 - 5 x24_2 - 2 x23_2 - 5 x35_3 - 2 x34_3

subject to

- x14_1 - x14_4 = -3
- x13_1 - x13_3 = -2
- x25_2 - x25_5 = -5
- x24_2 - x24_4 = -3
- x23_2 - x23_3 = -4
- x35_3 - x35_5 = -3
- x34_3 - x34_4 = -4

x13_1 + x14_1 - x1_2 = 0
x23_2 + x24_2 + x25_2 + x1_2 - x2_3 = 0
x13_3 + x23_3 + x35_3 + x34_3 + x2_3 - x3_4 = 6
x14_4 + x24_4 + x34_4 + x3_4 - x4_5 = 10
x25_5 + x35_5 + x4_5 = 8

x1_2 <= 5
x2_3 <= 6
x3_4 <= 8
x4_5 <= 7

end

```

```

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 4 rows and 0 columns.
Aggregator did 8 substitutions.
Reduced LP has 4 rows, 10 columns, and 20 nonzeros.
Presolve time = 0.00 sec.
Initializing dual steep norms . . .

```

```

Iteration log . . .
Iteration: 1 Dual objective = -71.000000

```

```

Dual simplex - Optimal: Objective = -6.0000000000e+01
Solution time = 0.00 sec. Iterations = 3 (0)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
x14_1              1.000000
x13_1              1.000000
x25_2              4.000000
x35_3              3.000000
x14_4              2.000000
x13_3              1.000000
x25_5              1.000000
x24_4              3.000000
x23_3              4.000000
x34_4              4.000000
x1_2               2.000000
x2_3               6.000000
x3_4               8.000000
x4_5               7.000000
All other variables in the range 1-18 are 0.

```

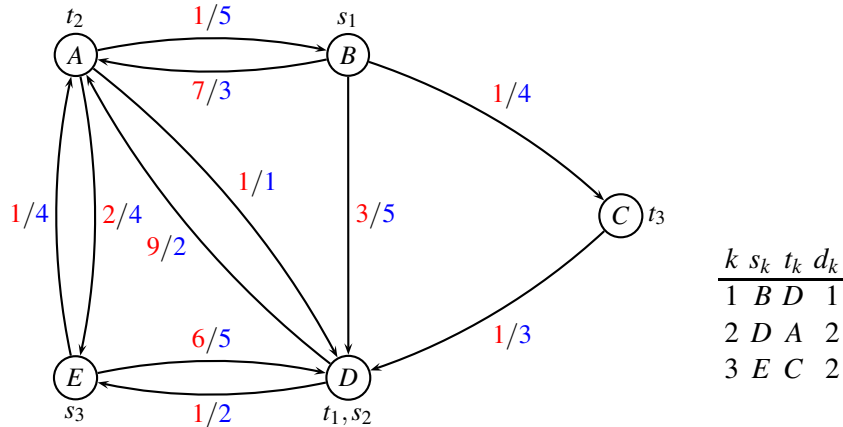
So, the solution is

start\end	Wed	Thu	Fri
Mon	1	1	-
Tue	0	0	4
Wed	-	0	3

With a total profit of 60\$. ■

Multicommodity flow

Consider the following unsplittable multicommodity flow problem where each edge has a cost and a capacity c_{ij}/u_{ij} . There are three commodities as indicated in the table, each commodity k having source s_k , terminal t_k and demand d_k .



A path formulation of the problem is given in the last page of the appendix.

Comment: In the following questions, the problem should be solved in the unsplittable version, but since it was not clear from the formulation, both splittable and unsplittable versions are accepted.

Problem 14: (small) Fill in the objective value in the path formulation, and solve the problem to LP-optimality and IP-optimality. Notice that the number of commodities flown is handled in the last constraints, as opposed to the project assignment. ■

Answer 14 (Unsplittable) We have the paths

number	commodity k	route r	d_k	$\sum c_{ij}$	cost
1	1	B→A→D	1	8	8
2	1	B→A→E→D	1	15	15
3	1	B→C→D	1	2	2
4	1	B→D	1	3	3
5	2	D→A	2	9	18
6	2	D→E→A	2	2	4
7	3	E→A→B→C	2	3	6
8	3	E→D→A→B→C	2	17	34

We modify the model from a splittable to an unsplittable problem:

```

minimize
+8x1+15x2+2x3+3x4+18x5+4x6+6x7+34x8
subject to
AB:                                +2x7+2x8 <= 5
AD: +1x1                            <= 1
AE:   +1x2                          <= 4
BA: +1x1+1x2                        <= 3
BC:   +1x3                          +2x7+2x8 <= 4
BD:   +1x4                          <= 5
CD:   +1x3                          <= 3
DA:   +2x5                          +2x8 <= 2
DE:   +2x6                          <= 2
EA:   +2x6+2x7                      <= 4
ED:   +1x2                          +2x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 1
K3: +x7+x8 >= 1

```

```

integer
x1 x2 x3 x4 x5 x6 x7 x8
end

```

The optimal solution is:

```

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 5 rows and 0 columns.
Aggregator did 1 substitutions.
Reduced LP has 8 rows, 7 columns, and 19 nonzeros.
Presolve time = 0.00 sec.

Iteration log . . .
Iteration: 1 Dual objective = 6.000000

Dual simplex - Optimal: Objective = 1.2000000000e+01
Solution time = 0.00 sec. Iterations = 2 (0)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
x3                 1.000000
x6                 1.000000
x7                 1.000000
All other variables in the range 1-8 are 0.

```

■

Answer 14 (Splittable)

```

minimize
+8x1+15x2+2x3+3x4+9x5+2x6+3x7+17x8
subject to
AB:                                +x7+x8 <= 5
AD: +x1                            <= 1
AE:   +x2                          <= 4
BA: +x1+x2                        <= 3
BC:   +x3                          +x7+x8 <= 4
BD:   +x4                          <= 5
CD:   +x3                          <= 3
DA:   +x5                          +x8 <= 2
DE:   +x6                          <= 2
EA:   +x6+x7                      <= 4
ED:   +x2                          +x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 2

```

```

K3: +x7+x8 >= 2

end

CPLEX> read i
Problem 'i.lp' read.
Read time = 0.03 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 5 rows and 0 columns.
Aggregator did 1 substitutions.
Reduced LP has 8 rows, 7 columns, and 19 nonzeros.
Presolve time = 0.00 sec.

Iteration log . . .
Iteration: 1 Dual objective = 10.000000

Dual simplex - Optimal: Objective = 1.2000000000e+01
Solution time = 0.02 sec. Iterations = 2 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
x3                  1.000000
x6                  2.000000
x7                  2.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
DE                  -7.000000
K1                   2.000000
K2                   9.000000
K3                   3.000000
All other dual prices in the range 1-14 are 0.

```

■

Problem 15: (large) Run column generation starting from columns corresponding to the variables x_1, x_5, x_7 . ■

Answer 15 (Unsplittable) When we use column generation we do not know all columns, and hence we should start with columns x_1, x_5, x_7 , and gradually find the next columns.

However, the present problem is very small, and we know all columns in advance, so we can add constraints $x_i \leq 0$ to all columns not being used.

First iteration

```

minimize
+8x1+15x2+2x3+3x4+18x5+4x6+6x7+34x8
subject to
AB:                +2x7+2x8 <= 5
AD: +1x1           <= 1
AE:   +1x2         <= 4
BA: +1x1+1x2       <= 3
BC:   +1x3         +2x7+2x8 <= 4
BD:           +1x4 <= 5
CD:   +1x3         <= 3
DA:           +2x5   +2x8 <= 2
DE:           +2x6   <= 2
EA:           +2x6+2x7 <= 4
ED:   +1x2         +2x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 1
K3: +x7+x8 >= 1

```

```

\\ P1: x1 <= 0
P2: x2 <= 0
P3: x3 <= 0
P4: x4 <= 0
\\ P5: x5 <= 0
P6: x6 <= 0
\\ P7: x7 <= 0
P8: x8 <= 0
end

CPLEX> read m1
Problem 'm1.lp' read.
Read time = 0.00 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 19 rows and 8 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.01 ticks)

Dual simplex - Optimal: Objective = 3.2000000000e+01
Solution time = 0.01 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (0.81 ticks/sec)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
x5                  1.000000
x7                  1.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
K1                   8.000000
K2                  18.000000
K3                   6.000000
P3                  -6.000000
P4                  -5.000000
P6                  -14.000000
All other dual prices in the range 1-19 are 0.

```

Second iteration

```

minimize
+8x1+15x2+2x3+3x4+18x5+4x6+6x7+34x8
subject to
AB:                +2x7+2x8 <= 5
AD: +1x1           <= 1
AE:   +1x2         <= 4
BA: +1x1+1x2       <= 3
BC:   +1x3         +2x7+2x8 <= 4
BD:   +1x4         <= 5
CD:   +1x3         <= 3
DA:   +2x5         +2x8 <= 2
DE:   +2x6         <= 2
EA:   +2x6+2x7     <= 4
ED:   +1x2         +2x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 1
K3: +x7+x8 >= 1

```

```

\\ P1: x1 <= 0
P2: x2 <= 0
P3: x3 <= 0
P4: x4 <= 0
\\ P5: x5 <= 0
\\ P6: x6 <= 0

```



```

\\ P7: x7 <= 0
P8: x8 <= 0
end

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 18 rows and 8 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.01 ticks)

Dual simplex - Optimal: Objective = 1.8000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (10.90 ticks/sec)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
x6                  1.000000
x7                  1.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
DE                   -7.000000
K1                   8.000000
K2                   18.000000
K3                   6.000000
P3                   -6.000000
P4                   -5.000000
All other dual prices in the range 1-18 are 0.

```

Third iteration

```

minimize
+8x1+15x2+2x3+3x4+18x5+4x6+6x7+34x8
subject to
AB:                +2x7+2x8 <= 5
AD: +1x1           <= 1
AE:    +1x2        <= 4
BA: +1x1+1x2       <= 3
BC:    +1x3         +2x7+2x8 <= 4
BD:    +1x4         <= 5
CD:    +1x3         <= 3
DA:    +2x5         +2x8 <= 2
DE:    +2x6         <= 2
EA:    +2x6+2x7     <= 4
ED:    +1x2         +2x8 <= 5

```

```

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 1
K3: +x7+x8 >= 1

```

```

\\ P1: x1 <= 0
P2: x2 <= 0
\\ P3: x3 <= 0
P4: x4 <= 0
\\ P5: x5 <= 0
\\ P6: x6 <= 0
\\ P7: x7 <= 0
P8: x8 <= 0
end

```

```

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 17 rows and 8 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.01 ticks)

```

Dual simplex - Optimal: Objective = 1.2000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (10.57 ticks/sec)

```
CPLEX> dis sol var -
Variable Name      Solution Value
x3                  1.000000
x6                  1.000000
x7                  1.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
DE                  -7.000000
K1                   2.000000
K2                  18.000000
K3                   6.000000
All other dual prices in the range 1-17 are 0.
```

■

Answer 15 (Unsplittable, not all columns known) If we did not know all columns in advance, we had to generate them gradually as follows:

First iteration

```
minimize
    +8x1      +18x5  +6x7
subject to
AB:              +2x7  <= 5
AD: +1x1              <= 1
AE:              <= 4
BA: +1x1              <= 3
BC:              +2x7  <= 4
BD:              <= 5
CD:              <= 3
DA:              +2x5  <= 2
DE:              <= 2
EA:              +2x7  <= 4
ED:              <= 5

K1: +x1 >= 1
K2: +x5 >= 1
K3: +x7 >= 1
end
```

```
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 14 rows and 3 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)
```

Dual simplex - Optimal: Objective = 3.2000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (6.24 ticks/sec)

```
CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
x5                  1.000000
x7                  1.000000
CPLEX> dis sol dua -
Constraint Name      Dual Price
K1                   8.000000
K2                  18.000000
K3                   6.000000
All other dual prices in the range 1-14 are 0.
```

Second iteration

```
minimize
    +8x1      +18x5+4x6+6x7
subject to
AB:          +2x7      <= 5
AD: +1x1      <= 1
AE:          <= 4
BA: +1x1      <= 3
BC:          +2x7      <= 4
BD:          <= 5
CD:          <= 3
DA:          +2x5      <= 2
DE:          +2x6      <= 2
EA:          +2x6+2x7  <= 4
ED:          <= 5

K1: +x1 >= 1
K2: +x5+x6 >= 1
K3: +x7 >= 1
end
```

```
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 14 rows and 4 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal: Objective = 1.8000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (6.76 ticks/sec)
```

```
CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
x6                  1.000000
x7                  1.000000
All other variables in the range 1-4 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
DE                  -7.000000
K1                   8.000000
K2                  18.000000
K3                   6.000000
All other dual prices in the range 1-14 are 0.
```

Third iteration

```
minimize
    +8x1    +2x3    +18x5+4x6+6x7
subject to
AB:          +2x7      <= 5
AD: +1x1      <= 1
AE:          <= 4
BA: +1x1      <= 3
BC:          +1x3      +2x7      <= 4
BD:          <= 5
CD:          +1x3      <= 3
DA:          +2x5      <= 2
DE:          +2x6      <= 2
EA:          +2x6+2x7  <= 4
ED:          <= 5

K1: +x1+x3 >= 1
K2: +x5+x6 >= 1
K3: +x7 >= 1
end
```

```

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 14 rows and 5 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal: Objective = 1.2000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (7.47 ticks/sec)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
x3                  1.000000
x6                  1.000000
x7                  1.000000
All other variables in the range 1-5 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
DE                   -7.000000
K1                    2.000000
K2                   18.000000
K3                    6.000000
All other dual prices in the range 1-14 are 0.

```

■

Answer 15 (Splittable)

First iteration

```

minimize
+8x1+15x2+2x3+3x4+9x5+2x6+3x7+17x8
subject to
AB:                +x7+x8 <= 5
AD: +x1            <= 1
AE:  +x2          <= 4
BA: +x1+x2        <= 3
BC:  +x3          +x7+x8 <= 4
BD:      +x4      <= 5
CD:  +x3          <= 3
DA:      +x5      +x8 <= 2
DE:      +x6      <= 2
EA:      +x6+x7   <= 4
ED:  +x2          +x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 2
K3: +x7+x8 >= 2

\\ X1: x1 <= 0
X2: x2 <= 0
X3: x3 <= 0
X4: x4 <= 0
\\ X5: x5 <= 0
X6: x6 <= 0
\\ X7: x7 <= 0
X8: x8 <= 0

end

```

```

CPLEX> read i0.lp
Problem 'i0.lp' read.
Read time = 0.02 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 19 rows and 8 columns.

```

All rows and columns eliminated.

Presolve time = 0.00 sec.

Dual simplex - Optimal: Objective = 3.2000000000e+01

Solution time = 0.00 sec. Iterations = 0 (0)

CPLEX> dis sol var -

Variable Name	Solution Value
x1	1.000000
x5	2.000000
x7	2.000000

All other variables in the range 1-8 are 0.

CPLEX> dis sol dua -

Constraint Name	Dual Price
K1	8.000000
K2	9.000000
K3	3.000000
X3	-6.000000
X4	-5.000000
X6	-7.000000

All other dual prices in the range 1-19 are 0.

Second iteration

minimize

+8x1+15x2+2x3+3x4+9x5+2x6+3x7+17x8

subject to

AB: +x7+x8 <= 5

AD: +x1 <= 1

AE: +x2 <= 4

BA: +x1+x2 <= 3

BC: +x3 +x7+x8 <= 4

BD: +x4 <= 5

CD: +x3 <= 3

DA: +x5 +x8 <= 2

DE: +x6 <= 2

EA: +x6+x7 <= 4

ED: +x2 +x8 <= 5

K1: +x1+x2+x3+x4 >= 1

K2: +x5+x6 >= 2

K3: +x7+x8 >= 2

\\ X1: x1 <= 0

X2: x2 <= 0

X3: x3 <= 0

X4: x4 <= 0

\\ X5: x5 <= 0

\\ X6: x6 <= 0

\\ X7: x7 <= 0

X8: x8 <= 0

end

CPLEX> read i1.lp

Problem 'i1.lp' read.

Read time = 0.00 sec.

CPLEX> opt

Tried aggregator 1 time.

LP Presolve eliminated 18 rows and 8 columns.

All rows and columns eliminated.

Presolve time = 0.00 sec.

Dual simplex - Optimal: Objective = 1.8000000000e+01

Solution time = 0.00 sec. Iterations = 0 (0)

CPLEX> dis sol var -

Variable Name	Solution Value
x1	1.000000
x6	2.000000
x7	2.000000

All other variables in the range 1-8 are 0.

CPLEX> dis sol dua -

Constraint Name	Dual Price
DE	-7.000000
K1	8.000000
K2	9.000000
K3	3.000000
X3	-6.000000
X4	-5.000000

All other dual prices in the range 1-18 are 0.

Third iteration

```

minimize
+8x1+15x2+2x3+3x4+9x5+2x6+3x7+17x8
subject to
AB:          +x7+x8 <= 5
AD: +x1          <= 1
AE:  +x2          <= 4
BA: +x1+x2        <= 3
BC:  +x3          +x7+x8 <= 4
BD:  +x4          <= 5
CD:  +x3          <= 3
DA:  +x5          +x8 <= 2
DE:  +x6          <= 2
EA:  +x6+x7       <= 4
ED:  +x2          +x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 2
K3: +x7+x8 >= 2

\\ X1: x1 <= 0
X2: x2 <= 0
\\ X3: x3 <= 0
X4: x4 <= 0
\\ X5: x5 <= 0
\\ X6: x6 <= 0
\\ X7: x7 <= 0
X8: x8 <= 0

end

CPLEX> read i2.lp
Problem 'i2.lp' read.
Read time = 0.01 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 17 rows and 8 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec.

Dual simplex - Optimal: Objective = 1.2000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
x3                 1.000000
x6                 2.000000
x7                 2.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price

```

DE	-7.000000
K1	2.000000
K2	9.000000
K3	3.000000

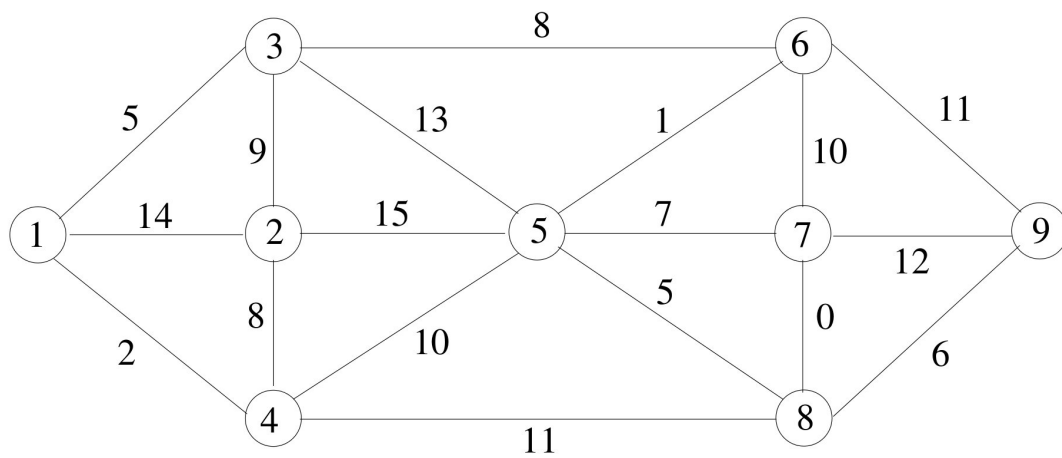
All other dual prices in the range 1-17 are 0.

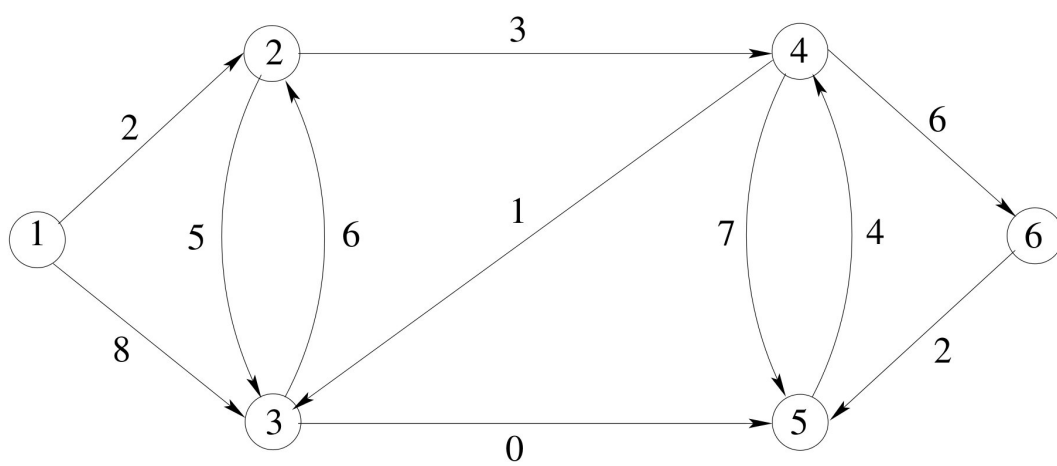
■

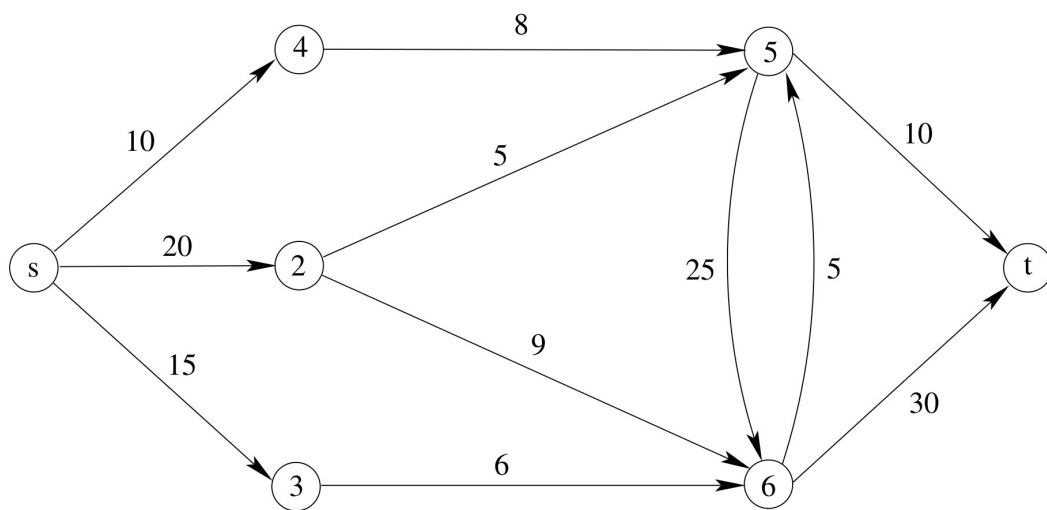
Hint: In the last question you can add a constraint $x_i \leq 0$ to all columns not being used. It is sufficient to show the input (written as an LP-problem) and the primal/dual solution in each iteration. You may use CPLEX or any other LP-solve. When reporting the input it should be written as a normal LP-problem.

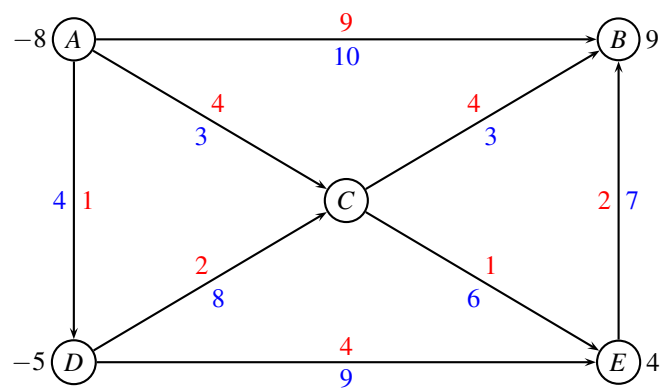
Appendix

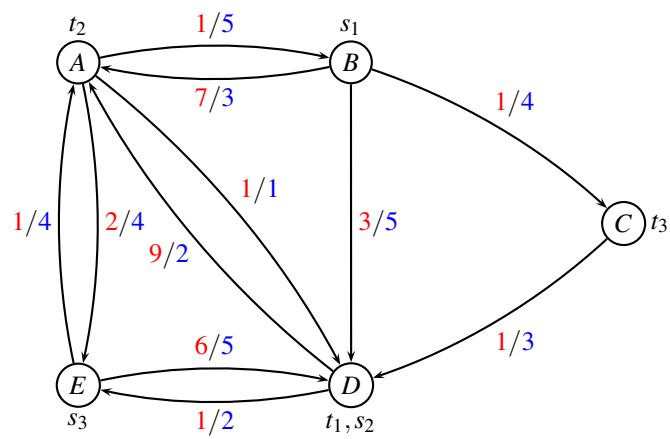
The figures on the following pages can be used when solving the problem. It is allowed to write by hand on the figures in your answer.











```

minimize
  + x1+ x2+ x3+ x4+ x5+ x6+ x7+ x8
subject to
AB:                                     +x7+x8 <= 5
AD: +x1                                <= 1
AE:   +x2                               <= 4
BA: +x1+x2                             <= 3
BC:   +x3                               +x7+x8 <= 4
BD:   +x4                               <= 5
CD:   +x3                               <= 3
DA:   +x5                               +x8 <= 2
DE:   +x6                               <= 2
EA:   +x6+x7                           <= 4
ED:   +x2                               +x8 <= 5

K1: +x1+x2+x3+x4 >= 1
K2: +x5+x6 >= 2
K3: +x7+x8 >= 2

end

```

Take-home exam: December 19, 2012.

Course name: Network Optimization

Course no: 42115

Name	study number

I hereby declare that I have worked alone on the project assignment and not discussed it with anybody else.

Date

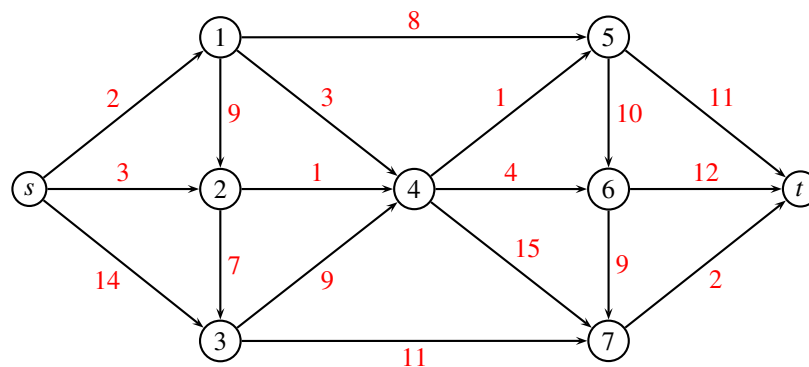
Signature

Take-home exam: December 18, 2013.

Course name: Network Optimization

Course no: 42115

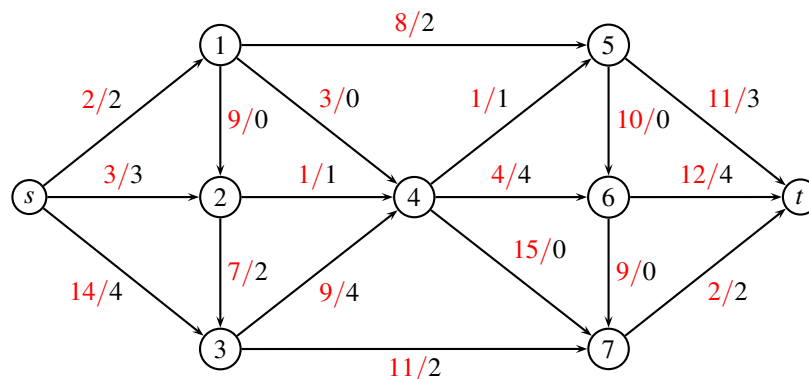
Maximum Flow



Answer 1 We have the following iterations

- $s346t$, 4 units
- $s15t$, 2 units
- $s237t$, 2 units
- $s245t$, 1 unit

This results in the following solution

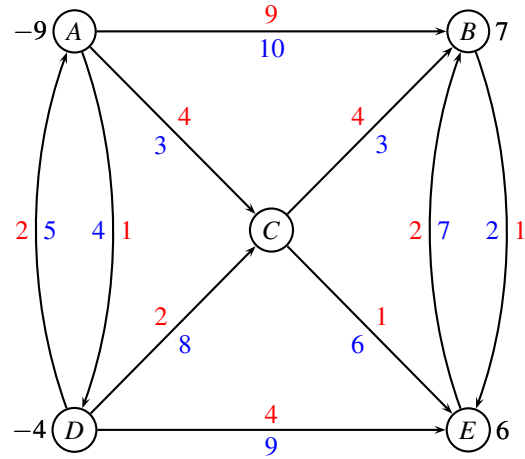


■

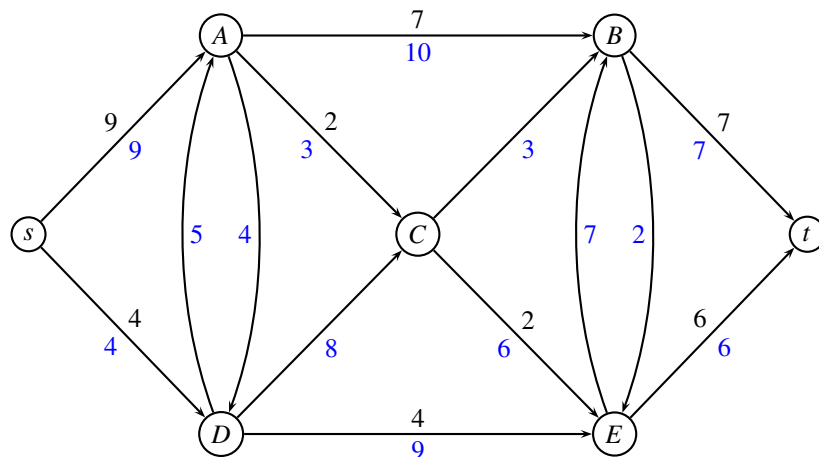
Answer 2 A minimum cut is $S = \{s, 2, 3, 4, 7\}$ and $T = \{1, 5, 6, t\}$ with capacity $2 + 1 + 4 + 2 = 9$. The value of the flow is also 9. ■

Min cost flow

We have the problem

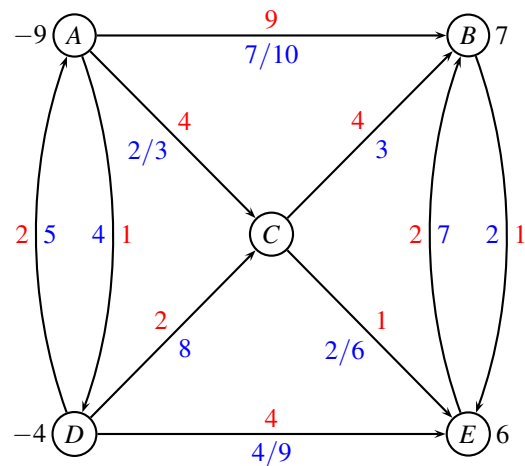


Answer 3 We introduce two extra nodes s and t . Edges are added from s to all nodes with $b_i < 0$, and from all nodes with $b_i > 0$ to t . Capacity of edges is set to $|b_i|$. All costs are ignored.



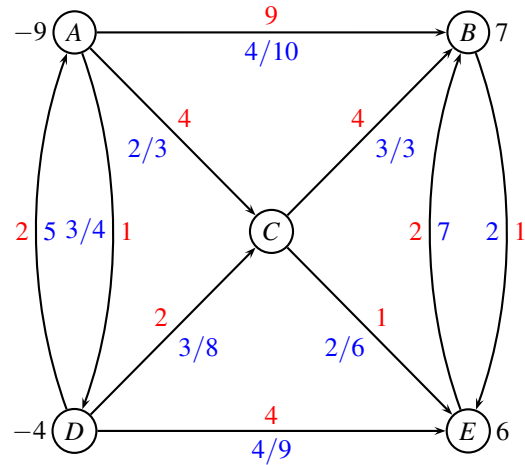
A feasible flow is indicated with black numbers. ■

Answer 4 We have the initial solution

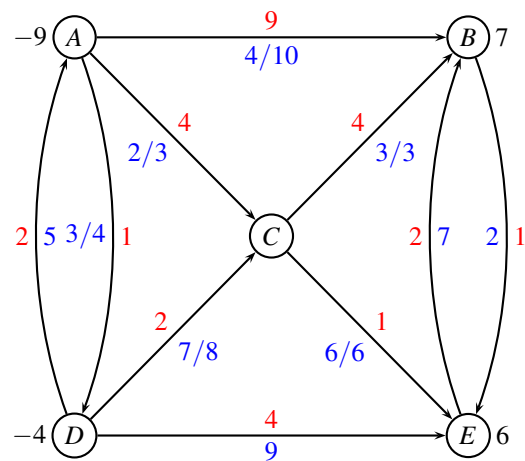


with objective $7 \cdot 9 + 2 \cdot 4 + 2 \cdot 1 + 4 \cdot 4 = 89$

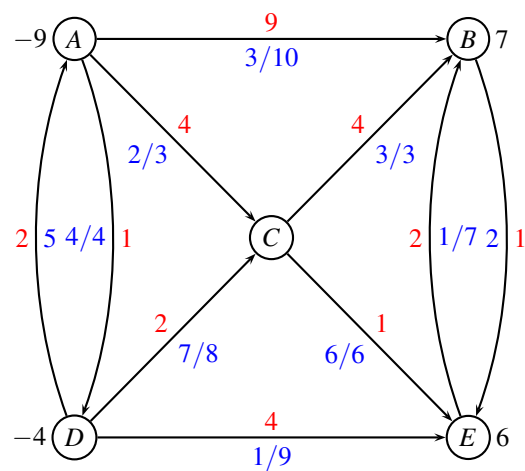
Finding augmenting cycle A, D, C, B, A with cost $1 + 2 + 4 - 9 = -2$. Flowing 3 units.



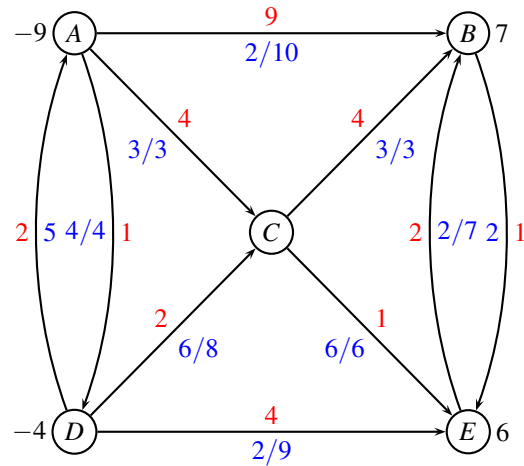
Finding augmenting cycle D, C, E, D with cost $2 + 1 - 4 = -1$. Flowing 4 units.



Finding augmenting cycle A, D, E, B, A with cost $1 + 4 + 2 - 9 = -2$. Flowing 1 unit.



Finding augmenting cycle A, C, D, E, B, A with cost $4 - 2 + 4 + 2 - 9 = -1$. Flowing 1 unit.



No more negative cycles. The optimal solution value is $z = 76$.

Checking solution with CPLEX:

```

minimize
  9xAB+4xAC+1xAD+1xBE+4xCB+1xCE+2xDA+2xDC+4xDE+2xEB

subject to
  -xAB-xAC-xAD+xDA=-9
  xAB-xBE+xCB+xEB=7
  xAC-xCB-xCE+xDC=0
  xAD-xDA-xDC-xDE=-4
  xBE+xCE+xDE-xEB=6

  xAB <= 10
  xAC <= 3
  xAD <= 4
  xBE <= 2
  xCB <= 3
  xCE <= 6
  xDA <= 5
  xDC <= 8
  xDE <= 9
  xEB <= 7
end

CPLEX> read mcf
Problem 'mcf.lp' read.
Read time = 0.02 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 10 rows and 0 columns.
Aggregator did 1 substitutions.
Reduced LP has 4 rows, 9 columns, and 18 nonzeros.
Presolve time = 0.00 sec.

Iteration log . . .
Iteration: 1 Dual objective = 50.000000

Dual simplex - Optimal: Objective = 7.6000000000e+01
Solution time = 0.00 sec. Iterations = 3 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
xAB                2.000000

```

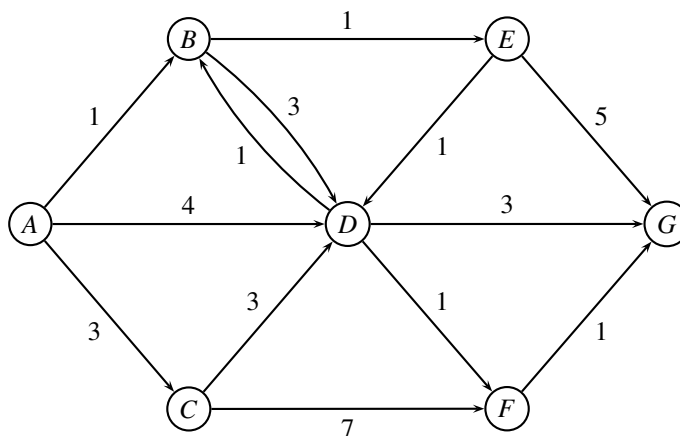
```

xAC          3.000000
xAD          4.000000
xCB          3.000000
xCE          6.000000
xDC          6.000000
xDE          2.000000
xEB          2.000000
All other variables in the range 1-10 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
c1                  -7.000000
c2                   2.000000
c3                  -2.000000
c4                  -4.000000
c7                  -1.000000
c8                  -2.000000
c11                 -1.000000
All other dual prices in the range 1-15 are 0.

```

■

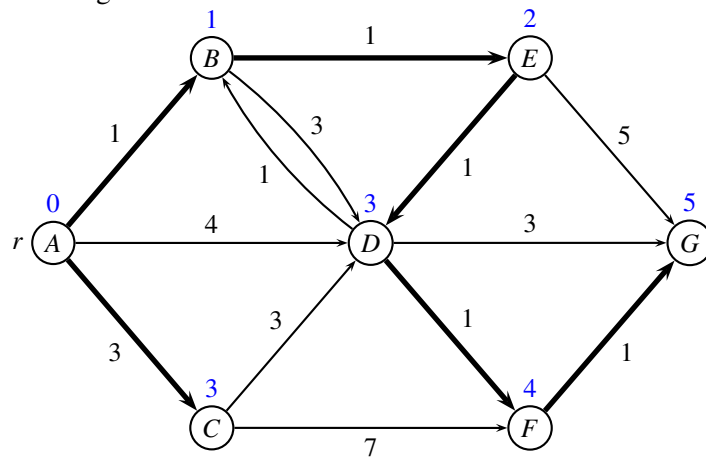
Shortest path



Answer 5 Running Dijkstra we have the following potentials in each iteration:

relaxing	potential						
	A	B	C	D	E	F	G
init	0	∞	∞	∞	∞	∞	∞
A		1	3	4			
B					2		
E				3			7
C						10	
D						4	6
F							5
G							

This results in the following tree



■

Answer 6 The travel times t are

	distance						
	A	B	C	D	E	F	G
		1	1	3	2	4	5

■

Answer 7 We use the label correcting algorithm as follows:

- We start with the following labels:

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5

and we have the queue $Q = \{A, B, C, D, E, F, G\}$.

- Extract A from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1						

and we have the queue $Q = \{B, C, D, E, F, G\}$.

- Extract B from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1						
						4	2						

The label (4,2) for D is dominated, so we remove it. The queue is $Q = \{C, D, E, F, G\}$.

- Extract C from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1						
						6	2						

The label (6,2) for D is dominated, so we remove it. The queue is $Q = \{D, E, F, G\}$.

- Extract D from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
		4	4			4	1			5	2	6	4
		5	2									7	2

The labels (4,4) and (5,2) for B are dominated so we remove them. This means that no labels for B has been changed, and we do not need to add B to the queue. Labels for F and G have been modified, but the nodes are already in the queue. The queue is $Q = \{E, F, G\}$.

- Extract E from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1			5	2	6	4
												7	2
												7	3

The label (7,3) for G is dominated so we remove it. The queue is $Q = \{F, G\}$.

- Extract F from the queue, and update all outgoing edges

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1			5	2	6	4
												7	2
												6	3

The label (6,4) for G is dominated so we remove it. The queue is $Q = \{G\}$.

- Extract G from the queue. No outgoing edges from G .

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	5	5
						4	1			5	2	6	3
										7	2		

The queue is empty, we stop.

We were asked to find the shortest distance to every node using at most 4 time units. Hence the cheapest cost with $t \leq 4$ in each column

A		B		C		D		E		F		G	
w	t	w	t	w	t	w	t	w	t	w	t	w	t
0	0	1	1	3	1	3	3	2	2	4	4	6	3

■

Nurse scheduling

Answer 8 We have the requirements

shift i	1	2	3	4	5	6
time	6:00-10:00	10:00-14:00	14:00-18:00	18:00-22:00	22:00-2:00	2:00-6:00
min nurses d_i	70	80	50	60	40	30

First we notice that we can have the following duties:

duty j	start a_j	end b_j	cost c_j
1	1	2	2
2	2	3	2
3	3	4	2
4	4	5	3
5	5	6	4
6	6	6	2

Writing up the constraints in an IP model, where x_i denotes the number of nurses following duty i , we get

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} \geq \begin{pmatrix} 70 \\ 80 \\ 50 \\ 60 \\ 40 \\ 30 \end{pmatrix}$$

Since the model has consecutive 1's in every column, we can use the modeling technique from Application 9.6 in Ahuja, Orlin, Magnanti. First, we add surplus variables to each row to get equations, and

then we add an extra zero-row.

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 70 \\ 80 \\ 50 \\ 60 \\ 40 \\ 30 \\ 0 \end{pmatrix}$$

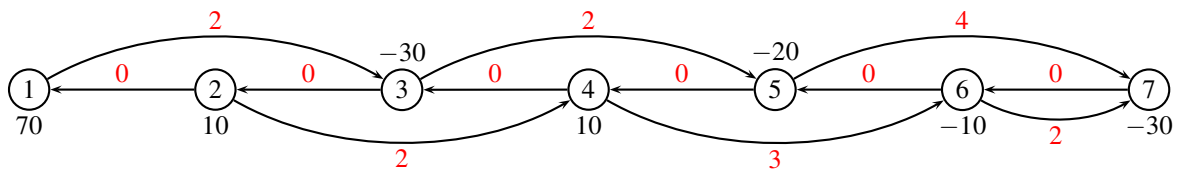
Using row-operations we can get it to the form

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 70 \\ 10 \\ -30 \\ 10 \\ -20 \\ -10 \\ -30 \end{pmatrix}$$

This can be recognised as a min-cost-flow problem.

To be more specific, we construct a node for every shift $1, \dots, 6$ and an additional end node 7. The demand b_i in node i is $-(d_i - d_{i-1})$. (Negative values indicate sinks, positive values sources).

For every duty j we introduce an edge from node a_j to $b_j + 1$ having cost c_j . The flow on this edge indicates how many nurses follow duty j . Moreover, we introduce edges from node $i + 1$ to i having cost 0. These edges allow for "over-covering" shifts, i.e. nurses can end later than we actually need them. The flow on these edges corresponds to the surplus variables y_i . The capacity of all edges is set to ∞ .



■

Answer 9 We get the LP-model

minimize $2x_{13} + 2x_{24} + 2x_{35} + 3x_{46} + 4x_{57} + 2x_{67}$

subject to
 $x_{13} - x_{21} = 70$ \\ flow balance node 1
 $x_{24} + x_{21} - x_{32} = 10$ \\ flow balance node 2
 $x_{35} + x_{32} - x_{43} - x_{13} = -30$ \\ flow balance node 3
 $x_{46} + x_{43} - x_{54} - x_{24} = 10$ \\ flow balance node 4
 $x_{57} + x_{54} - x_{65} - x_{35} = -20$ \\ flow balance node 5
 $x_{67} + x_{65} - x_{76} - x_{46} = -10$ \\ flow balance node 6
 $+x_{76} - x_{57} - x_{67} = -30$ \\ flow balance node 7
end

■

Answer 10 The optimal solution is $x_{13} = 70$, $x_{24} = 10$, $x_{35} = 40$, $x_{57} = 20$, $x_{43} = 20$, $x_{76} = 10$. So,

duty j	start a_j	end b_j	cost c_j	nurses
1	1	2	2	70
2	2	3	2	10
3	3	4	2	40
4	4	5	3	20
5	5	6	4	20
6	6	6	2	10

No "over-covering" is necessary since no "backward" edges are used. The overall cost is $z = 400$.

```
CPLEX> read nurses
Problem 'nurses.lp' read.
Read time =    0.04 sec.
CPLEX> opt
Tried aggregator 1 time.
Aggregator did 1 substitutions.
Reduced LP has 6 rows, 11 columns, and 22 nonzeros.
Presolve time =    0.00 sec.
Initializing dual steep norms . . .

Iteration log . . .
Iteration:    1   Dual objective      =          260.000000

Dual simplex - Optimal:  Objective =  4.000000000000e+02
Solution time =    0.02 sec.  Iterations = 5 (0)

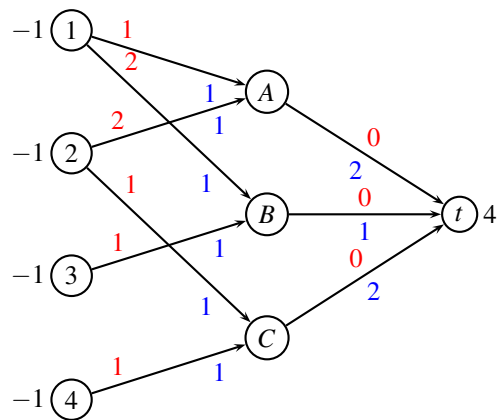
CPLEX> dis sol var -
Variable Name      Solution Value
x13                70.000000
x24                10.000000
x35                40.000000
x46                20.000000
x57                20.000000
x67                10.000000
All other variables in the range 1-12 are 0.
```

■

Student assignment

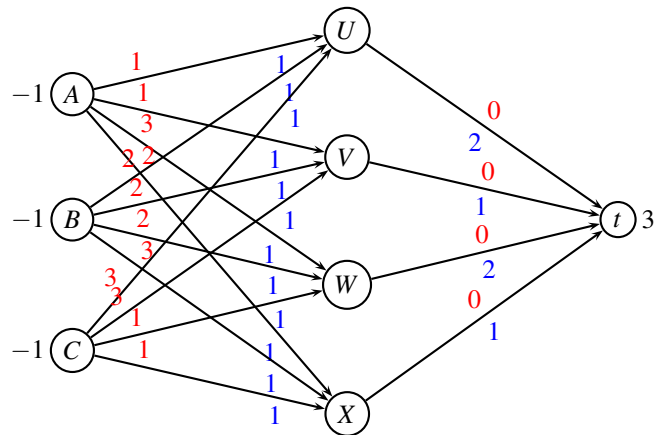
Answer 11 The problem can be solved as two min-cost flow problems. First we assign students to projects. For every student we create a node, and for every project we create a node. The edge between a student node and a project node gets a cost corresponding to the priority, and a capacity equal to 1. An additional sink t is introduced. Every edge from a project node to t gets a capacity corresponding to the capacity of the project, and a cost of 0. Every student node gets demand -1 and t gets demand

equal to the sum of students.



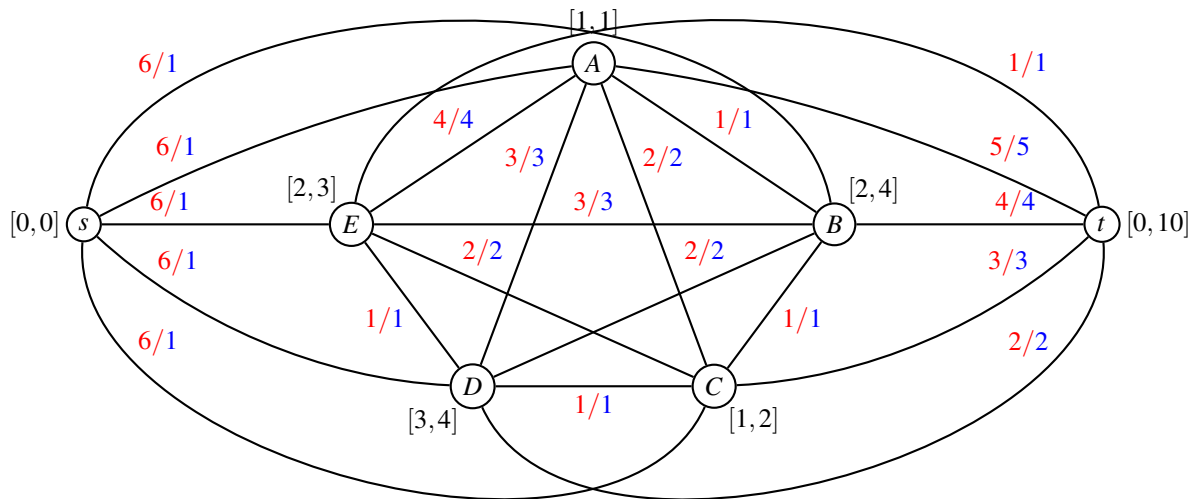
Red values indicate the cost, blue values the capacity of an edge. The node potentials are negative for a source, and positive for a sink.

Next, we assign projects to supervisors. As before we introduce a node for every project and a node for every professor. Edges between student nodes and professor nodes get cost equal to the preference, and capacity equal to 1. An additional sink t is introduced. Every edge from a professor node to t gets a capacity corresponding to the capacity of the professor, and a cost of 0. Every project node gets demand -1 and t gets demand equal to the sum of projects.



■

Vehicle Routing Problem with time constraints



Answer 12 A possible solution is

- Vehicle 1: $sABDt$
- Vehicle 2: $sCEt$

The overall cost is 20 ■

Answer 13 We have the route costs:

route	nodes	cost
1	s,A,B,D,t	11
2	s,B,D,t	10
3	s,C,B,D,t	11
4	s,C,D,t	9
5	s,C,E,t	9
6	s,C,E,D,t	11
7	s,D,t	8
8	s,E,D,t	9
9	s,E,t	7

■

Answer 14 Waiting times are indicated in the below table. Each (wait) indicates one time unit.

route	nodes	wait
1	s,A,B,D,t	0
2	s,(wait),B,D,t	1
3	s,C,B,D,t	0
4	s,C,(wait),D,t	1
5	s,C,E,t	0
6	s,C,E,D,t	0
7	s,(wait),(wait),D,t	2
8	s,(wait),E,D,t	1
9	s,(wait),E,t	1

■

Answer 15 Since every customer must be visited at least once, we use a set-covering formulation. The problem with all 9 routes becomes:

```

minimize
    11x1+10x2+11x3+ 9x4+ 9x5+11x6+ 8x7+ 9x8+ 7x9

subject to
c1:  +x1                                     >= 1
c2:  +x1 +x2 +x3                             >= 1
c3:           +x3 +x4 +x5 +x6                 >= 1
c4:  +x1 +x2 +x3 +x4             +x6 +x7 +x8   >= 1
c5:           +x5 +x6             +x8 +x9 >= 1
end

CPLEX> read vrp
Problem 'vrp.lp' read.
Read time =    0.05 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 8 rows and 11 columns.
Reduced LP has 2 rows, 3 columns, and 4 nonzeros.
Presolve time =    0.00 sec.

Iteration log . . .
Iteration:    1    Dual objective      =          20.000000

Dual simplex - Optimal:  Objective =  2.0000000000e+01
Solution time =    0.02 sec.  Iterations = 2 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
x5                  1.000000
All other variables in the range 1-14 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
c1                   11.000000
c3                   9.000000

```

All other dual prices in the range 1-10 are 0.

The IP-optimal solution gives the solution value $z_{IP} = 20$. Routes 1 and 5 are chosen. ■

Answer 16 We run column generation as follows:

- **Initial Solution.** We can choose the trivial routes:

tour	cost
0,A,6	11
0,B,6	10
0,C,6	9
0,D,6	8
0,E,6	7

which gives a solution of 45.

- **Column generation.** We start with formulation

$$\begin{array}{llll}
 \min & 11x_a + 10x_b + 9x_c + 8x_d + 7x_e \\
 \text{s.t.} & x_a & & \geq 1 \\
 & & x_b & \geq 1 \\
 & & & x_c \geq 1 \\
 & & & & x_d \geq 1 \\
 & & & & & x_e \geq 1
 \end{array}$$

the dual variables corresponding to the constraints are

$$y_a = 11, y_b = 10, y_c = 9, y_d = 8, y_e = 7$$

The reduced cost of a route is now the “original” cost of the edges minus the dual variables corresponding to the customers visited.

route	nodes	original cost	dual	reduced cost
1	s,A,B,D,t	11	11+10+8	-18
2	s,B,D,t	10	10+8	-8
3	s,C,B,D,t	11	10+9+8	-16
4	s,C,D,t	9	9+8	-8
5	s,C,E,t	9	9+7	-7
6	s,C,E,D,t	11	9+8+7	-13
7	s,D,t	8	8	0
8	s,E,D,t	9	8+7	-6
9	s,E,t	7	7	0

The most beneficial route is route 1, i.e. (s, A, B, D, t) having the cost $11 - (11 + 10 + 8) = -18$. Since the reduced cost of the new column is negative, we add it to the problem, getting:

$$\begin{array}{llll}
 \min & 11x_a + 10x_b + 9x_c + 8x_d + 7x_e + 11x_1 \\
 \text{s.t.} & x_a & & +x_1 \geq 1 \\
 & & x_b & +x_1 \geq 1 \\
 & & & x_c \geq 1 \\
 & & & & x_e +x_1 \geq 1 \\
 & & & & & x_e \geq 1
 \end{array}$$

the dual variables are

$$y_a = 11, y_b = 0, y_c = 9, y_d = 0, y_e = 7$$

The reduced costs are

route	nodes	original cost	dual	reduced cost
1	s,A,B,D,t	11	11+0+0	0
2	s,B,D,t	10	0+0	10
3	s,C,B,D,t	11	0+9+0	2
4	s,C,D,t	9	9+0	0
5	s,C,E,t	9	9+7	-7
6	s,C,E,D,t	11	9+0+7	-5
7	s,D,t	8	8	0
8	s,E,D,t	9	0+7	2
9	s,E,t	7	7	0

The most beneficial route is now route 5 i.e. (s, C, E, t) having the reduced cost $9 - (9 + 7) = -7$, which is added to the problem:

$$\begin{array}{llllll}
 \min & 11x_a + 10x_b + 9x_c + 8x_d + 7x_e + 11x_1 + 9x_5 \\
 \text{s.t.} & x_a & & & & +x_1 & \geq 1 \\
 & & x_b & & & +x_1 & \geq 1 \\
 & & & x_c & & +x_5 & \geq 1 \\
 & & & & x_d & +x_1 & \geq 1 \\
 & & & & & x_e & +x_5 \geq 1
 \end{array}$$

the dual variables are

$$y_a = 11, y_b = 0, y_c = 9, y_d = 0, y_e = 0$$

Reduced costs are

route	nodes	original cost	dual	reduced cost
1	s,A,B,D,t	11	11+0+0	0
2	s,B,D,t	10	0+0	10
3	s,C,B,D,t	11	0+9+0	2
4	s,C,D,t	9	9+0	0
5	s,C,E,t	9	9+0	0
6	s,C,E,D,t	11	9+0+0	2
7	s,D,t	8	8	0
8	s,E,D,t	9	0+0	9
9	s,E,t	7	0	0

Now, all routes have reduced costs ≥ 0 , so we are done. The bound found by column generation is $z_{LP} = 20$.

■

Answer 17 Compare bounds. We have $z_{IP} = z_{LP}$ so the bounds are tight. Actually the LP-solution returned an integer solution, and hence solved the problem. ■

Total model

```

minimize
    11x1+10x2+11x3+ 9x4+ 9x5+11x6+ 8x7+ 9x8+ 7x9+11xa+10xb+ 9xc+ 8xd+ 7xe

subject to
c1:  +x1                                     +xa                                     >= 1
c2:  +x1  +x2  +x3                                     +xb                                     >= 1
c3:                                     +x3  +x4  +x5  +x6                                     +xc                                     >= 1
c4:  +x1  +x2  +x3  +x4                                     +x6  +x7  +x8                                     +xd                                     >= 1
c5:                                     +x5  +x6                                     +x8  +x9                                     +xe >= 1

\\ r1: x1 <= 0
\\ r2: x2 <= 0
\\ r3: x3 <= 0
\\ r4: x4 <= 0
\\ r5: x5 <= 0
\\ r6: x6 <= 0
\\ r7: x7 <= 0
\\ r8: x8 <= 0
\\ r9: x9 <= 0

s1: xa <= 0
s2: xb <= 0
s3: xc <= 0
s4: xd <= 0
s5: xe <= 0

end

CPLEX> read vrp
Problem 'vrp.lp' read.
Read time = 0.05 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 8 rows and 11 columns.
Reduced LP has 2 rows, 3 columns, and 4 nonzeros.
Presolve time = 0.00 sec.

Iteration log . . .
Iteration: 1 Dual objective = 20.000000

Dual simplex - Optimal: Objective = 2.0000000000e+01
Solution time = 0.02 sec. Iterations = 2 (0)

CPLEX> dis sol var -
Variable Name          Solution Value
x1                     1.000000
x5                     1.000000
All other variables in the range 1-14 are 0.
CPLEX> dis sol dua -
Constraint Name          Dual Price
c1                     11.000000
c3                      9.000000
All other dual prices in the range 1-10 are 0.

```

Iteration 0

```
minimize
    11x1+10x2+11x3+ 9x4+ 9x5+11x6+ 8x7+ 9x8+ 7x9+11xa+10xb+ 9xc+ 8xd+ 7xe

subject to
c1:  +x1                                +xa                                >= 1
c2:  +x1  +x2  +x3                                +xb                                >= 1
c3:                                +x3  +x4  +x5  +x6                                +xc                                >= 1
c4:  +x1  +x2  +x3  +x4                                +x6  +x7  +x8                                +xd                                >= 1
c5:                                +x5  +x6                                +x8  +x9                                +xe >= 1

r1: x1 <= 0
r2: x2 <= 0
r3: x3 <= 0
r4: x4 <= 0
r5: x5 <= 0
r6: x6 <= 0
r7: x7 <= 0
r8: x8 <= 0
r9: x9 <= 0

end

CPLEX> read vrp0.lp
Problem 'vrp0.lp' read.
Read time =    0.03 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 14 rows and 14 columns.
All rows and columns eliminated.
Presolve time =    0.00 sec.

Dual simplex - Optimal: Objective = 4.5000000000e+01
Solution time =    0.00 sec. Iterations = 0 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
xa                  1.000000
xb                  1.000000
xc                  1.000000
xd                  1.000000
xe                  1.000000
All other variables in the range 1-14 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
c1                   11.000000
c2                   10.000000
c3                    9.000000
c4                    8.000000
c5                    7.000000
r1                   -18.000000
r2                    -8.000000
r3                   -16.000000
r4                    -8.000000
r5                    -7.000000
r6                   -13.000000
r8                    -6.000000
All other dual prices in the range 1-14 are 0.
```

Iteration 1

```
minimize
    11x1+10x2+11x3+ 9x4+ 9x5+11x6+ 8x7+ 9x8+ 7x9+11xa+10xb+ 9xc+ 8xd+ 7xe

subject to
c1:  +x1                                     +xa                                     >= 1
c2:  +x1  +x2  +x3                                     +xb                                     >= 1
c3:                                     +x3  +x4  +x5  +x6                                     +xc                                     >= 1
c4:  +x1  +x2  +x3  +x4                                     +x6  +x7  +x8                                     +xd                                     >= 1
c5:                                     +x5  +x6                                     +x8  +x9                                     +xe >= 1

\\ r1: x1 <= 0
r2: x2 <= 0
r3: x3 <= 0
r4: x4 <= 0
r5: x5 <= 0
r6: x6 <= 0
r7: x7 <= 0
r8: x8 <= 0
r9: x9 <= 0

end

CPLEX> read vrp1
Problem 'vrp1.lp' read.
Read time = 0.03 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 10 rows and 10 columns.
Reduced LP has 3 rows, 4 columns, and 6 nonzeros.
Presolve time = 0.00 sec.

Iteration log . . .
Iteration: 1 Dual objective = 27.000000

Dual simplex - Optimal: Objective = 2.7000000000e+01
Solution time = 0.00 sec. Iterations = 2 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  1.000000
xc                  1.000000
xe                  1.000000
All other variables in the range 1-14 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
c1                   11.000000
c3                   9.000000
c5                   7.000000
r5                   -7.000000
r6                   -5.000000
All other dual prices in the range 1-13 are 0.
```


Iteration 2

```
minimize
    11x1+10x2+11x3+ 9x4+ 9x5+11x6+ 8x7+ 9x8+ 7x9+11xa+10xb+ 9xc+ 8xd+ 7xe

subject to
c1:  +x1                                +xa                                >= 1
c2:  +x1  +x2  +x3                                +xb                                >= 1
c3:                                +x3  +x4  +x5  +x6                                +xc                                >= 1
c4:  +x1  +x2  +x3  +x4                                +x6  +x7  +x8                                +xd                                >= 1
c5:                                +x5  +x6                                +x8  +x9                                +xe >= 1

\\ r1: x1 <= 0
r2: x2 <= 0
r3: x3 <= 0
r4: x4 <= 0
\\ r5: x5 <= 0
r6: x6 <= 0
r7: x7 <= 0
r8: x8 <= 0
r9: x9 <= 0

end

CPLEX> read vrp2
Problem 'vrp2.lp' read.
Read time =    0.02 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 7 rows and 7 columns.
Reduced LP has 5 rows, 7 columns, and 10 nonzeros.
Presolve time =    0.00 sec.

Iteration log . . .
Iteration:    1    Dual objective      =          11.000000

Dual simplex - Optimal:  Objective =  2.0000000000e+01
Solution time =    0.00 sec.  Iterations = 4 (0)

CPLEX> dis sol var -
Variable Name          Solution Value
x1                      1.000000
x5                      1.000000
All other variables in the range 1-14 are 0.
CPLEX> dis sol dua -
Constraint Name          Dual Price
c1                      11.000000
c3                      9.000000
All other dual prices in the range 1-12 are 0.
```

Take-home exam: December 18, 2014.

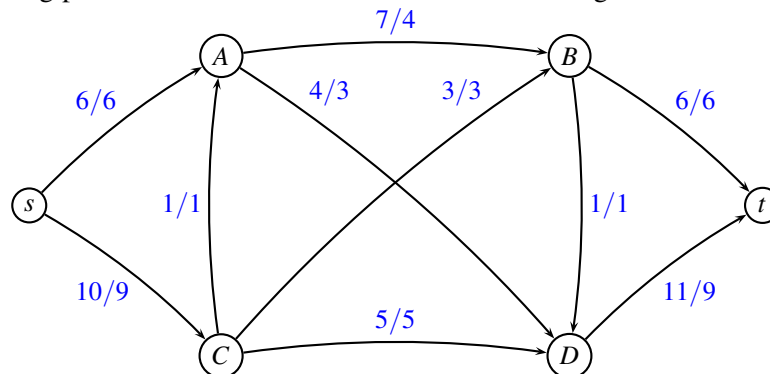
Course name: Network Optimization

Course no: 42115

Answer 1 Augmenting paths

iteration	augmenting path	flow
1	sABt	6
2	sCDt	5
3	sCBADt	3
4	sCABDt	1

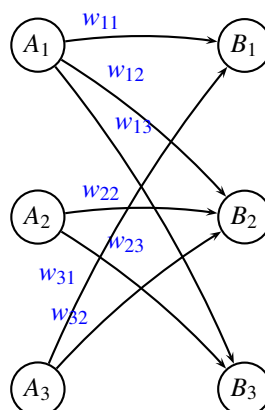
No more augmenting paths can be found. This leads to the following flow:



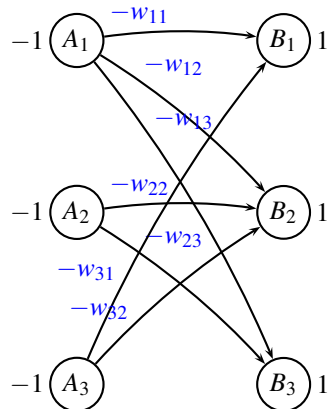
with objective value $z = 15$. ■

Answer 2 A minimum cut is found as $S = \{s, C\}$ and $T = \{A, B, D, t\}$. The capacity of the cut is $6 + 1 + 3 + 5 = 15$ which corresponds to the flow found above $z = 15$. ■

Answer 3 We have a bipartite graph, where the nodes can be split into two sets A and B , so that all edges (i, j) have $i \in A$ and $j \in B$. We may assume that $|A| = |B|$ since otherwise a perfect matching does not exist.



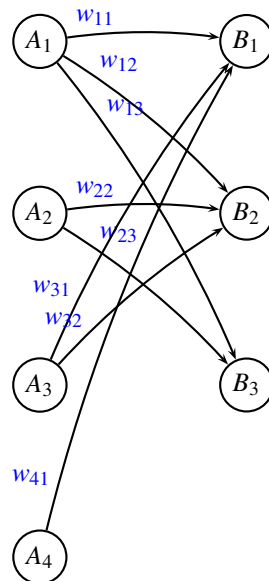
In order to transform the problem to a minimum cost flow problem, we set the demand to -1 for nodes in A , and to $+1$ for nodes in B . The weights are set to $-w_{ij}$ and the capacities are set to $u_{ij} = 1$.



A solution to the minimum cost flow problem will exactly find the perfect matching of largest weight.

■

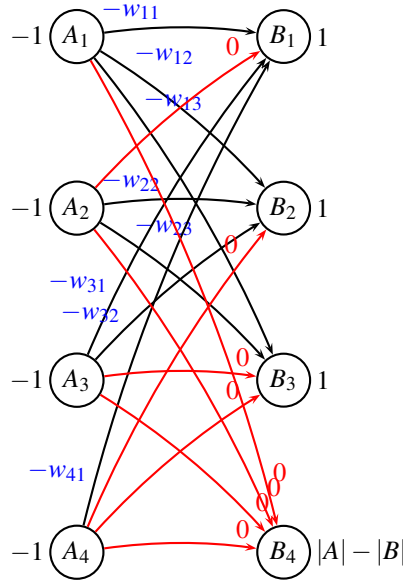
Answer 4 As before we have a bipartite graph, where the nodes can be split into two sets A and B , so that all edges (i, j) have $i \in A$ and $j \in B$. For symmetry reasons we may assume that $|A| \geq |B|$.



In order to transform the problem to a minimum cost flow problem, we set the demand to -1 for nodes in A , and to $+1$ for nodes in B . The weights are set to $-w_{ij}$. We introduce an extra node to the set B having demand $|A| - |B|$.

Finally, if two nodes i, j are *not* connected, we introduce a new edge (i, j) with weight $w_{ij} = 0$.

The capacities of all edges are set to $u_{ij} = 1$.



A solution to the minimum cost flow problem will exactly find a matching of largest weight. So, the problem can be solved in one run of a minimum cost flow algorithm. ■

Answer 5 Only model 5 can handle capacities on land. This is due to the fact that All transshipments between arrival of vessel H_i and vessel H_{i+1} share the same edge.

Model 3 and 4 have edges for land transportation, but we have direct edges between all pairs of vessels, so it is not possible to model a shared capacity.

Model 1 and 2 do not have any edges for land transportation. ■

Answer 6 In order to model tax costs, we need to have direct edges between all pairs of vessels.

Model 3 and 4 have direct edges between all pairs of vessels, so it is possible to model transshipment taxes.

Model 1, 2 and 5 do not have direct edges between vessels, so it is impossible to distinguish cargo from different vessels arriving at the port. ■

Answer 7 The reduced cost of a route r is calculated as

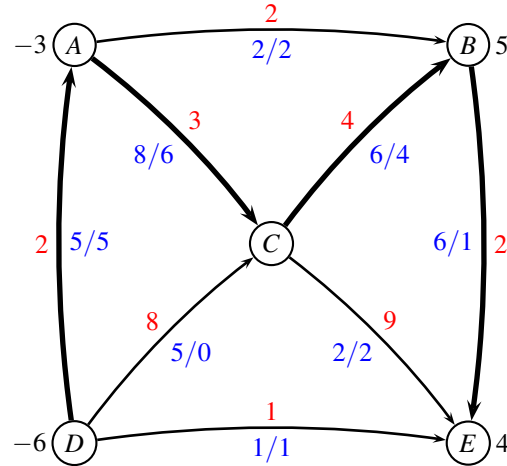
$$\bar{c}_r^k = d_k \sum_{(i,j) \in r} (c_{ij} - y_{ij}) - \bar{y}_k$$

Having the dual variables $y_{AB} = -2$, and $\bar{y}_1 = 14$, $\bar{y}_2 = 14$, $\bar{y}_3 = 6$, we get the following table:

number	commodity k	route r	d_k	$\sum c_{ij} - y_{ij}$	$\bar{y}_k$	reduced cost $\bar{c}_r^k$
1	1	E→A→B→D	2	7	14	0
2	1	E→D	2	7	14	0
3	2	D→C	2	8	14	2
4	2	D→E→A→B→C	2	7	14	0
5	3	E→A→B→C	1	6	6	0
6	3	E→A→B→D→C	1	15	6	9
7	3	E→A→D→B→C	1	5	6	-1
8	3	E→A→D→C	1	11	6	5
9	3	E→D→B→C	1	9	6	3
10	3	E→D→C	1	15	6	9

The most promising route to add to the model is route number 7 with reduced cost -1. ■

Answer 8 Every partitioning of edges in T, U, L defines a unique flow as shown in the notes. In the specific case we have



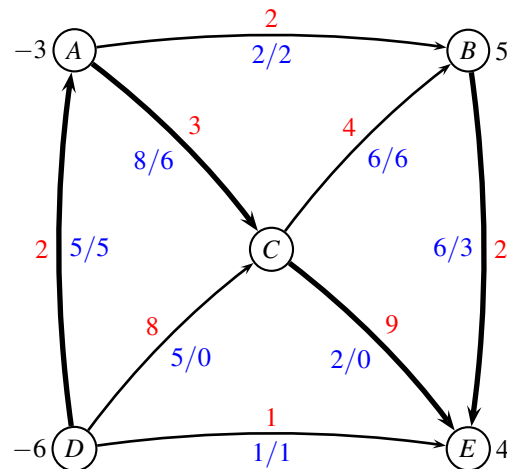
Notice that edges in T are allowed to be at the lower or upper bound as well as any value inbetween. The solution value is $z = 2 \cdot 2 + 6 \cdot 3 + 4 \cdot 4 + 5 \cdot 2 + 1 \cdot 2 + 2 \cdot 9 + 1 \cdot 1 = 69$ ■

Answer 9 Choosing D as root node, we find potentials (distances) $y_D = 0$, $y_A = 2$, $y_C = 5$, $y_B = 9$, $y_E = 11$. The reduced costs are

$$\begin{aligned}\bar{w}_{AB} &= 2 + 2 - 9 = -5 && U \text{ edge} \\ \bar{w}_{CE} &= 9 + 5 - 11 = 3 && U \text{ edge} \\ \bar{w}_{DC} &= 8 + 0 - 5 = 3 && L \text{ edge} \\ \bar{w}_{DE} &= 1 + 0 - 11 = -10 && U \text{ edge}\end{aligned}$$

Since edge (C, E) is a U edge and it has positive reduced cost, we can improve the solution. The edge (C, E) imposes the cycle E, C, B, E (we are going in the backward direction of edge (C, E) since it is a U edge).

The capacity we can send along the cycle is $\theta = \min\{2, 2, 5\} = 2$. Sending two items along cycle E, C, B means that edge (C, E) enters T while (C, B) enters U .

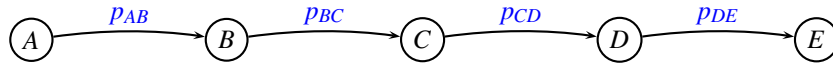


Choosing D as root node, we find potentials (distances) $y_D = 0$, $y_A = 2$, $y_C = 5$, $y_E = 14$, $y_B = 12$. The reduced costs are

$$\begin{aligned}
\bar{w}_{AB} &= 2 + 2 - 12 = -8 & U \text{ edge} \\
\bar{w}_{CB} &= 4 + 5 - 12 = -3 & U \text{ edge} \\
\bar{w}_{DC} &= 8 + 0 - 5 = 3 & L \text{ edge} \\
\bar{w}_{DE} &= 1 + 0 - 14 = -13 & U \text{ edge}
\end{aligned}$$

Since all U edges have negative reduced cost, and all L edges have positive reduced cost, we have found the optimal solution. The solution value is $z = 2 \cdot 2 + 6 \cdot 3 + 6 \cdot 4 + 5 \cdot 2 + 3 \cdot 2 + 1 \cdot 1 = 63$ ■

Answer 10 As an example consider a path $ABCDE$.



The probability for *not* being lost on the path is $(1 - p_{AB})(1 - p_{BC})(1 - p_{CD})(1 - p_{DE})$.

In general, for a path P going from s to t , we want to minimize the following expression

$$\max \prod_{(i,j) \in P} (1 - p_{ij})$$

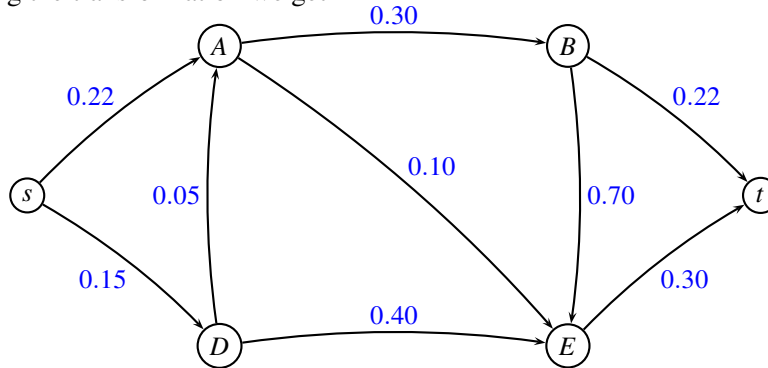
Taking the logarithm to this expression we can use the objective

$$\max \log \prod_{(i,j) \in P} (1 - p_{ij}) = \max \sum_{(i,j) \in P} \log(1 - p_{ij}) = \min \sum_{(i,j) \in P} -\log(1 - p_{ij})$$

So, using $w_{ij} = -\log(1 - p_{ij})$ as edge weights, the problem can be solved as a shortest path problem.

Notice that we can use the logarithm in the objective function $f(x)$, since the logarithm is a strictly growing function. This means that if $f(x_1) < f(x_2)$ then also $\log f(x_1) < \log f(x_2)$, so we will find the same minimum x , although the objective function $\log f(x)$ in the minimum will be different. ■

Answer 11 Using the transformation we get



■

Answer 12 Since $(1 - p_{ij}) \leq 1$ we have $\log(1 - p_{ij}) \leq 0$ and hence $w_{ij} = -\log(1 - p_{ij}) \geq 0$. We can therefore use Dijkstra's algorithm.

The following table shows the iterations:

considered node	potential					
	s	A	B	D	E	t
init	0	∞	∞	∞	∞	∞
s		0.22		0.15		
D		0.20			0.55	
A			0.50		0.30	
E						0.60
B						
t						

The shortest s - t path is therefore $sDAEt$ with cost 0.60.

To find the original probability of losing a message along the path $sDAEt$, we use the original probabilities. The probability for *not* losing a message is:

$$(1 - p_{sD})(1 - p_{DA})(1 - p_{AE})(1 - p_{Et}) = (1 - 0.3)(1 - 0.1)(1 - 0.2)(1 - 0.5) = 0.7 \cdot 0.9 \cdot 0.8 \cdot 0.5 = 0.252$$

So the probability for losing a message is around 75%. ■

CPLEX model for question 7:

```

minimize
+10x1+14x2+16x3+10x4+4x5+13x6+5x7+11x8+9x9+15x10
subject to
AB: +2x1          +2x4+1x5+1x6          <= 4
AD:              +1x7+1x8              <= 1
BC:              +2x4+1x5      +1x7      +1x9      <= 5
BD: +2x1          +1x6              <= 4
DB:              +1x7      +1x9      <= 1
DC:              +2x3          +1x6      +1x8      +1x10 <= 3
DE:              +2x4              <= 3
EA: +2x1          +2x4+1x5+1x6+1x7+1x8      <= 5
ED:      +2x2              +1x9+1x10 <= 3

K1: +x1+x2 >= 1
K2: +x3+x4 >= 1
K3: +x5+x6+x7+x8+x9+x10 >= 1

\\ R1: x1 <= 0
\\ R2: x2 <= 0
R3: x3 <= 0
\\ R4: x4 <= 0
\\ R5: x5 <= 0
R6: x6 <= 0
R7: x7 <= 0
R8: x8 <= 0
R9: x9 <= 0
R10: x10 <= 0
end

CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 18 rows and 10 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec.

Dual simplex - Optimal: Objective = 2.600000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  0.500000
x2                  0.500000
x4                  1.000000
x5                  1.000000
All other variables in the range 1-10 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
AB                   -2.000000
K1                   14.000000
K2                   14.000000
K3                   6.000000
R7                   -1.000000
All other dual prices in the range 1-18 are 0.

```

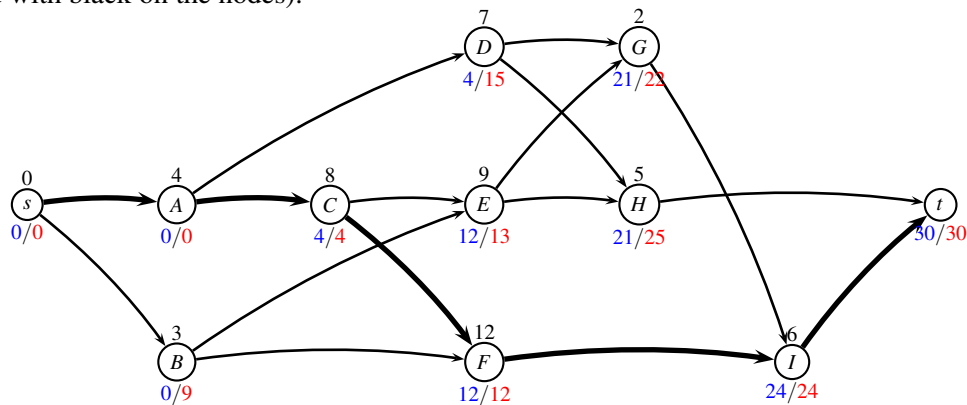

Answer, December 17, 2015.

Course name: Network Optimization

Course no: 42115

Critical path problem

Answer 1 We insert a new node s and connect it to the first two activities A, B . We insert a new node t and connect it to the last two activities H, I . This gives the following graph (duration of activities is indicated with black on the nodes):



Now we solve the problem

- *Forward procedure:* Starting at time zero, calculate the earliest time U_j each job j can be started $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$. (marked with blue, above)
- *Makespan:* $T = U_t$.
- *Backward procedure:* Starting at time equal to the makespan, calculate the latest time V_j each job j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$ (marked with red, above)

The critical path schedule solution for this project is given below:

Activity	Duration	Earliest start time U_j	Latest start time V_j
s	0	0	0
A	4	0	0
B	3	0	9
C	8	4	4
D	7	4	15
E	9	12	13
F	12	12	12
G	2	21	22
H	5	21	25
I	6	24	24
t	0	30	30

The makespan is $T = 30$. ■

Answer 2 The critical path is found for activities where earliest start time U_j is equal to the latest start time V_j in the above table. This means projects A, C, F, I . The critical path is marked with bold edges in the above graph. ■

Fixed charge transportation problem

Answer 3 We have the following possible patterns

	S_1			S_2		S_3		
T_1	3	2	1	1	0	2	1	0
T_2	0	1	2	0	1	0	1	2
transp. cost	12	11	10	5	2	14	8	2
fixed charges	5	10	10	5	5	5	10	5
sum of costs	17	21	20	10	7	19	18	7

This leads to the Dantzig-Wolfe decomposed model:

```

minimize
17p11+21p12+20p13+10p21+ 7p22+19p31+18p32+ 7p33
subject to
3p11+ 2p12+ 1p13+ 1p21+ 0p22+ 2p31+ 1p32+ 0p33 = 4    (u1)
0p11+ 1p12+ 2p13+ 0p21+ 1p22+ 0p31+ 1p32+ 2p33 = 2    (u2)

p11+  p12+  p13                                     = 1    (v1)
          p21+  p22                                     = 1    (v2)
                        p31+  p32+  p33 = 1    (v3)

```

```

binary p11 p12 p13 p21 p22 p31 p32 p33
end

```

Note: In principle, one can add a pattern p_{14} for S_1 transporting $(0,3)$ but this pattern will never be used. However it does not make any harm. ■

Answer 4 The greedy solution uses patterns $p_{13} = (1,2), p_{21} = (1,0), p_{31} = (2,0)$. ■

Answer 5 Let a_{ip} be the coefficients in the two first lines of the pattern formulation, and b_{ip} be the coefficients in the last three lines. Reduced cost for pattern p is then calculated as

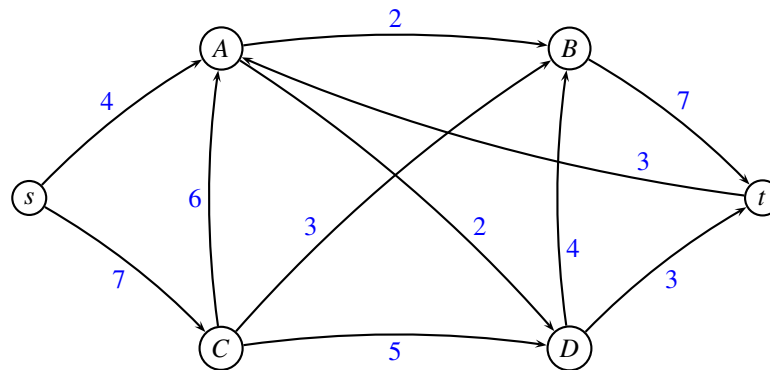
$$\bar{c}_p = c_p - \sum_i a_{pi} u_i - \sum_i b_{pi} v_i$$

where we have $u_1 = 0, u_2 = 10, v_1 = 0, v_2 = 10, v_3 = 19$. This gives the following reduced costs (we do not need to calculate reduced costs of paths which already are present in the master problem)

pattern p	c_p	$\sum_i a_{pi} u_i$	$\sum_i b_{pi} v_i$	$\bar{c}_p$
p_{11}	17	0	0	17
p_{12}	21	10	0	11
p_{22}	7	10	10	-13
p_{32}	18	10	19	-11
p_{33}	7	20	19	-32

Pattern p_{33} has the most negative reduced cost. Notice that in pattern p_{33} the sum $\sum_i a_{pi}u_i = 20$ since we are sending two units. ■

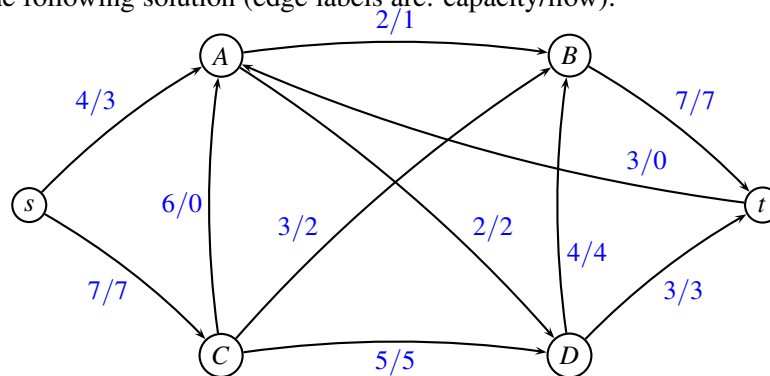
Maximum flow



Answer 6 We use the paths:

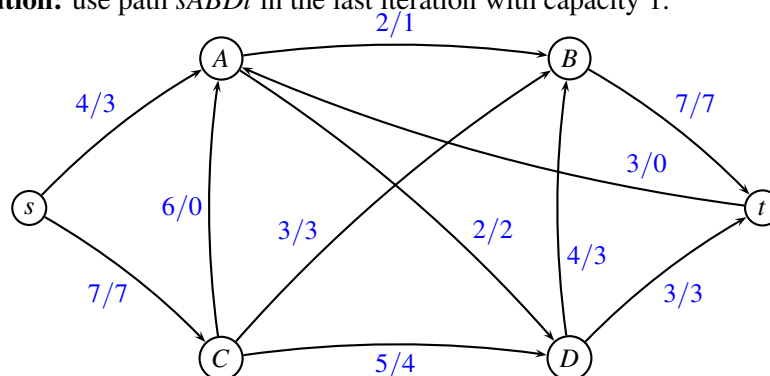
path	capacity
sCDBt	4
sCBt	3
sADt	2
sABCDt	1

This results in the following solution (edge labels are: capacity/flow):



The maximum flow is 10.

Alternative solution: use path $sABDt$ in the last iteration with capacity 1.



■

Answer 7 A minimum cut is $\{s, A, B, C, D\}$ and $\{t\}$. The capacity of the cut is $7 + 3 = 10$. This corresponds to the solution found above. ■

Minimum cost flow problem

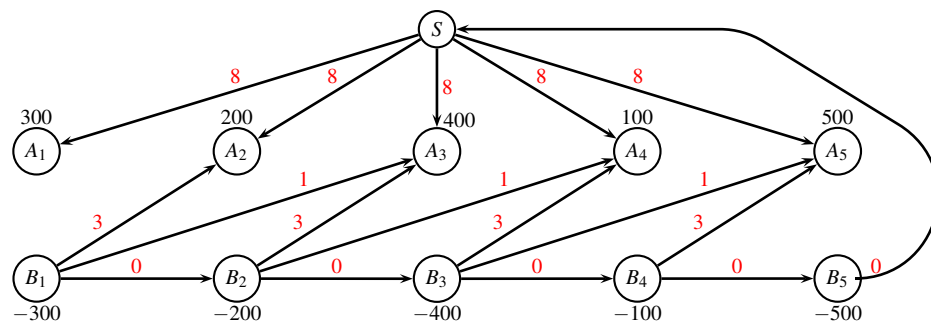
Answer 8 In order to formulate the problem as a *minimum cost flow problem*, we introduce two nodes for each day. Node A_i represents the start of day i , and B_i represents the end of the day. If the demand of napkins at day i is d_i , then node A_i has a demand d_i and node B_i has demand $-d_i$.

At the end of day i we can wash napkins in two possible ways. At cost 3 they can be ready at the beginning of day $i + 1$, or at cost 1 they can be ready at day $i + 2$. This means that each node B_i is connected to node A_{i+1} at cost 3, and to node A_{i+2} at cost 1. The capacity of the edges is infinite.

We can also buy napkins every day at cost 8. We therefore introduce a source s representing buying new napkins. Every node A_i is connected with an edge from s at a cost of 8 with infinite capacity.

Finally, we need to get rid of all napkins not washed. This can e.g. be modeled by connecting node B_i to node B_{i+1} at cost 0, and from node B_5 to node s at cost 0. The edge capacities are infinite.

This gives the following graph formulation:



All edge capacities are infinite.

Alternative solution: Add a terminal node t for all napkins not washed. Connect all B_i nodes to t at cost 0. Give s a demand of $-\sum d_i = -1500$ and t a demand of $\sum d_i = 1500$. Finally, add an extra edge from s to t having cost 0. In this way a sufficient amount of new napkins is available at s , and when napkins are not used any more they disappear at t . Excess napkins which are not used in the catering are sent directly from s to t at cost 0. ■

Arbitrage opportunity

	USD	EUR	DKK	CHF	GBP
USD	1.00	.88	6.54	.96	.65
EUR	1.14	1.00	7.46	1.09	.74
DKK	.15	.13	1.00	.15	.10
CHF	1.05	.92	6.84	1.00	.68
GBP	1.54	1.35	10.05	1.47	1.00

Answer 9 The problem can be formulated by use of a complete directed graph, where each node corresponds to a currency, and edge weights correspond to the exchange rate. We are looking for a cycle where the product of the edges is larger than 1.00.

To solve this problem we first take the logarithm to the exchange rates, and then we multiply by -1 to change maximization to minimization. In this way the problem becomes to find a *negative cost cycle* in the graph. This can be done by use of *Bellman-Ford*.

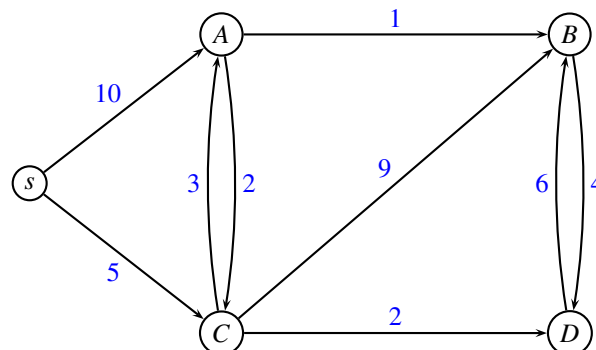
Note: It is not possible to solve the above problem as a *shortest path problem* since if we have a negative cost cycle, the distances are not welldefined, and the algorithm may not terminate.

Note: It should not be necessary to add a source s and a sink t to the graph. Indeed, if all nodes (currencies) are connected to the source s , and also all nodes (currencies) are connected to the sink t then you may risk that you start with one currency and end up with a different currency! It is always easy to exchange e.g. GBP to DKK so that you get more units, but the overall value will not necessarily be better. ■

The following table illustrates the transformation. Each entry is the $-\log(e_{ij})$ of the original exchange rate e_{ij} . Each currency will be a node, and the edge weights are given in the table.

	USD	EUR	DKK	CHF	GBP
USD	0	0,05558	-0,8156	0,0177	0,1871
EUR	-0,0569	0	-0,8727	-0,0374	0,1308
DKK	0,8239	0,886	0	0,8239	1
CHF	-0,0212	0,0362	-0,835	0	0,167
GBP	-0,1875	-0,1303	-1,002	-0,167	0

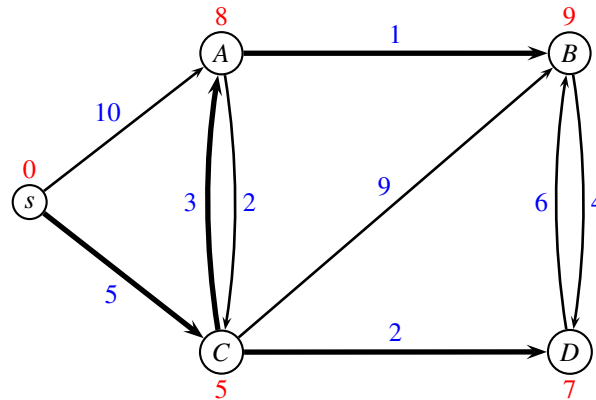
Shortest path



Answer 10 The following table states for each iteration which node is considered, and what the so far best distances are to each node. When a node has been considered, the distance is not changed any more.

considered node	distance				
	s	A	B	C	D
init	0	∞	∞	∞	∞
s		10		5	
C		8	14		7
D			13		
A			9		
B					

The shortest path tree is marked with fat edges below (shortest distance marked with red):

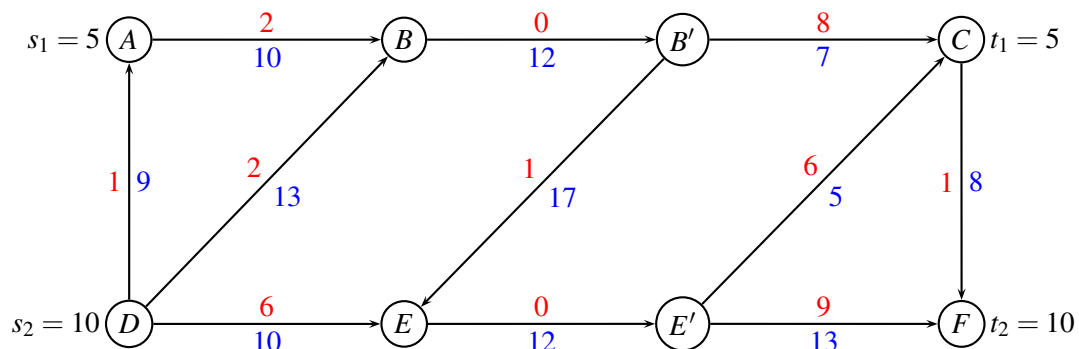


■

Multi-commodity Flow

Answer 11 Every node v with a capacity c_v is split into two nodes v, v' . All ingoing edges are connected to v while all outgoing edges are connected to v' . A new edge (v, v') is added with capacity c_v and cost 0.

This gives us the following standard multi-commodity flow problem



■

Answer 12 If all demands are to be transported unsplittable, we notice that commodity 1 can either use path ABC or path $ABEC$.

Due to the capacity constraints in nodes B and E commodity 1 and commodity 2 cannot pass through nodes B and E at the same time. Therefore, if commodity 1 chooses path $ABEC$ it will not be possible to send commodity 2 through the network. We can therefore conclude that commodity 1 will use path ABC .

Now, commodity 2 can only use path DEF since no other paths have sufficient capacity.

This gives the solution: Commodity 1 use path ABC and commodity 2 use path DEF . The total cost is $5 \cdot (2 + 8) + 10 \cdot (6 + 9) = 50 + 150 = 200$. ■

Answer, December 19, 2016.

Course name: Network Optimization

Course no: 42115

Shortest path problem

Answer 1 Since we have negative edge weights, we cannot use Dijkstra. Instead we can use Bellman-Ford. However, since there are no cycles in the graph, the fastest method is to order the nodes in topological order, and use dynamic programming. ■

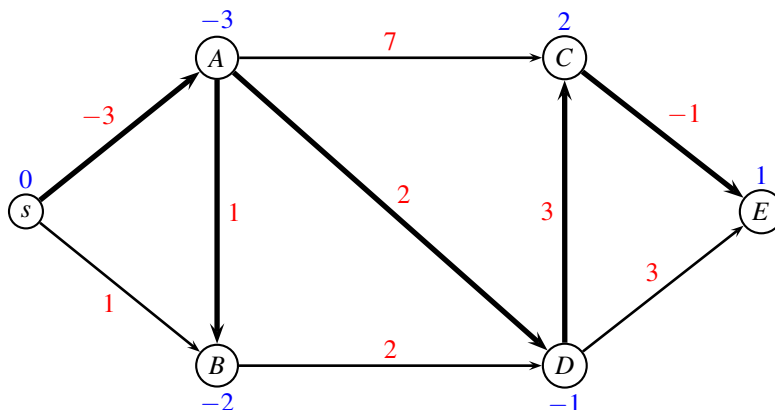
Answer 2 We start by sorting the nodes in topological order A, B, D, C, E . We now run dynamic programming using recursion

$$d(j) = \min_{(i,j) \in E} d(i) + w_{ij}$$

where w_{ij} are the edge weights. This gives

$$\begin{aligned} d(s) &= 0 \\ d(A) &= \min\{0 - 3\} = -3 \\ d(B) &= \min\{0 + 1, -3 + 1\} = -2 \\ d(D) &= \min\{-3 + 2, -2 + 2\} = -1 \\ d(C) &= \min\{-3 + 7, -1 + 3\} = 2 \\ d(E) &= \min\{2 - 1, -1 + 3\} = 1 \end{aligned}$$

The solution becomes:



The bold edges mark the shortest path tree.

Alternative solution

Running Bellman-Ford takes longer time, but is a correct approach. We have the potentials:

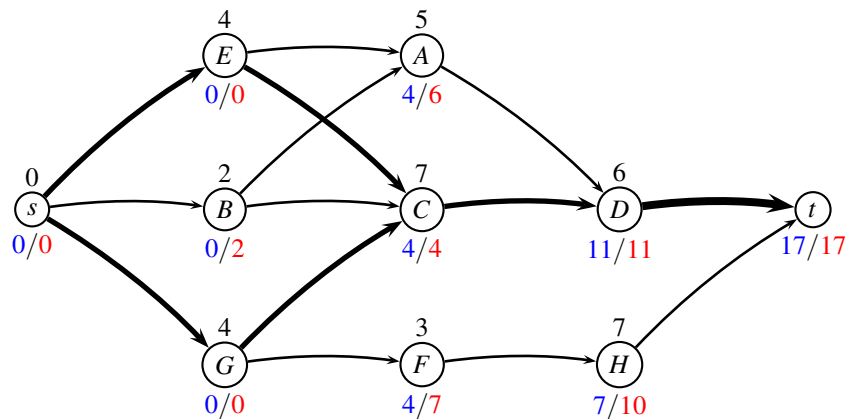
iteration	s	A	B	C	D	E
0	0	∞	∞	∞	∞	∞
1	0	-3	-2	2	-1	2
2	0	-3	-2	2	-1	1
3	0	-3	-2	2	-1	1

In principle we should run $n - 1 = 6 - 1$ iterations, but since the last two iterations do not change the potentials, we can stop. ■

Critical path problem

Answer 3 a possible topological ordering is E, B, G, A, C, F, D, H ■

Answer 4 We insert a new node s and connect it to the three activities E, B, G with no predecessors. We insert a new node t and connect it to the two activities D, H with no successors. This gives the following graph (duration of activities is indicated with black on the nodes):



Now we solve the problem

- *Forward procedure:* Starting at time zero, calculate the earliest time U_j each job j can be started $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$. (marked with blue, above)
- *Makespan:* $T = U_t$.
- *Backward procedure:* Starting at time equal to the makespan, calculate the latest time V_j each job j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$ (marked with red, above)

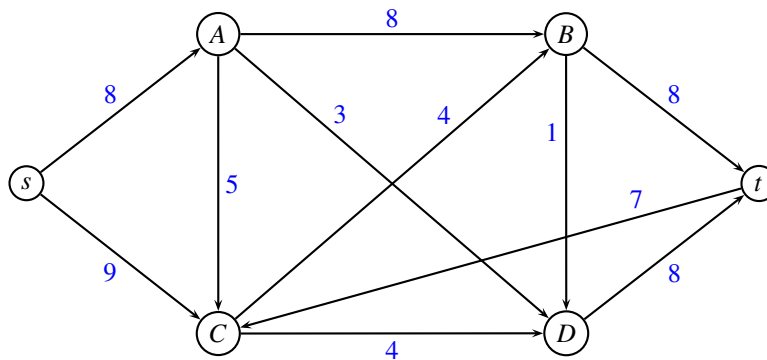
The critical path schedule solution for this project is given below:

Activity	Duration	Earliest start time U_j	Latest start time V_j
s	0	0	0
A	5	4	6
B	2	0	2
C	7	4	4
D	6	11	11
E	4	0	0
F	3	4	7
G	4	0	0
H	7	7	10
t	0	17	17

The makespan is $T = 17$.

The critical path is found for activities where earliest start time U_j is equal to the latest start time V_j in the above table. This means projects C, D, E, G . The critical path(s) are marked with bold edges in the above graph. ■

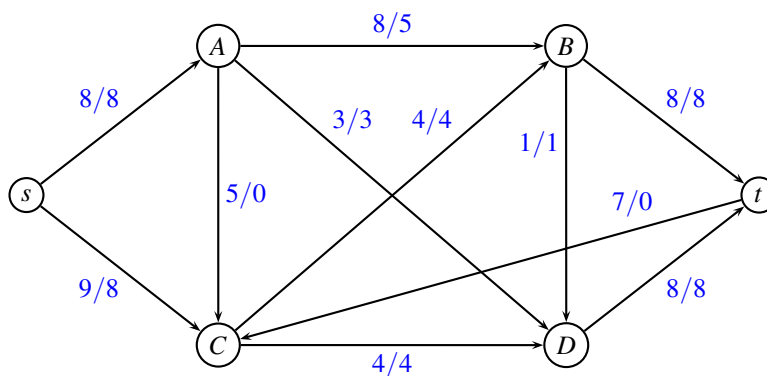
Maximum flow



Answer 5 We use the paths:

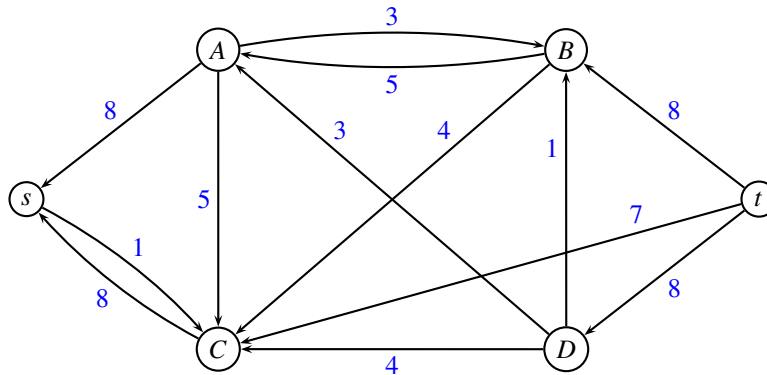
path	capacity
sABt	8
sCDt	4
sCBADt	3
sCBDt	1

This results in the following solution (edge labels are: capacity/flow):



The maximum flow is 16. ■

Answer 6 The residual graph is



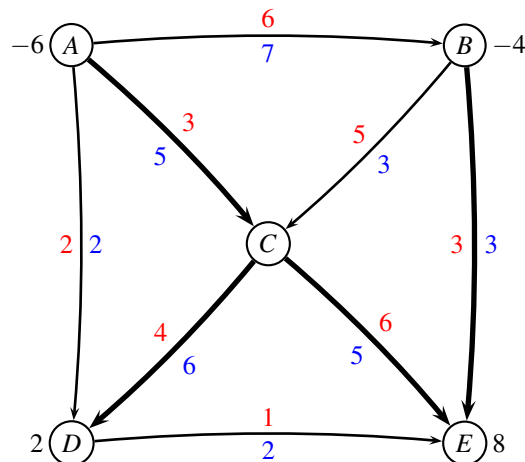
A minimum cut is found by marking all nodes that are reachable from s in the residual graph. This gives the sets $\{s, C\}$ and $\{A, B, D, t\}$. The capacity of the cut is $8 + 1 + 3 + 4 = 16$.

Alternative solution

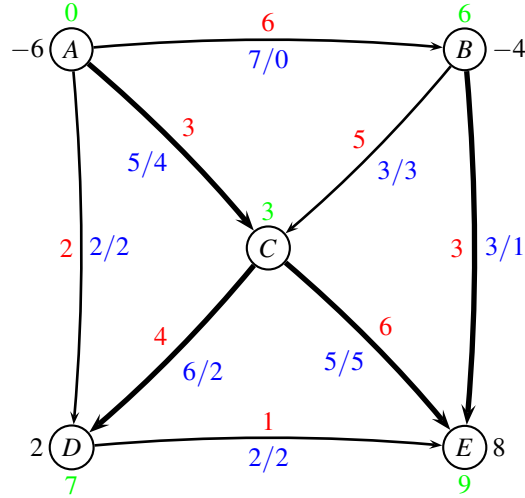
Another minimum cut (found by inspection) is $\{s, A, B, C\}$ and $\{D, t\}$. ■

Minimum cost flow problem

Answer 7 We have the following network



We are told that $L = \{(A, B)\}$ and $U = \{(A, D), (B, C), (D, E)\}$. The flow on edges L is at the lower bound, while the flow on edges in U is at the upper bound. Based on this information, we can find the flow as



This is our initial flow. The cost of the flow is

$$z = 3 \cdot 4 + 2 \cdot 2 + 5 \cdot 3 + 3 \cdot 1 + 4 \cdot 2 + 6 \cdot 5 + 1 \cdot 2 = 74$$

■

Answer 8 Calculating the distances from A to all nodes, only using the edges in T gives the dual variables u_i , indicated with green on the figure.

We can now calculate reduced costs of all edges not in T using the equation $\bar{c}_{ij} = c_{ij} + u_i - u_j$. This gives

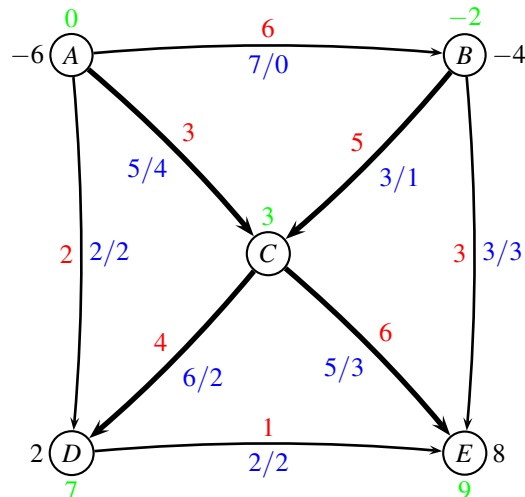
$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - 6 = 0 \quad (L) \\ \bar{c}_{AD} &= 2 + 0 - 7 = -5 \quad (U) \\ \bar{c}_{BC} &= 5 + 6 - 3 = 8 \quad (U) \\ \bar{c}_{DE} &= 1 + 7 - 9 = -1 \quad (U)\end{aligned}$$

For edges $(i, j) \in L$ the reduced cost $\bar{c}_{ij}$ should be negative. For edges $(i, j) \in U$ the reduced cost $\bar{c}_{ij}$ should be positive.

Therefore, we pivot on edge (B, C) with cycle C, B, E, C . The maximum number of units we can send is

$$\theta = \min\{3, 2, 5\} = 2$$

Sending two units along the cycle gives the graph



Calculating the distances from A to all nodes, only using the edges in T gives the dual variables u_i , indicated with green on the figure.

We can now calculate reduced costs of all edges not in T

$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - (-2) = 8 & (L) \\ \bar{c}_{AD} &= 2 + 0 - 7 = -5 & (U) \\ \bar{c}_{BE} &= 3 + (-2) - 9 = -8 & (U) \\ \bar{c}_{DE} &= 1 + 7 - 9 = -1 & (U)\end{aligned}$$

For edges $(i, j) \in L$ the reduced cost $\bar{c}_{ij}$ should be negative. For edges $(i, j) \in U$ the reduced cost $\bar{c}_{ij}$ should be positive. This is not the case, so we stop.

The cost of the optimal solution is:

$$z = 3 \cdot 4 + 2 \cdot 2 + 5 \cdot 1 + 3 \cdot 3 + 4 \cdot 2 + 6 \cdot 3 + 1 \cdot 2 = 58$$

Since we started from a solution with $z = 74$ and sent 2 units along a cycle with reduced cost -8 the optimal solution corresponds to what we would expect. ■

Column generation

Answer 9 We have the dual solution given by $y_{AB} = -10$, while all other $y_{ij} = 0$. Moreover, we have $\bar{y}_1 = 48, \bar{y}_2 = 21, \bar{y}_3 = 36$.

We calculate the reduced costs using the equation:

$$\bar{c}_r^k = \sum_{(i,j) \in r} d_k (c_{ij} - y_{ij}) - \bar{y}_k$$

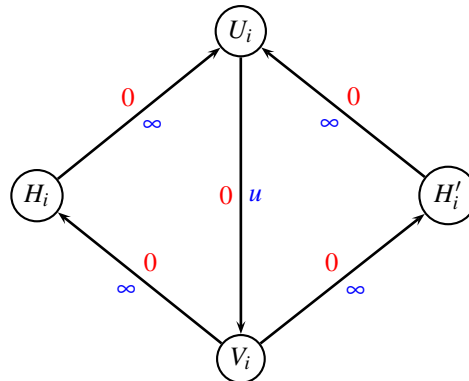
This gives the following table:

number	commodity k	route r	d_k	$\sum_{(i,j) \in r} c_{ij}$	$\sum_{(i,j) \in r} (c_{ij} - y_{ij})$	$d_k \sum_{(i,j) \in r} (c_{ij} - y_{ij})$	$\bar{y}_i$	$\bar{c}_r^k$
1	1	ABC	3	6	16	48	48	0
2	1	ABDC	3	20	30	90	48	42
3	1	ADBC	3	14	14	42	48	-6
4	1	ACD	3	16	16	48	48	0
5	2	EABC	1	11	21	21	21	0
6	2	EABDC	1	25	35	35	21	14
7	2	EADBC	1	19	19	19	21	-2
8	2	EADC	1	21	21	21	21	0
9	3	BCDEA	3	8	8	24	36	-12
10	3	BDEA	3	12	12	36	36	0

The most negative reduced cost is route $r = 9$ with reduced cost -12 . ■

Modeling a hub

Answer 10 We use the modeling trick from assignment 17.21 in Ahuja, Orlin, Magnanti *Network Flows*: The edges between H_i and H'_i are replaced by the construction



Costs are red, and capacities are blue. Here, u is the capacity of the crane (8 or 5). This is done for all pairs (H_1, H'_1) , (H_2, H'_2) , and (H_3, H'_3) . ■

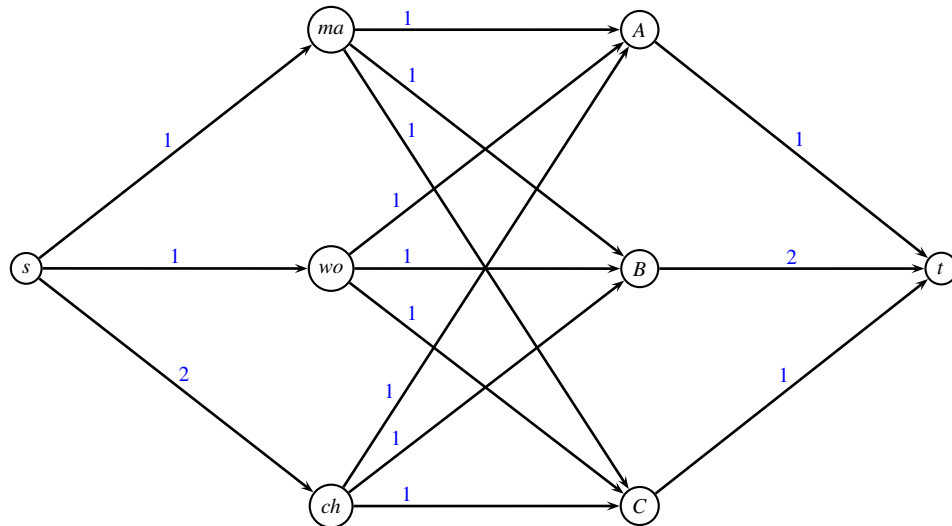
Census rounding

Answer 11 We are given a matrix of fractional values:

Table 3				
Country	A	B	C	Total
grown-up men	0,1	0,7	0,2	1,0
grown-up women	0,4	0,6	0,0	1,0
children	0,5	0,7	0,8	2,0
total	1,0	2,0	1,0	4,0

Since there is no objective function associated with the problem, we can formulate it as a maximum flow problem. Rows in the matrix (men, women, children) correspond to nodes ma , wo , ch , and columns in the matrix correspond to the nodes A, B, C . Edges with capacity 1 are added between each row and column node. Row nodes are connected to s with edges having a capacity corresponding to the demanded sum in the row. Column nodes are connected to s with edges having a capacity

corresponding to the demanded sum in the column.



Notice that an optimal solution always will have integral flow since the problem is TU.

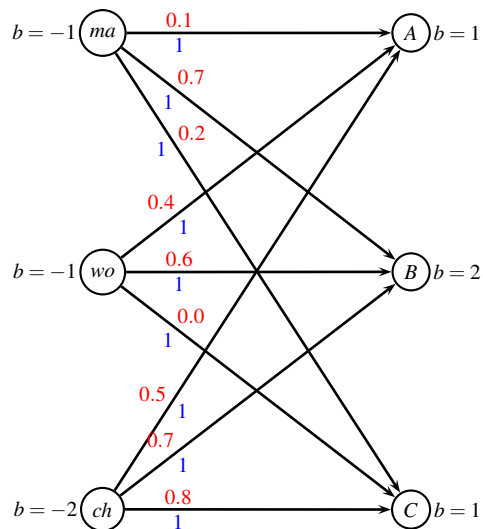
If there is a flow of 1 on an edge between a row node and a column node, then the corresponding entry in the matrix should be rounded up to 1. Otherwise it should be rounded down to 0.

Alternative solution

We can also model the problem as a minimum-cost-flow problem.

We construct a bi-partitate graph as follows: Rows in the matrix (men,women,children) correspond to nodes *ma*, *wo*, *ch*, and columns in the matrix correspond to the nodes *A*, *B*, *C*. Edges with capacity $u_{ij} = 1$ are added between each row node and column node.

Since there is no objective function associated with the problem, we can choose any edge costs c_{ij} but a value of $c_{ij} = 1 - f_{ij}$ where f_{ij} is the fractional value in Table 3, can be used.

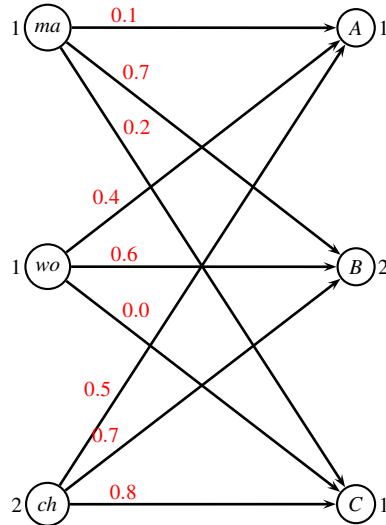


Notice that an optimal solution always will have integral flow since the problem is TU.

If there is a flow of 1 on an edge between a row node and a column node, then the corresponding entry in the matrix should be rounded up to 1. Otherwise it should be rounded down to 0.

(Wrong) alternative solution

One could try to model the problem as a transportation problem.



Since there is no objective function associated with the problem, we can again choose any edge costs c_{ij} . A value of $c_{ij} = 1 - f_{ij}$ where f_{ij} is the fractional value in Table 3, can be used.

Modeling the problem as a transportation problem will not work in general, since there are no capacities on the edges, and hence we can get a flow larger than one on edges. In the above example we could thus find a solution with a flow of 2 from *ch* to *B*. This means that the fractional value should be round up twice, which is not allowed. ■

Answer 12 In the general problem we are given the matrix

Table 1

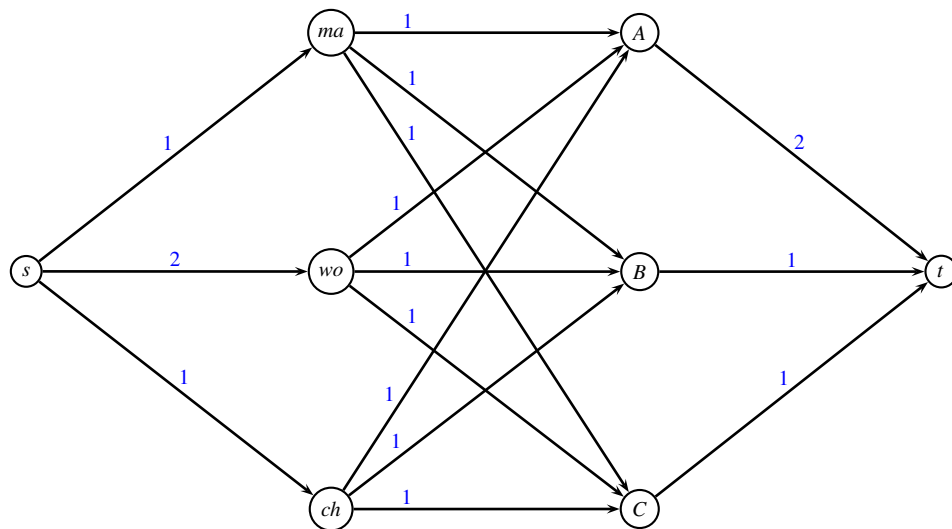
Country	A	B	C	Total
grown-up men	11,6	9,4	3,0	24,0
grown-up women	12,7	10,5	3,8	27,0
children	1,7	2,1	0,2	4,0
total	26,0	22,0	7,0	55,0

We first subtract the integer part from each entry, and subtract it from the row and column sums. This gives the matrix

Table 1*

Country	A	B	C	Total
grown-up men	0,6	0,4	0,0	1,0
grown-up women	0,7	0,5	0,8	2,0
children	0,7	0,1	0,2	1,0
total	2,0	1,0	1,0	4,0

This leads to the following maximum flow problem



Notice that an optimal solution always will have integral flow since the problem is TU.

If there is a flow of 1 on an edge between a row node and a column node, then the corresponding entry in the matrix should be rounded up, otherwise it should be rounded down.

Alternative solution

We can also model the problem as a minimum-cost-flow problem. The same principles as in previous answer are used. ■

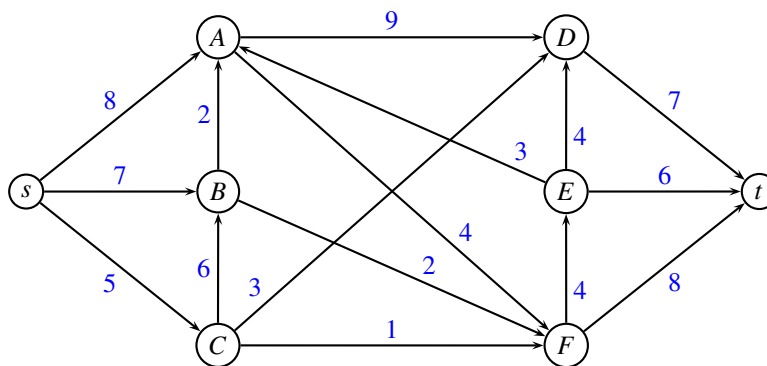
Answer, exam December, 2017.

Course name: Network Optimization

Course no: 42115

Maximum flow

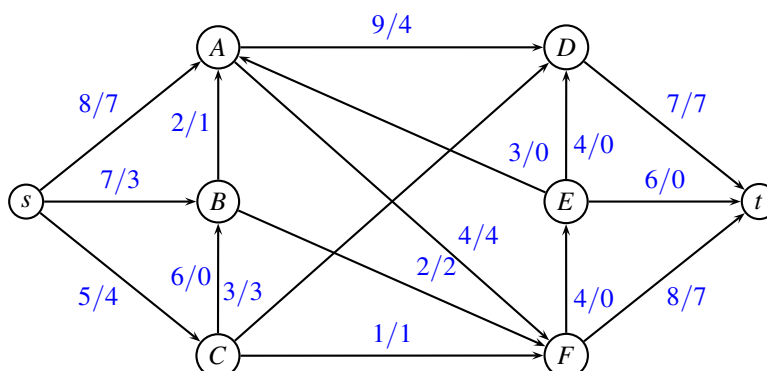
Consider the following maximum flow problem where edge weights denote the capacity of an edge.



Answer 1 We use the augmenting paths

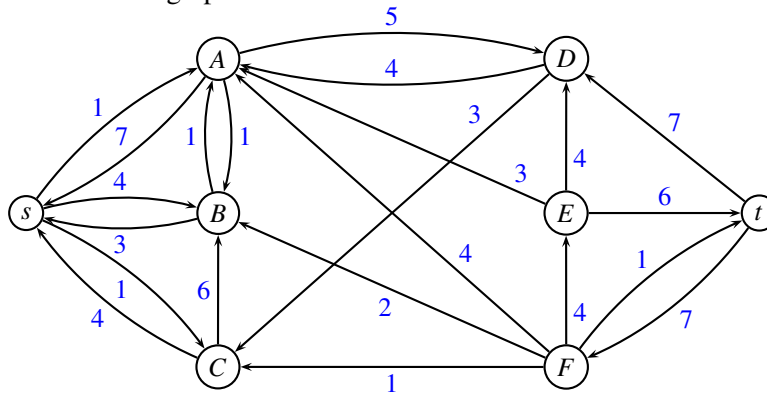
- $sADt$ capacity 7
- $sBFt$ capacity 2
- $sCFt$ capacity 1
- $sCDAFt$ capacity 3
- $sBAFt$ capacity 1

(There are many alternative ways of finding augmenting paths). This results in the flow



The maximum flow is 14. ■

Answer 2 We get the residual graph



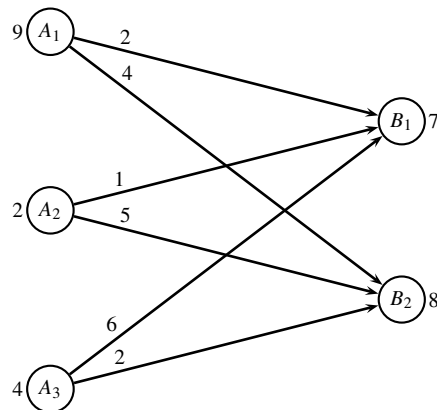
A minimal cut is $S = \{s, A, B, C, D\}$ and $T = \{E, F, t\}$. The capacity of the cut is 14. ■

Transportation problem

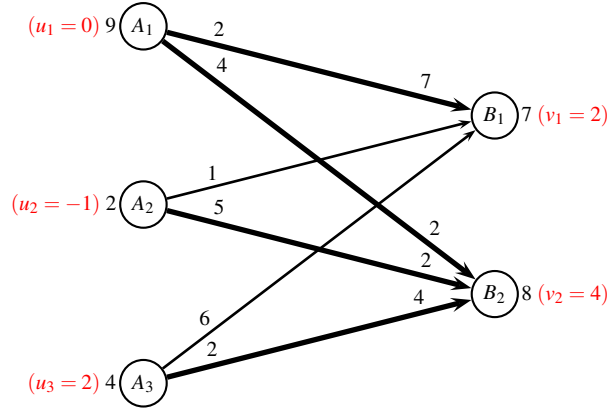
Consider the following transportation problem. We have three suppliers A_1, A_2, A_3 and two customers B_1, B_2 . The cost of transporting one unit from A_i to B_j is given in the following table

	B_1	B_2	supply
A_1	2	4	9
A_2	1	5	2
A_3	6	2	4
demand	7	8	

Answer 3 The problem can be illustrated as



We solve the problem with the greedy algorithm getting the solution



The cost of the solution is $2 \cdot 7 + 4 \cdot 2 + 5 \cdot 2 + 2 \cdot 4 = 40$.

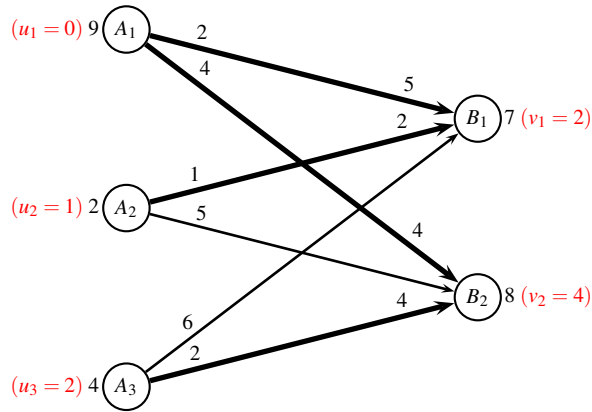
Fat edges on the graph indicate the basis. Dual values (indicated with red) are found by finding the shortest path from node A_1 , using only the fat edges.

We now find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, getting

$$\begin{aligned}\bar{c}_{21} &= 1 + (-1) - 2 = -2 \\ \bar{c}_{31} &= 6 + 2 - 2 = 6\end{aligned}$$

We choose edge (A_2, B_1) corresponding to the cycle (A_2, B_1, A_1, B_2) . We can send at most $\delta = \min\{7, 2\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $2 \cdot 5 + 4 \cdot 4 + 1 \cdot 2 + 2 \cdot 4 = 36$.

Dual values (indicated with red) are found by finding the shortest path from node A_1 , using only the fat edges.

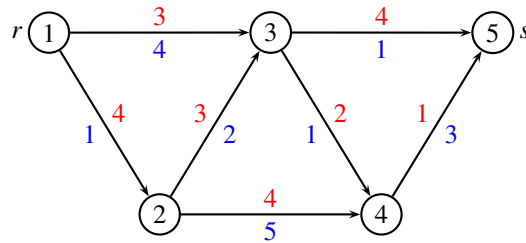
We now find the reduced costs as

$$\begin{aligned}\bar{c}_{22} &= 5 + 1 - 4 = 2 \\ \bar{c}_{31} &= 6 + 2 - 2 = 6\end{aligned}$$

Since no edges have negative reduced cost, the solution is optimal. ■

Resource constrained shortest path problem

Consider the following directed acyclic graph. Every edge (i, j) has an associated cost c_{ij} (red) and time t_{ij} (blue). We are searching for a shortest path (with respect to cost) from r to s . There is a time constraint T imposed on the path. We would like to find the shortest path for every relevant value of T .



Answer 4 First notice that the graph is a directed acyclic graph (DAG). We can therefore order the nodes in topological order, and use a label setting algorithm (i.e. dynamic programming) to solve the problem.

It is also OK to use a label correcting algorithm, but we only need to consider each node once, so it is not necessary to have a queue Q . ■

Answer 5 We consider the nodes in order 1,2,3,4,5 finding the lists

node 1	node 2	node 3	node 4	node 5
cost time	cost time	cost time	cost time	cost time
0 0	4 1	7 3	9 4	11 4
		3 4	5 5	7 5
			(8 6)	6 8
				(10 7)

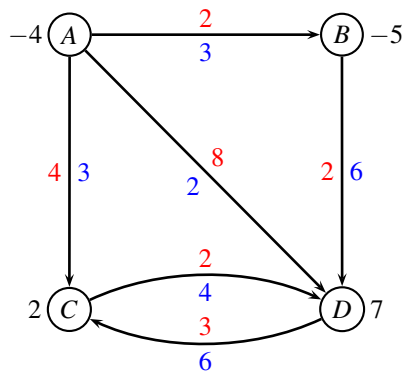
The dominated entries are marked with a paranthesis. This means that we can reach node s with the following costs and times:

cost	time
11	4
7	5
6	8

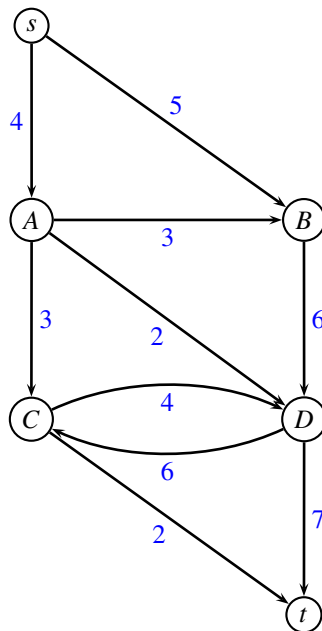
This means, for a time limit $T < 4$ we cannot find any solution. For time $T \geq 4$ cost is 11. For time $T \geq 5$ cost is 7. For time $T \geq 8$ cost is 6. ■

Min cost flow

Consider the following network with edge costs c_{ij} (red) and edge capacities u_{ij} (blue). The demand b_i (black) in every node is positive for a sink, and negative for a source.



Answer 6 We find an initial solution by solving a maximum flow problem

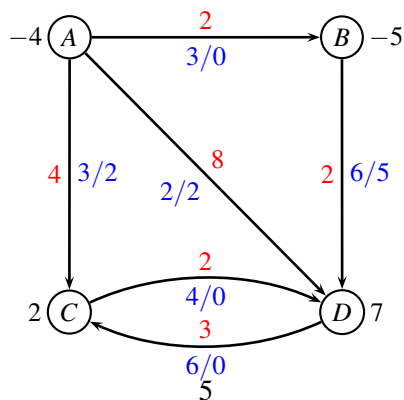


We use an augmenting path method, sending flow

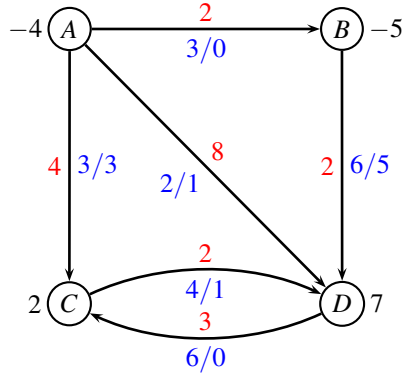
- $sACt$ flow 2
- $sADt$ flow 2
- $sBDt$ flow 5

The solution value is $4 \cdot 2 + 8 \cdot 2 + 2 \cdot 5 = 34$. ■

Answer 7 We have the initial solution

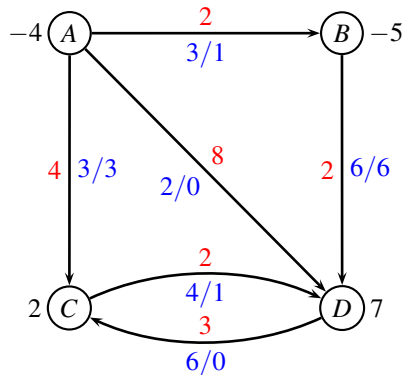


Using the augmenting cycle algorithm, we find a cycle $ACDA$ with cost $4 + 2 - 8 = -2$. The capacity we can send along the cycle is $\delta = \min\{1, 4, 2\} = 1$. Sending 1 unit along the cycle we get:



The solution value is $4 \cdot 3 + 8 \cdot 1 + 2 \cdot 5 + 2 \cdot 1 = 32$.

We find a new cycle $ABDA$ with cost $2 + 2 - 8 = -4$. The capacity we can send along the cycle is $\delta = \min\{3, 1, 1\} = 1$. Sending 1 unit along the cycle we get:



No more negative cycles can be found, so the solution is optimal. The solution value is $2 \cdot 1 + 4 \cdot 3 + 2 \cdot 6 + 2 \cdot 1 = 28$. ■

Critical path problem

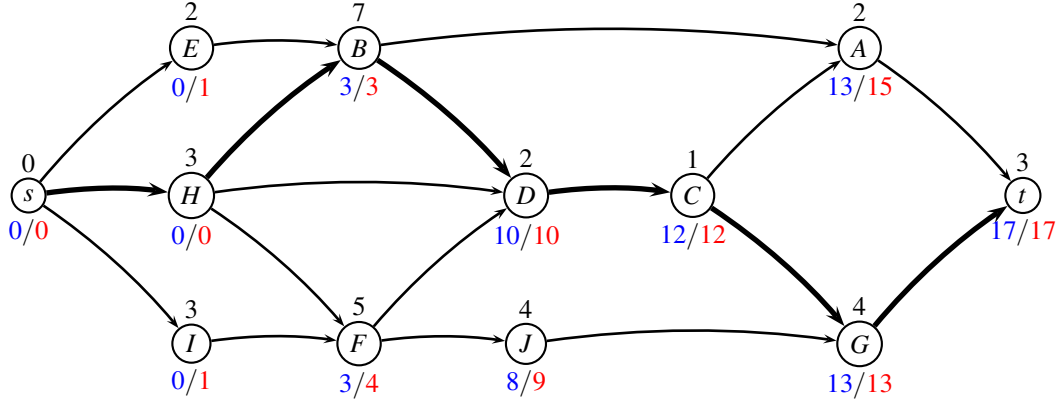
We have the activities

Activity	Description	Predecessors	Duration
A	Chain	B, C	2
B	Crankset	E, H	7
C	Chainring	D	1
D	Handlebar	B, F, H	2
E	Spindle		2
F	Wheel	H, I	5
G	Pedals	C, J	4
H	Fork		3
I	Rear gear		3
J	Brakes	F	4

Answer 8 We order the tasks in topological order, getting the ordering

$$E, H, I, B, F, D, J, D, A, G$$

which also can be drawn as follows:



In this graph, we inserted a new node s and connect it to the three activities E, H, I with no predecessors. Moreover, we inserted a new node t and connect it to the two activities A, G with no successors.

■

Answer 9 Now we solve the problem

- *Forward procedure:* Starting at time zero, calculate the earliest time U_j each job j can be started $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$. (marked with blue, above)
- *Makespan:* $T = U_t = 17$.
- *Backward procedure:* Starting at time equal to the makespan, calculate the latest time V_j each job j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$ (marked with red, above)

■

Answer 10 The critical path is marked with fat edges. There is only one critical path. Although both node H and D are critical, the edge (H, D) is not on the critical path. ■

Arranging a dinner

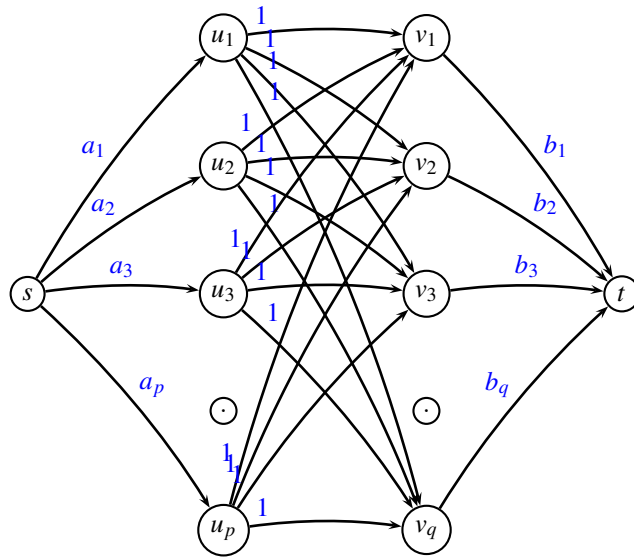
Answer 11 We define a capacitated graph $G = (V, E)$ as follows

$$V = \{s, t\} \cup \{u_i, 1 \leq i \leq p\} \cup \{v_j, 1 \leq j \leq q\}$$

and

$$E = \{(s, u_i), 1 \leq i \leq p\} \cup \{(u_i, v_j), 1 \leq i \leq p, 1 \leq j \leq q\} \cup \{(v_j, t), 1 \leq j \leq q\}$$

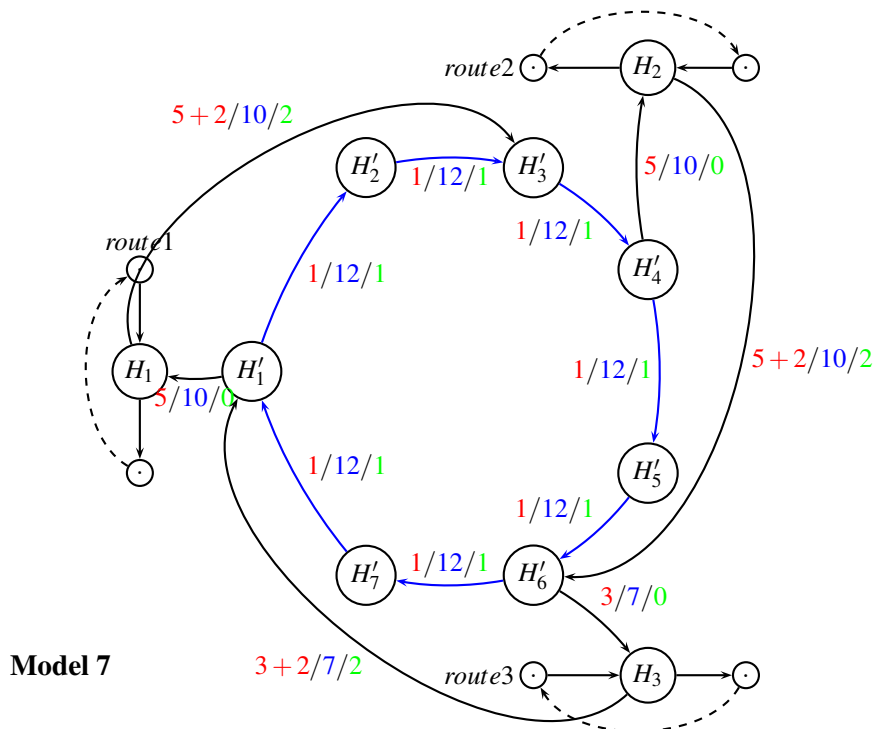
The capacity of edges (s, u_i) is set to a_i , the capacity of edges (u_i, v_j) is set to 1, and the capacity of edges (v_j, t) is set to b_j .



An integral solution to the maximum flow problem gives an assignment of families to tables. A flow from u_i to v_j is either zero or one, with one meaning that a member of the family i sits at table j . The maximum flow maximizes the number of people that can be seated under the stated constraints. ■

Modeling a hub

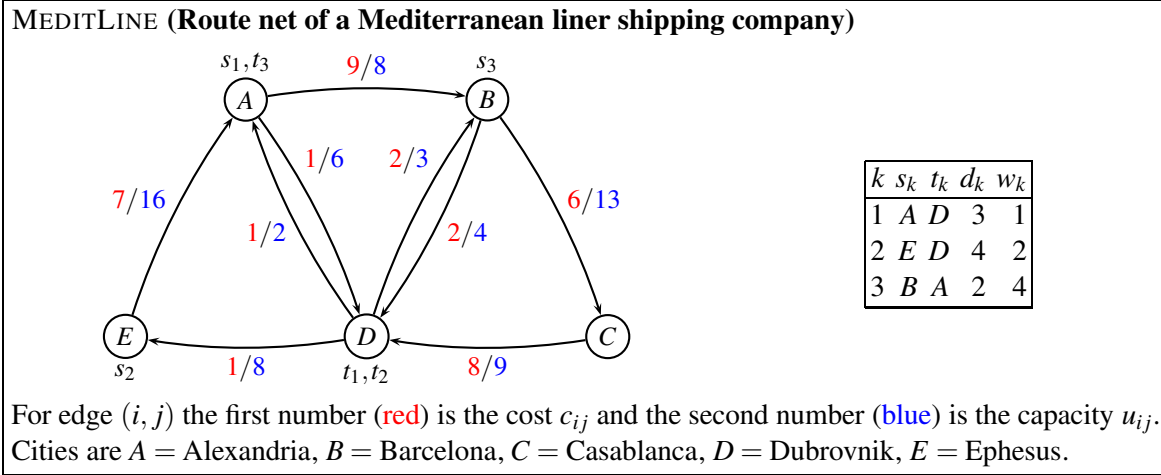
Answer 12 The loading and unloading edges are connected to the week day when a container is available after custom declaration.



In this formulation it is possible to handle transshipment cost and time, as well as crane capacity and storage cost and capacity (i.e. all the given data). Costs are indicated with **red**, capacities with **blue**, and time with **green** in the above figure.

Transshipment costs and crane capacity are assigned to edges between H_i and H'_i . Since the custom has infinite capacity for storing containers in a separate stock pile, it is only the crane capacity we need to respect on the unloading edges. Since it costs 1\$ per day the containers are stored in the custom, this cost has to be added to the cost of unloading edges. The capacity of edges between H'_i and H'_j is set to the storage capacity $\bar{c}$. Transshipment times are easily handled since every blue edge takes one day, loading edges take 0 days, and unloading edges take 2 days. ■

Flowing containers in a network



The dual variables are $y_{BD} = -12$, $\bar{y}_1 = 1$, $\bar{y}_2 = 100$, $\bar{y}_3 = 88$. All other dual variables are zero.

Answer 13 The reduced cost of a route r is calculated as

$$\bar{c}_r^k = \sum_{(i,j) \in r} w_k c_{ij} - \sum_{(i,j) \in r} w_k y_{ij} - \bar{y}_k$$

This leads to the following table:

number	commodity k	route r	w_k	$\sum w_k c_{ij}$	$\sum w_k y_{ij}$	$\bar{y}_k$	reduced cost $\bar{c}_r^k$
1	1	ABCD	1	23	0	1	22
2	1	ABD	1	11	-12	1	22
3	1	AD	1	1	0	1	0
4	2	EABCD	2	60	0	100	-40
5	2	EABD	2	36	-24	100	-40
6	2	EAD	2	16	0	100	-84
7	3	BCDEA	4	88	0	88	0
8	3	BDEA	4	40	-48	88	0

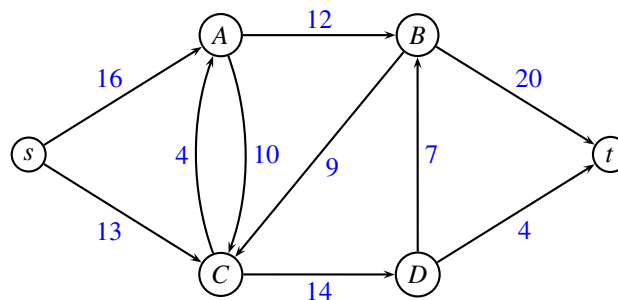
The most promising route to add to the model is route number 6 with reduced cost -84. ■

Solution, exam 19 December, 2018.

Course title: Network Optimization

Course no: 42115

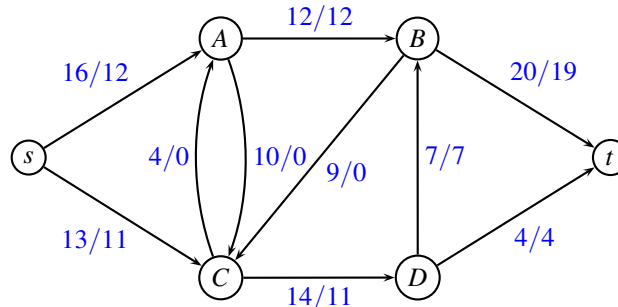
Maximum Flow problem



Answer 1 We solve the maximum flow problem using an augmenting path algorithm.

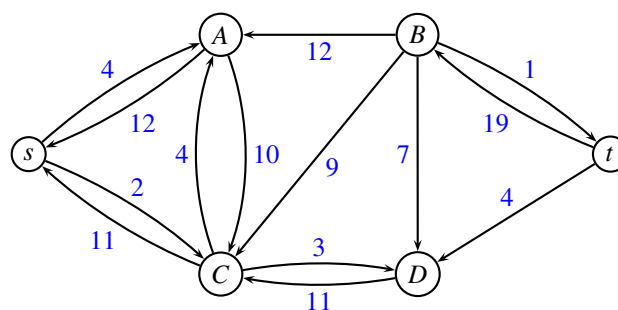
- path $sABt$ capacity 12
- path $sCDBt$ capacity 7
- path $sCDt$ capacity 4

No more augmenting paths can be found. This gives the solution:



with flow $12 + 11 = 23$. ■

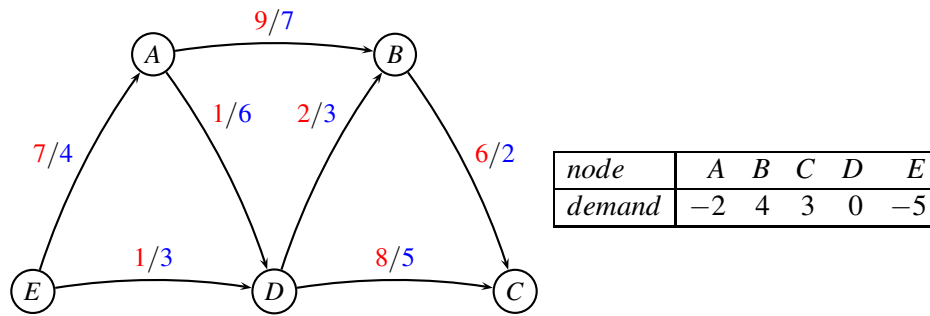
Answer 2 The residual graph is:



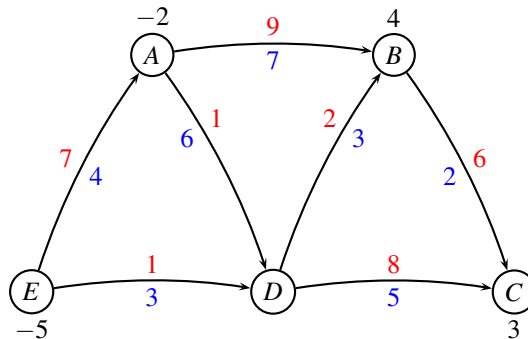
A minimum cut is $S = \{s, A, C, D\}$ and $T = \{B, t\}$. The capacity of the cut is $12 + 7 + 4 = 23$. ■

Empty container repositioning

MEDITLINE: Route net of a Mediterranean liner shipping company



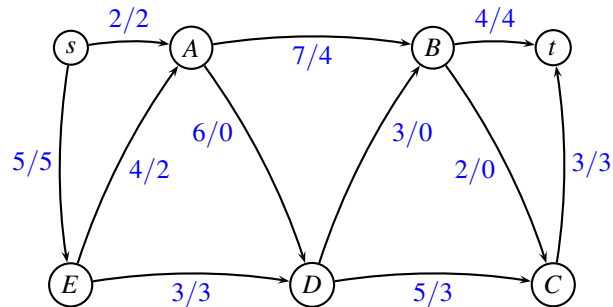
Answer 3 Since all containers are the same, we formulate the problem as a min cost flow problem, and solve it using the augmenting cycle algorithm or network simplex algorithm. The demands are set to $b_i := d_i$. The network becomes:



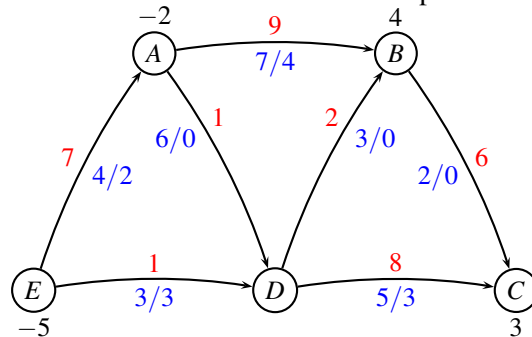
■

Answer 4 In order to find an initial solution we solve a maximum flow problem. Augmenting paths are: $sEDCt$ with capacity 3. $sABt$ with capacity 2. $sEABt$ with capacity 2. This is illustrated in the

following:

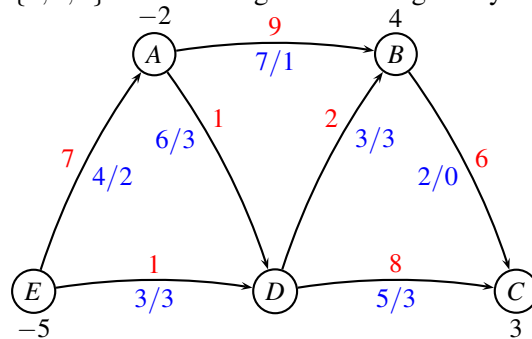


This leads to the following initial solution to the min cost flow problem:



Cost of the solution is $z = 77$.

We use the *augmenting cycle* algorithm. An augmenting cycle is $ADBA$ with cost $1 + 2 - 9 = -6$. Capacity of the cycle is $\min\{6, 3, 4\} = 3$. Sending 3 units along the cycle gives



No more negative cycles can be found, so optimal solution found. Cost of the solution is $z = 59$.

Note: One can also use the *Network Simplex* algorithm. In this case an initial solution should be found as described above, and then the algorithm is run. ■

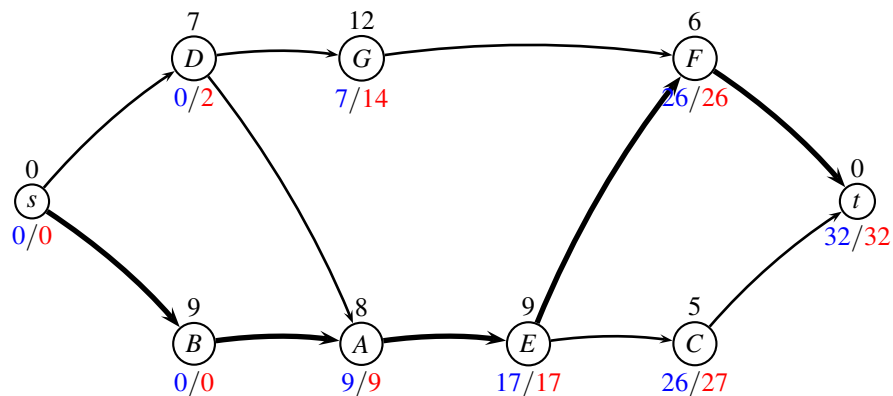
Critical path problem

Activity i	Description	Predecessors	Duration d_i
A	Start oven at 250 degrees	D, B	8
B	Weigh rye flour and wheat flour		9
D	Dissolve yeast in the water		7
C	Prepare a barrel for storage	E	5
E	Cut the dough into squares	A	9
F	Bake the biscuits	G, E	6
G	Mix ingredients and add salt	D	12

Answer 5 A possible ordering of the activities is: *sDBGAEF**Ct*. This is also illustrated in the graph below. ■

Answer 6 Now we solve the problem

- *Forward procedure:* Starting at time zero, calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j) \in E} \{U_i + d_i\}$. (marked with blue)
- *Makespan:* $T = U_t = 32$.
- *Backward procedure:* Starting at time equal to the makespan, calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - d_i\}$ (marked with red)



Answer 7 An activity j is on the critical path if $U_j = V_j$. The critical path is marked with fat edges. There is only one critical path. ■

I found the following recepy for making ship's biscuits. It is a recepy from the 18th century. Unfortunately it is in Danish, so you must use *Google translate* to get it in English.

Beskøjter

650 gram rugmel
 350 gram hvedemel
 50 gram gær
 15 gram salt
 1 liter vand

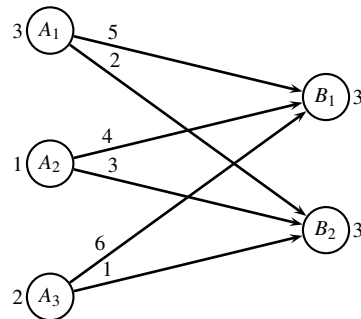
Gæren opløses i vandet, det øvrige blandes i og æltes godt. Om nødvendigt tilsættes lidt mere hvedemel for at gøre dejen egnet til udrulning.

Dejen stilles til hævnning $\frac{1}{2}$ -1 time, udrulles til plader på ca. 1 centimeters tykkelse, skæres i firkanter og sættes på plader med bagepapir. Beskøjterne efterhæves og prikkes med en gaffel.

Bages for 250 grader i et kvarter og tørres derefter for svag varme og åben luge i 1-3 timer, alt efter om de skal have et blødt indre eller være helt hårde.

Opskriften stammer fra 1800-tallet og findes på Folkklivsarkivet i Lund.

Fixed charge transportation problem



Answer 8 We have the following possible patterns

	A_1				A_2		A_3		
B_1	3	2	1	0	1	0	2	1	0
B_2	0	1	2	3	0	1	0	1	2
transp. cost	15	12	9	6	4	3	12	7	2
fixed charges	4	8	8	4	4	4	4	8	4
sum of costs	19	20	17	10	8	7	16	15	6

This leads to the Dantzig-Wolfe decomposed model:

```

minimize
19p11+20p12+17p13+10p14+ 8p21+7p22+16p31+15p32+ 6p33
subject to
3p11+ 2p12+ 1p13+ 0p14+ 1p21+0p22+ 2p31+ 1p32+ 0p33 = 3    (u1)
0p11+ 1p12+ 2p13+ 3p14+ 0p21+1p22+ 0p31+ 1p32+ 2p33 = 3    (u2)

p11+  p12+  p13+  p14                                     = 1    (v1)
                                p21 +p22                    = 1    (v2)
                                p31+  p32+  p33              = 1    (v3)

binary p11 p12 p13 p14 p21 p22 p31 p32 p33
end

```

■

Answer 9 The initial solution corresponds to the restricted master problem

```

minimize
    20p12                      + 7p22          +15p32
subject to
    2p12                      + 0p22          + 1p32          = 3    (u1)
    1p12                      + 1p22          + 1p32          = 3    (u2)

    p12                      = 1    (v1)
                                p22                      = 1    (v2)
                                p32                      = 1    (v3)

end

```

We are told that the dual variables for the restricted master problem are $u_1 = 0, u_2 = 0, v_1 = 20, v_2 = 7, v_3 = 15$.

Let a_{ip} be the coefficients in the two first lines of the pattern formulation, and b_{ip} be the coefficients in the last three lines. Reduced cost for pattern p is then calculated as

$$\bar{c}_p = c_p - \sum_i a_{pi}u_i - \sum_i b_{pi}v_i$$

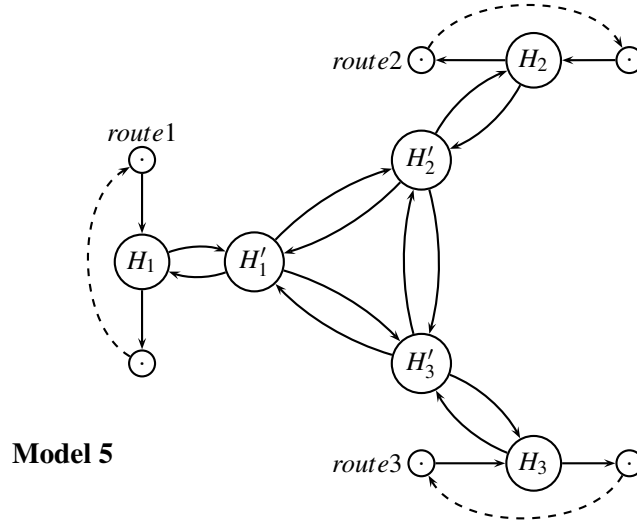
This gives the following reduced costs (we do not need to calculate reduced costs of patterns that are present in the restricted master problem)

pattern p	c_p	$\sum_i a_{pi}u_i$	$\sum_i b_{pi}v_i$	$\bar{c}_p$
p_{11}	19	0	20	-1
p_{13}	17	0	20	-3
p_{14}	10	0	20	-10
p_{21}	8	0	7	1
p_{31}	16	0	15	1
p_{33}	6	0	15	-9

Pattern p_{14} has the most negative reduced cost, so it should be added to the restricted master problem.

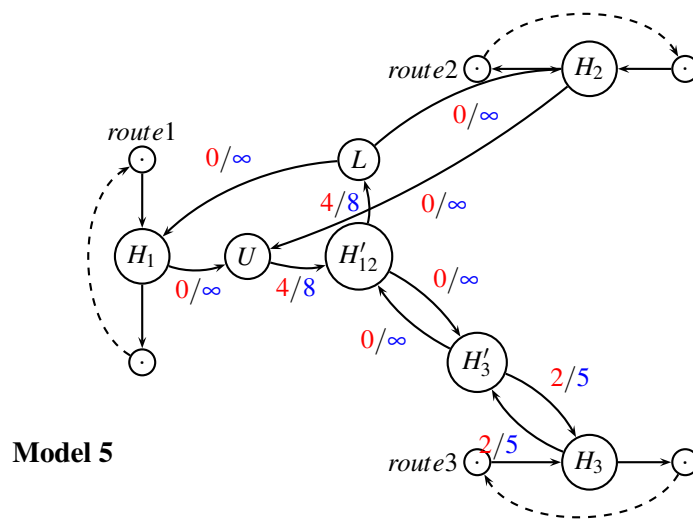
■

Modeling a hub port



We are considering Model 5 depicted above. Two vessels arrive the same day (route 1) and (route 2). Therefore, they have to share the crane capacity for loading and unloading. This means that the number of containers unloaded from vessels at route1 and route2 together should be below $u = 8$. Similarly, the number of containers loaded to the two routes together should be below $u = 8$.

Answer 10 Since the two vessels arrive the same day, we can merge nodes H'_1 and H'_2 into one node H'_{12} . Moreover we introduce two auxiliary nodes, L and U for loading and unloading. The capacity between U and H'_{12} , and between H'_{12} and L is set to $u = 8$. The capacity between H_1 and U, L , and between U, L and H_2 is set to $u = \infty$. The costs should be assigned in a similar way.

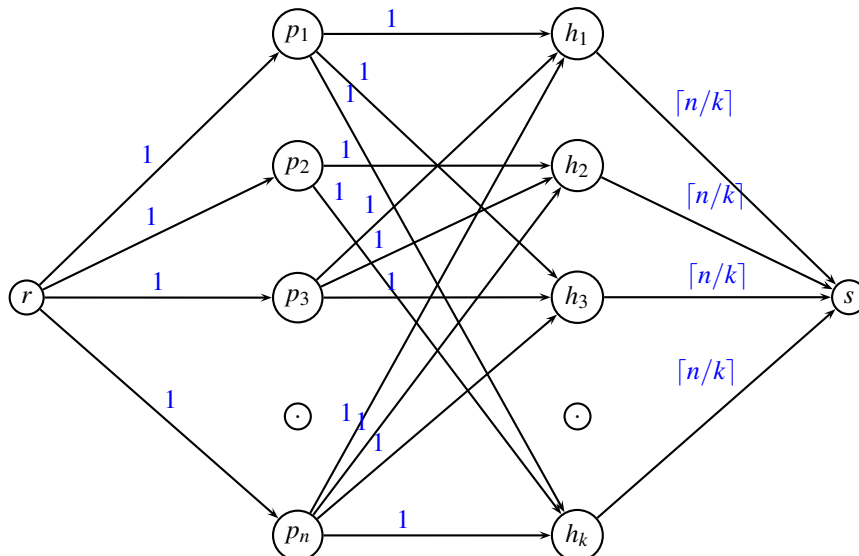


Note: It is not sufficient to merge nodes H'_1 and H'_2 into one node H'_{12} , without introducing nodes U and L . Otherwise, it is not possible to model the shared capacity (or you will be able to transship an unlimited number of containers from route1 to route2). ■

Flooding

Answer 11 We have n people, and k hospitals. Each hospital may take at most $\lceil n/k \rceil$ people. We model the problem as a *maximum flow problem* on a bi-partite graph (P, H) . Let the set of nodes P correspond to the people, and the set of nodes H correspond to hospitals. For every patient p_i and hospital h_j we have an edge with capacity 1, if the patient can be brought to the hospital within a half-hour's drive. If the patient cannot be brought to the hospital within a half-hour's drive, the edge is omitted.

Introduce a root r and a sink s . The root r is connected to all patients p_i with capacity 1. All hospitals h_j are connected to sink s with capacity $\lceil n/k \rceil$.

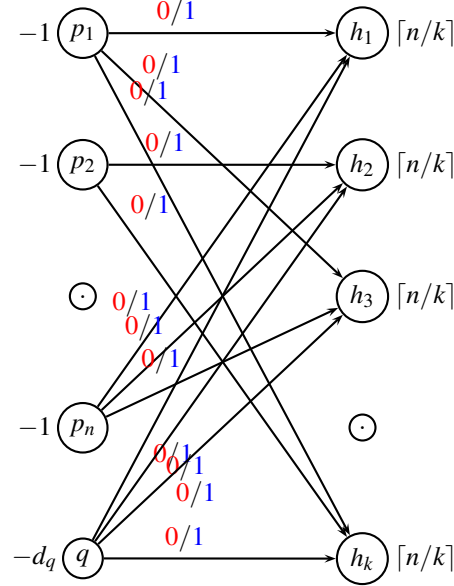


If a flow can be found covering all patients (i.e. flow of n) then it is possible to assign all patients to hospitals.

Alternative solution: We model the problem as a *min cost flow problem* on a bi-partite graph (P, H) . To have a balanced problem, we add a dummy patient node q to adjust for non-integer demands.

For every patient p_i and hospital h_j we have an edge with capacity 1 and cost 0, if the patient can be brought to the hospital within a half-hour's drive. If the patient cannot be brought to the hospital within a half-hour's drive, the edge is omitted. The dummy patient node q is connected to all hospitals h_j with an edge having capacity 1 and cost 0.

The demand of patient p_i is $d_i = -1$, while the demand for hospital h_j is $d_j = \lceil n/k \rceil$. The demand for dummy patient node q is $-d_q = -(k \cdot \lceil n/k \rceil - n)$.

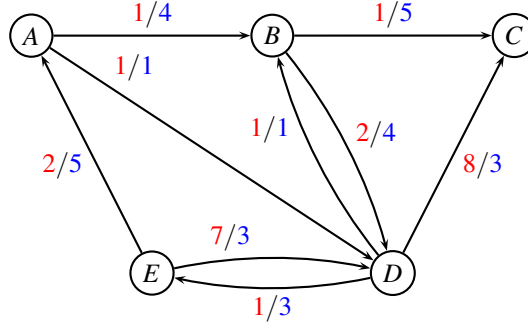


The problem is solved as a min cost flow problem which is solvable in polynomial time. If a feasible solution can be found, then it is possible to find an allocation of people to hospitals.

Note: It is not sufficient to have demand n/k for the hospitals, since if this is a fractional number, some patients will need to be split into fractions. ■

Unsplittable multi-commodity flow problem

ASIANET (Route net of a liner shipping company)



For edge (i, j) the first number (red) is the cost c_{ij} and the second number (blue) is the capacity u_{ij} . Cities are $A = \text{Manilla}$, $B = \text{Shanghai}$, $C = \text{HongKong}$, $D = \text{Xiamen}$, $E = \text{Singapore}$.

We have the following *restricted master problem*:

$$\begin{array}{llllllll}
 \min & 10x_1 & + & 16x_2 & + & 5x_3 & + & 15x_4 \\
 \text{s.t.} & 2x_1 & & & & & & \leq 4 & (y_{AB}) \\
 & & & & x_3 & & & \leq 1 & (y_{AD}) \\
 & & & & x_3 & & & \leq 5 & (y_{BC}) \\
 & 2x_1 & & & & & & \leq 4 & (y_{BD}) \\
 & & & & x_3 & & & \leq 1 & (y_{DB}) \\
 & & 2x_2 & & & + & x_4 & \leq 3 & (y_{DC}) \\
 & & & & & & & \leq 3 & (y_{DE}) \\
 & & & & x_3 & & & \leq 5 & (y_{EA}) \\
 & 2x_1 & & & & + & x_4 & \leq 3 & (y_{ED}) \\
 & x_1 & & & & & & \geq 1 & (\bar{y}_1) \\
 & & x_2 & & & & & \geq 1 & (\bar{y}_2) \\
 & & & & x_3 & + & x_4 & \geq 1 & (\bar{y}_3) \\
 & & & & & & & 0 \leq x_1, x_2, x_3, x_4 \leq 1
 \end{array} \tag{1}$$

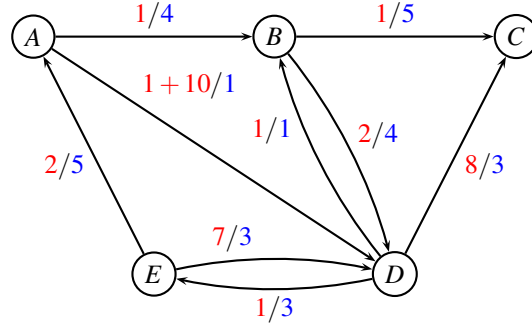
The dual solution to the model is: $y_{AD} = -10$, $\bar{y}_1 = 10$, $\bar{y}_2 = 16$, $\bar{y}_3 = 15$, while all other dual variables are zero.

Answer 12 The reduced cost of a route r is calculated as

$$\bar{c}_r^k = \sum_{(i,j) \in r} d_k(c_{ij} - y_{ij}) - \bar{y}_k$$

We can solve the pricing problem as a shortest path problem, by setting the cost of edge (i, j) to

$(c_{ij} - y_{ij})$. In our case the graph becomes:



Edges with capacity less than d_k are removed before solving the shortest path problem for commodity k .

Notice that $y_{ij} \leq 0$ since the restricted master problem is a minimization problem, and the constraints are “smaller-equal” constraints. This means that all edge costs will be non-negative, and we can therefore safely use Dijkstra’s algorithm. Once the shortest path problem is solved with solution value z , the reduced cost can be calculated as $\bar{c} = d_k \cdot z - \bar{y}_k$. ■

Answer 13 We solve the shortest path problem using Dijkstra’s algorithm

$k = 1$: We remove edges (AD) , (DB) , and find shortest path EABD (route 1) with cost $z = 5$. Reduced cost is $d_1 \cdot z - \bar{y}_1 = 2 \cdot 5 - 10 = 0$.

$k = 2$: We remove edges (AD) , (DB) , and find shortest path DEABC (route 4) with cost $z = 5$. Reduced cost is $d_2 \cdot z - \bar{y}_2 = 2 \cdot 5 - 16 = -6$.

$k = 3$: We do not remove any edges. Shortest path EABC (route 5) with cost $z = 4$. Reduced cost is $d_3 \cdot z - \bar{y}_3 = 1 \cdot 4 - 15 = -11$.

The most negative reduced cost is obtained with path EABC (route 5).

The resulting model becomes

$$\begin{array}{llllllllllllllll}
 \min & 10x_1 & + & 16x_2 & + & 5x_3 & + & 15x_4 & + & 4x_5 & & & & & & & & & \\
 \text{s.t.} & 2x_1 & & & & & & & & + & x_5 & \leq & 4 & & & & & & (y_{AB}) \\
 & & & & & x_3 & & & & & & \leq & 1 & & & & & & (y_{AD}) \\
 & & & & x_3 & & & + & x_5 & \leq & 5 & & & & & & & & (y_{BC}) \\
 & 2x_1 & & & & & & & & & & \leq & 4 & & & & & & (y_{BD}) \\
 & & & & x_3 & & & & & & & \leq & 1 & & & & & & (y_{DB}) \\
 & & 2x_2 & & & + & x_4 & & & & & \leq & 3 & & & & & & (y_{DC}) \\
 & & & & & & & & & & & \leq & 3 & & & & & & (y_{DE}) \\
 & & & & x_3 & & & + & x_5 & \leq & 5 & & & & & & & & (y_{EA}) \\
 & 2x_1 & & & & + & x_4 & & & & & \leq & 3 & & & & & & (y_{ED}) \\
 & x_1 & & & & & & & & & & \geq & 1 & & & & & & (\bar{y}_1) \\
 & & x_2 & & & & & & & & & \geq & 1 & & & & & & (\bar{y}_2) \\
 & & & x_3 & + & x_4 & + & x_5 & \geq & 1 & & & & & & & & & (\bar{y}_3)
 \end{array} \tag{2}$$

$$0 \leq x_1, x_2, x_3, x_4, x_5 \leq 1$$

■

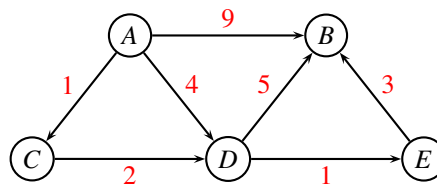
Solution, exam 18 December, 2019.

Course title: Network Optimization

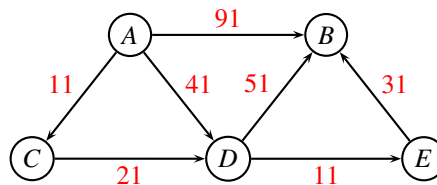
Course no: 42115

Shortest path problem

Directed graph where distances w_{ij} are marked with red.



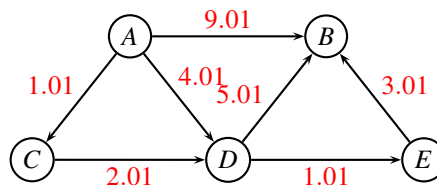
Answer 1 Choose a big number M larger than n (number of nodes). Introduce new edge weights $w'_{ij} = Mw_{ij} + 1$. In our case we can choose $M = 10$ getting:



In all calculations, the most significant digit will represent the distance, while the least significant digit will represent the number of edges. Since the graph has 5 nodes a shortest path can contain at most 4 edges, hence a carry will never propagate to the most significant digit.

Alternative formulation

Introduce new edge weights $w'_{ij} = w_{ij} + \epsilon$, where ϵ is a small number, e.g. 0.01. If two solutions have the same cost, then ϵ will count the number of edges:

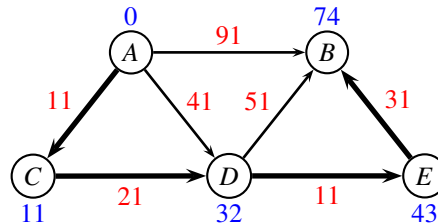


As can be seen, this is equivalent to the first transformation, but we need to use non-integer numbers in our calculations. ■

Answer 2 Since all costs are positive, we can use Dijkstra's algorithm. The following table states for each iteration which node is considered, and what the so far best distances are to each node. When a node has been considered, the distance is not changed any more.

considered node	distance d_i				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		91	11	41	
C				32	
D		83			43
E		74			
B					

The shortest path tree is marked with fat edges below (shortest distance d_i marked with blue):

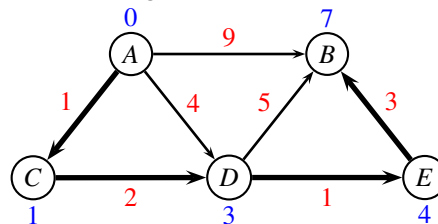


The first digit of d_i denotes distance, second digit denotes number of edges. ■

If Problem 1 was not solved, running Dijkstra's algorithm on original graph gives:

considered node	distance d_i				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		9	1	4	
C				3	
D		8			4
E		7			
B					

The shortest path tree is marked with fat edges below (shortest distance d_i marked with blue):



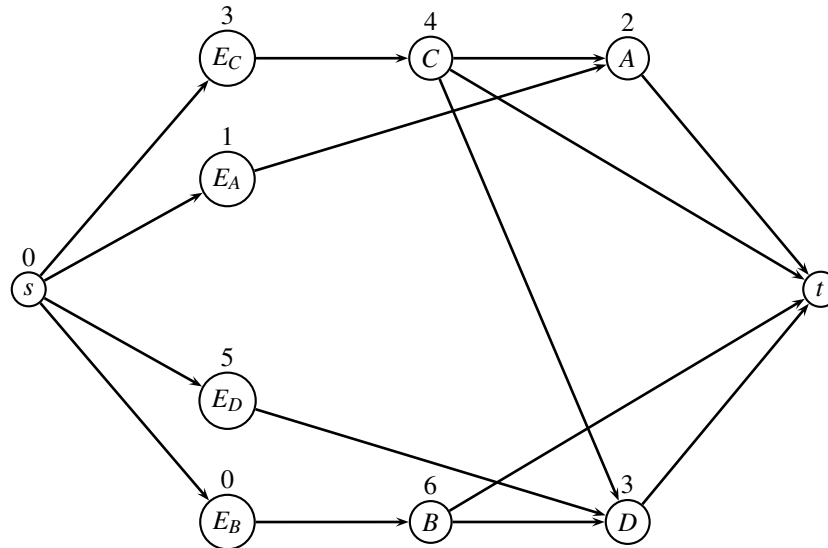
Critical path with time windows

Answer 3 For each activity i add an extra activity E_i with processing time e_i . Add E_i to the list of predecessors for activity i . All the activities E_i have s as predecessor. ■

Answer 4 With the above transformation, we get the problem:

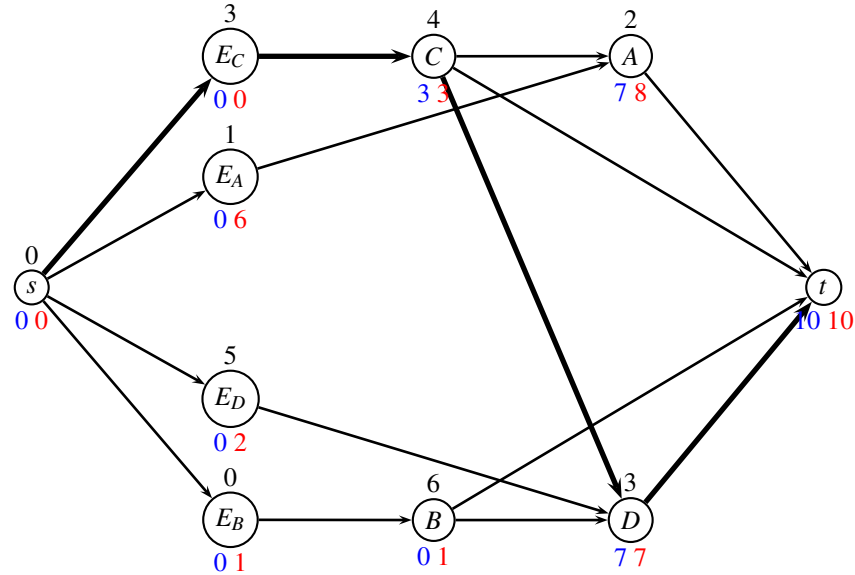
activity i	processing time p_i	predecessors
A	2	C, E_A
B	6	E_B
C	4	E_C
D	3	B, C, E_D
E_A	1	s
E_B	0	s
E_C	3	s
E_D	5	s

This leads to the topologically ordered graph, where processing time p_i is indicated on the nodes



Now we solve the problem

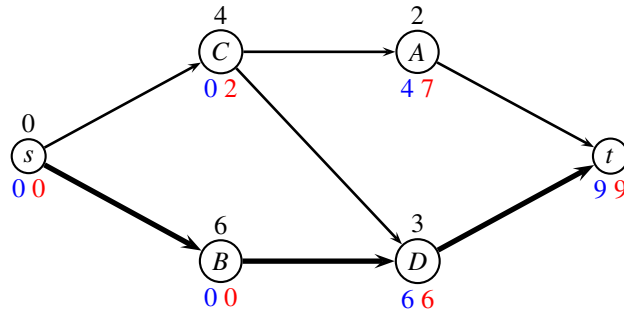
- *Forward procedure:* Consider the nodes j in topological order. Starting with $U_s = 0$, calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$. (U_j is indicated with blue)
- *Makespan:* $T = U_t = 10$.
- *Backward procedure:* Consider the nodes j in backward topological order. Starting with $V_t = T$, calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$. (V_j is indicated with red)



■

Answer 5 Nodes with $U_j = V_j$ define the critical path. It is indicated with fat edges on the figure. The critical path will not necessarily be on the tasks. As seen in the example, it can also be defined by the e_j nodes. ■

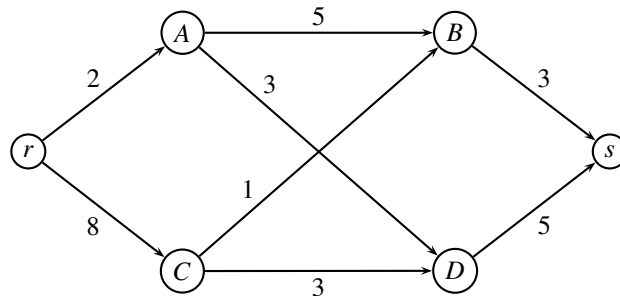
If Problem 3 was not solved, running critical path algorithm on original graph gives:



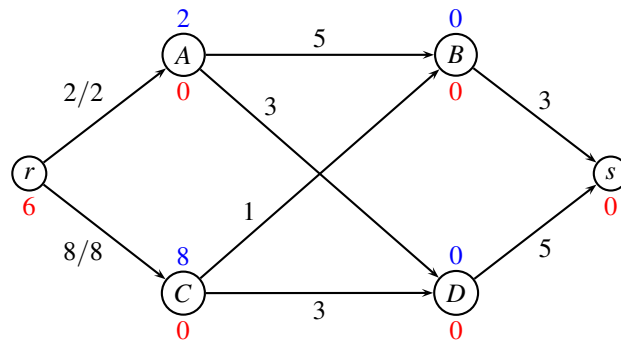
Makespan: $T = U_t = 9$. Critical path is indicated with fat edges.

Maximum flow problem

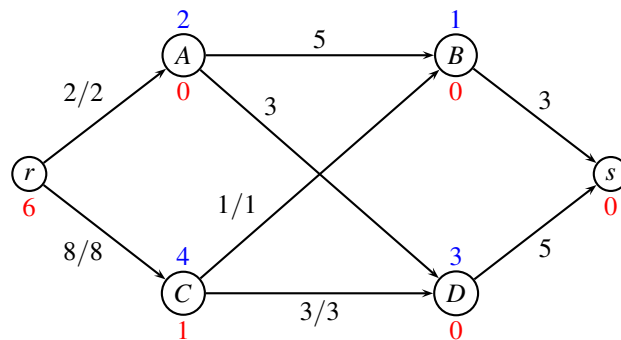
We have the maximum flow problem with capacity u_{ij} for each edge $(i, j) \in E$.



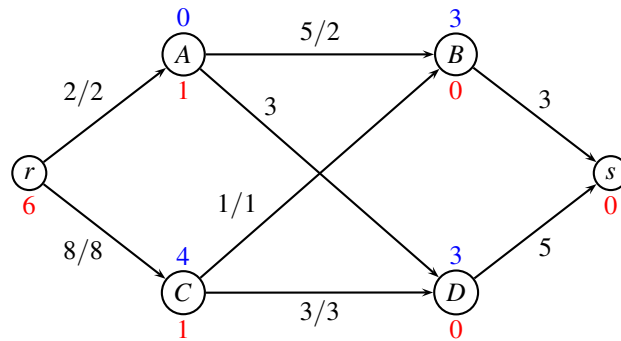
Answer 6 Initialize: For node i , red value is label d_i , blue value is flow excess $f_x(i)$



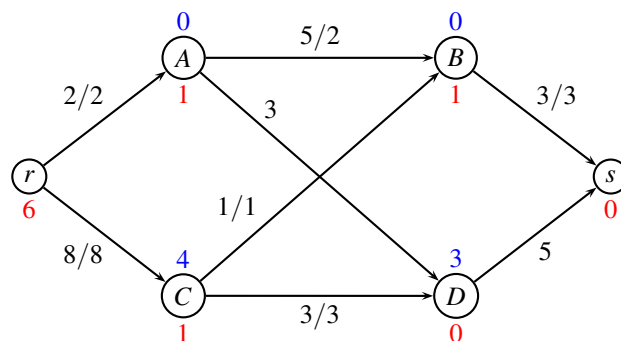
Iteration 1: relabel C, push



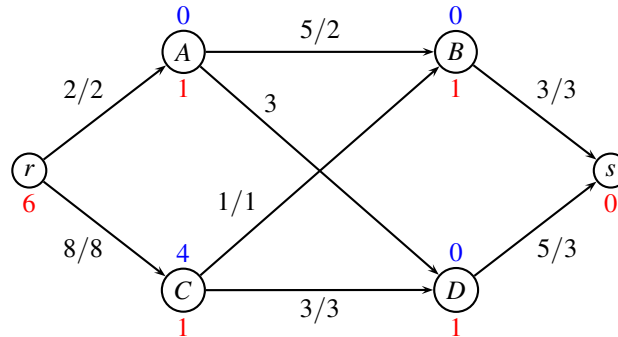
Iteration 2: relabel A, push



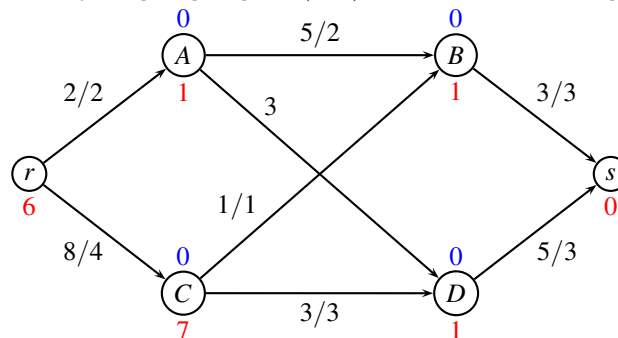
Iteration 3: relabel B, push



Iteration 4: relabel D , push



Iteration 5: relabel C (The only outgoing edge is (C, r) hence label is one bigger than label of r), push



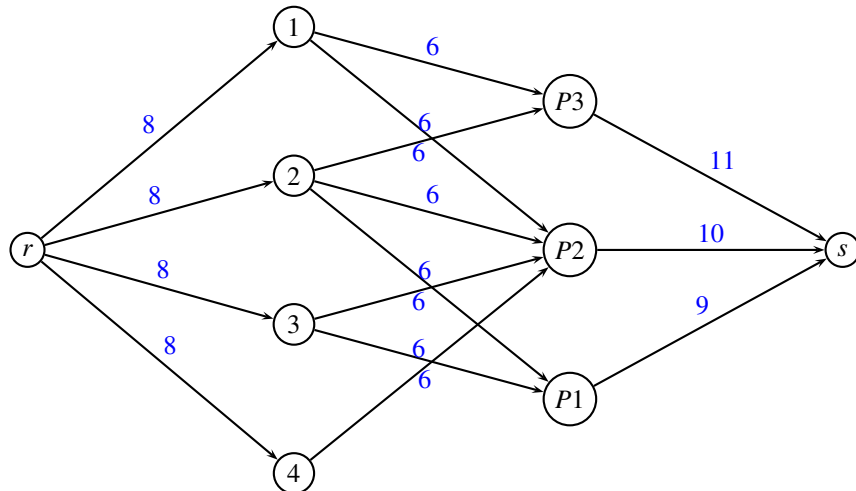
Flow is feasible, hence we stop. ■

Answer 7 A minimum cut is found by selecting an unused label value, e.g. $k = 2$. Then we have $R = \{i \mid d_i > k\} = \{r, C\}$ and $\bar{R} = \{i \mid d_i < k\} = \{A, B, D, s\}$. The capacity of the cut is $u(R) = 2 + 1 + 3 = 6$. ■

Workforce plan

Answer 8 We formulate the problem as a maximum flow problem: We introduce nodes 1, 2, 3, 4 corresponding to the four months m . Moreover we introduce nodes $P1, P2, P3$ for the projects. Each project Pi has an edge from month m if the month is within the the time window of the project. These edges have capacity 6 since only 6 engineers can work on a project in a given month. Each month m has an edge from s with capacity 8, since only 8 engineers are available each month. Finally, each project Pi has an edge to t with capacity equal to the required man-months.

The following network shows the resulting maximum flow problem:



If we can find a flow of value 30, it is possible to assign the correct assignment of engineers. The flow on edges between month m and project P_i say how many engineers should be assigned to the project in the given month. ■

Repositioning containers

Answer 9 We first calculate the demand of containers (in TEU) as

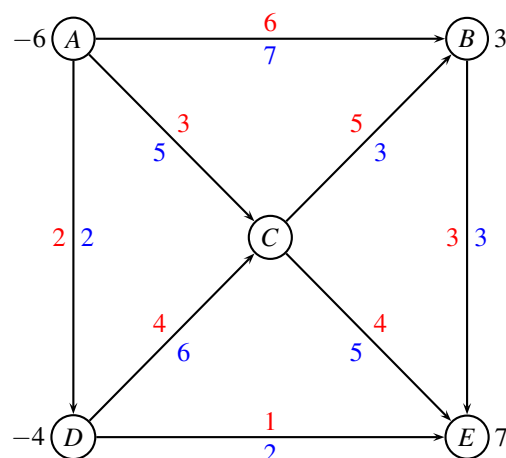
$$d_i = - \sum_{t=1,2,3} \sigma_{ti} \cdot w_i$$

where t is the container type (20", 40", high-cube) and w_i is the size of the container measured in TEU. This gives the demands

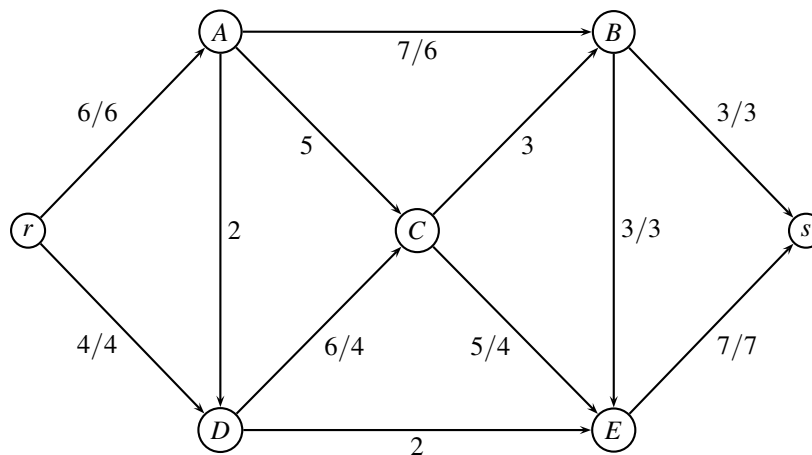
Node i	A	B	C	D	E
Demand d_i	-6	3	0	-4	7

Now, we can solving a minimum cost flow problem with the above demands. ■

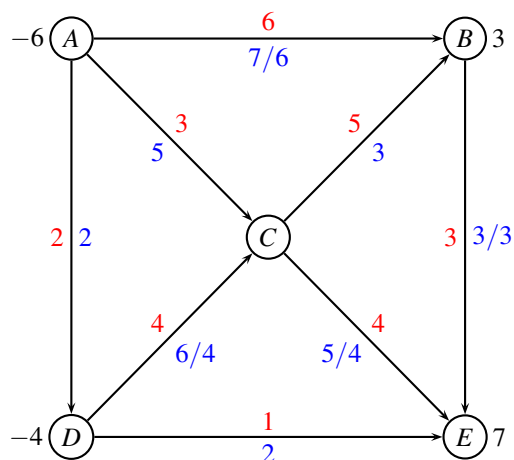
Answer 10 We have the following network



To find an initial solution, we solve a maximum flow problem. Augmenting paths: Path $rABs$ with $\delta = 3$. Path $rABEs$ with $\delta = 3$. Path $rDCEs$ with $\delta = 4$.



We now have the initial flow, and we can forget the demands:



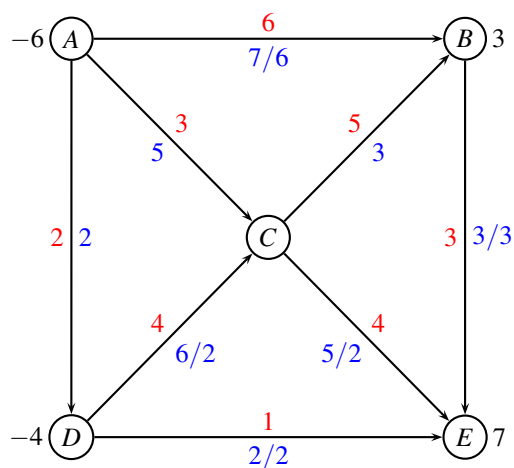
cost: $z = 6 \cdot 6 + 3 \cdot 3 + 4 \cdot 4 + 4 \cdot 4 = 77$

cycle: $C = \{DECD\}$

$w(C) = 1 - 4 - 4 = -7$

$\delta = \min\{2, 4, 4\} = 2$

Sending δ units along cycle C



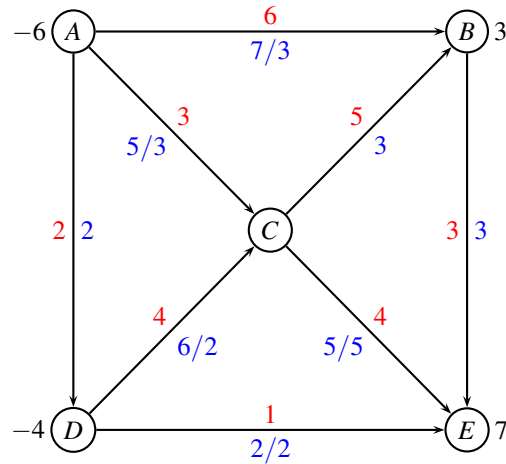
cost: $z = 6 \cdot 6 + 3 \cdot 3 + 4 \cdot 2 + 4 \cdot 2 + 2 \cdot 2 = 63$

cycle: $C = \{ACEBA\}$

$w(C) = 3 + 4 - 3 - 6 = -2$

$\delta = \min\{5, 3, 3, 6\} = 3$

Sending δ units along cycle C



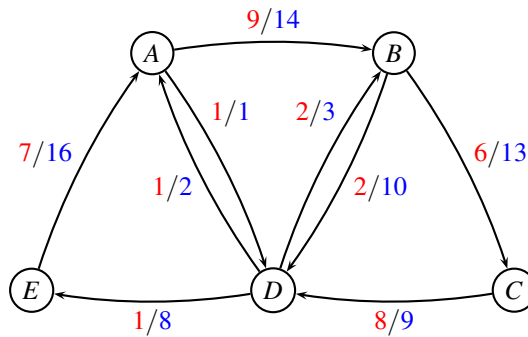
cost: $z = 6 \cdot 3 + 3 \cdot 3 + 4 \cdot 2 + 4 \cdot 5 + 2 \cdot 2 = 57$

no more cycles

stop ■

Multicommodity-flow problem

MEDITLINE (Route net of a Mediterranean liner shipping company)

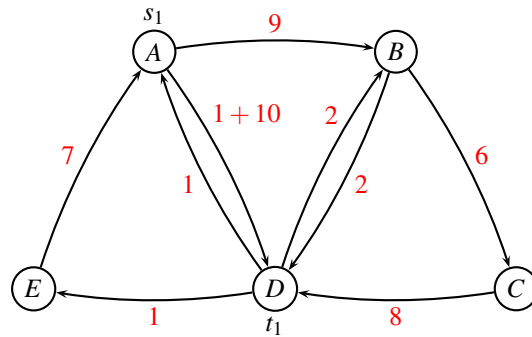


k	s_k	t_k	d_k	w_k
1	A	D	3	1
2	E	D	4	2
3	B	A	2	4

The dual variables are $y_{AD} = -10$ while $y_{ij} = 0$ for all other edges (i, j) . Moreover, $\bar{y}_1 = 11$, $\bar{y}_2 = 36$, $\bar{y}_3 = 100$.

Answer 11 For **commodity 1** we remove all edges having capacity less than $w_1 = 1$ getting the

shortest path problem:

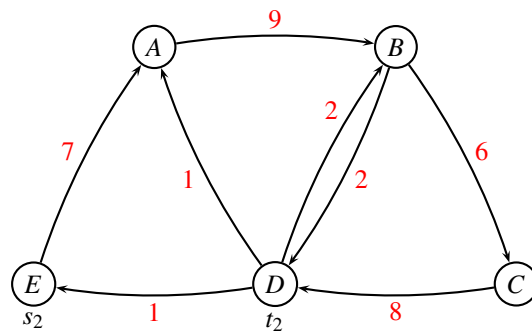


We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		9		11	
B			15		
D					12
E					
C					

The shortest path is AD with distance $z = 11$. The reduced cost is now $\bar{c}_1 = z \cdot w_1 - \bar{y}_1 = 11 \cdot 1 - 11 = 0$.

For **commodity 2** we remove all edges having capacity less than $w_2 = 2$ getting the shortest path problem:

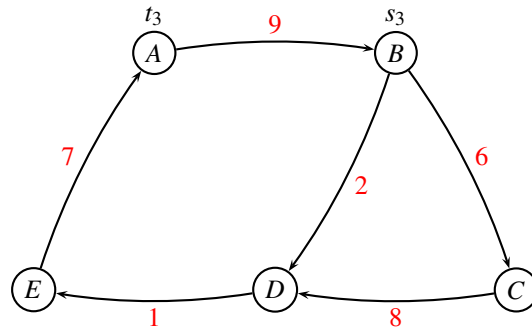


We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	∞	∞	∞	∞	0
E	7				
A		16			
B			22	18	
D					
C					

The shortest path is $EABD$ with distance $z = 18$. The reduced cost is now $\bar{c}_2 = z \cdot w_2 - \bar{y}_2 = 18 \cdot 2 - 36 = 0$.

For **commodity 3** we remove all edges having capacity less than $w_3 = 4$ getting the shortest path problem:



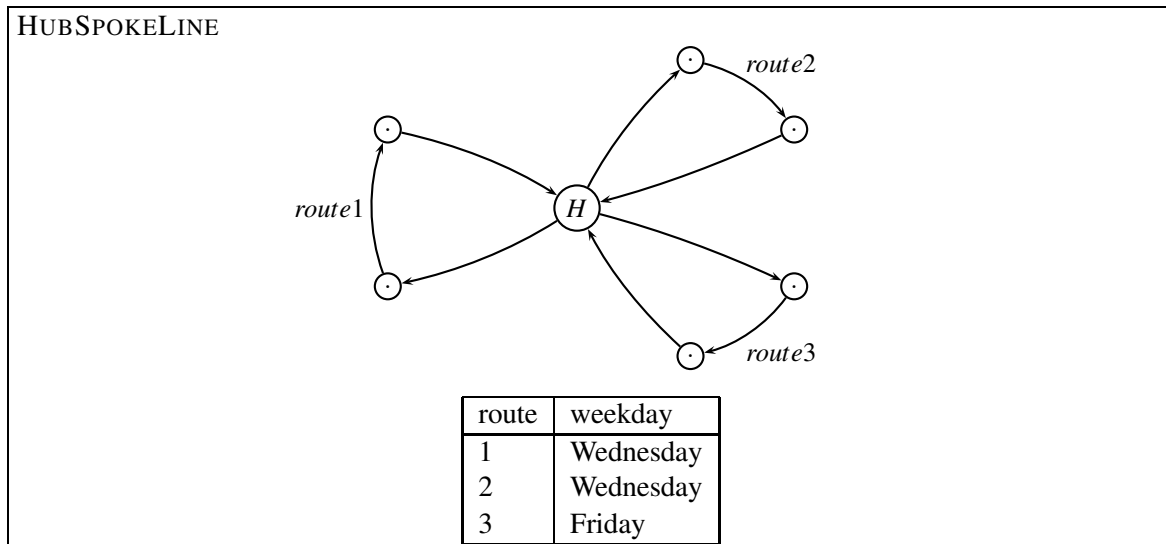
We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	∞	0	∞	∞	∞
B			6	2	
D					3
E					
C					
A					

The shortest path is $BDEA$ with distance $z = 10$. The reduced cost is now $\bar{c}_3 = z \cdot w_3 - \bar{y}_3 = 10 \cdot 4 - 100 = -60$. ■

Answer 12 The path $BDEA$ for commodity 3 should be added, since it has most negative reduced cost. ■

Modeling a hub port

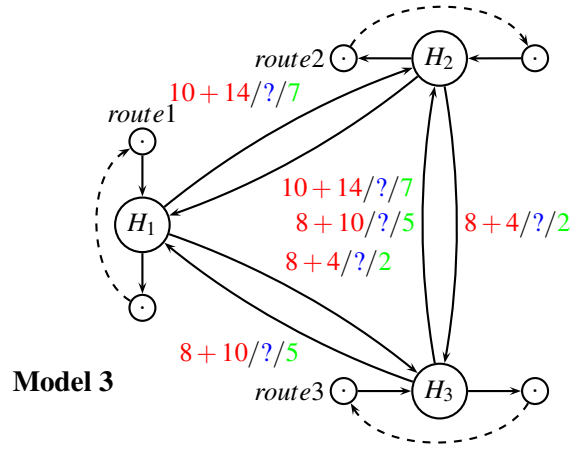


Answer 13 We consider the three models separately.

Model 3

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	yes	no	no	yes

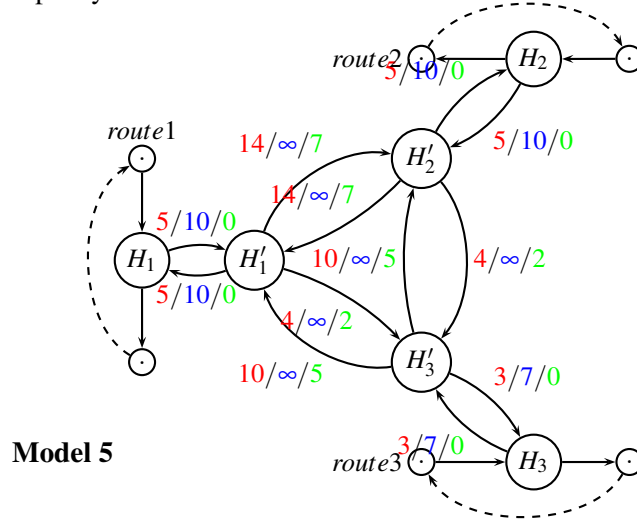
In this formulation it is possible to handle transshipment cost and time on edges between H_i and H_j . The time is set to the number of days between arrival of route i and route j . The cost is set to the unloading cost for route i plus the loading cost for route j . To the cost we add the storage cost for the given number of days. However we cannot handle crane capacity since e.g. edge (H_1, H_2) and edge (H_1, H_3) share the capacity of the crane. The same argument can be used to show that we cannot handle storage capacity.



Model 5

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	yes	yes	no	yes

In this formulation it is possible to handle transshipment cost and time, as well as crane capacity. Transshipment costs and crane capacity are assigned to edges between H_i and H'_i . The time (and storage cost) is assigned to edges between H'_i and H'_j as the number of days between arrival of route i and route j . However we cannot handle storage capacity since e.g. edge (H'_1, H'_2) and edge (H'_1, H'_3) share the same storage capacity.

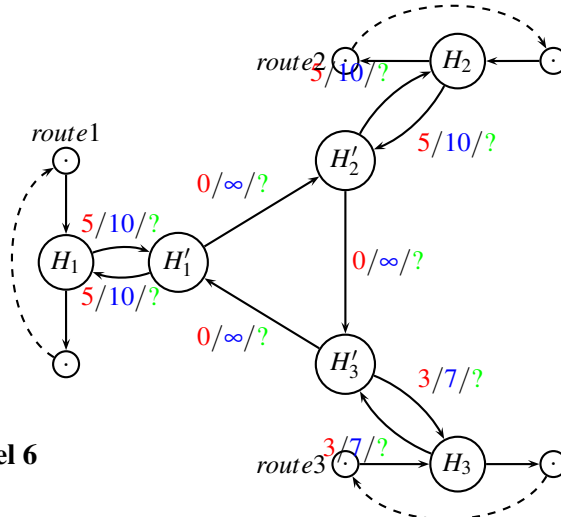


Model 5

Model 6

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	no	yes	no	no

We assume that the vessels are ordered according to increasing day of arrival. In this formulation it is possible to handle transshipment cost, crane capacity. Transshipment costs and crane capacity are assigned to edges between H_i and H'_i . Storage capacity and storage cost cannot be handled, since they need to be 0 from H'_1 to H'_2 as well as from H'_2 to H'_1 . Transshipment times cannot be modeled, since the time between H'_1 and H'_2 is 7 days, so a transshipment time from H'_3 to H'_2 (taking 5 days) cannot be modeled.



Model 6

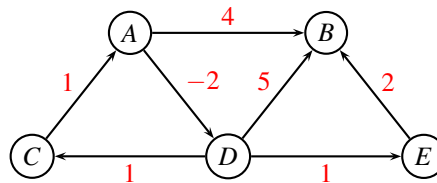
Solution, 17 December, 2020

Course title: Network Optimization

Course no: 42115

Shortest path problem

Consider the following directed graph where distances w_{ij} are marked with red. You are asked to find the shortest path from A to all other nodes.

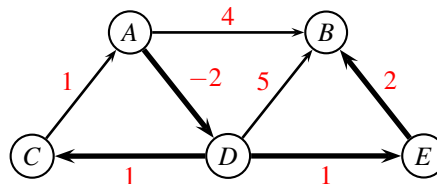


Answer 1 The single negative edge is only part of one cycle, $ADCA$ which has cost 0. So, there are no negative cycles, and the problem is well-defined. We can use Bellman-Ford, or we can use dynamic programming (All edges entering A can be removed, so the remaining problem becomes a DAG). ■

Answer 2 Using Bellman-Ford (considering edges top-down) we get

iteration	A	B	C	D	E
0	0	∞	∞	∞	∞
1	0	4	∞	-2	-1
2	0	1	-1	-2	-1
3	0	1	-1	-2	-1

We terminate after iteration 3, since the values do not change. The shortest-path tree is:



Using dynamic programming, we order the edges in topological ordering A, D, C, E, B , and use the recursion:

$$d_j = \min_{(i,j) \in E} (d_i + w_{ij})$$

In this way we find:

A	D	C	E	B
0	-2	-1	-1	1

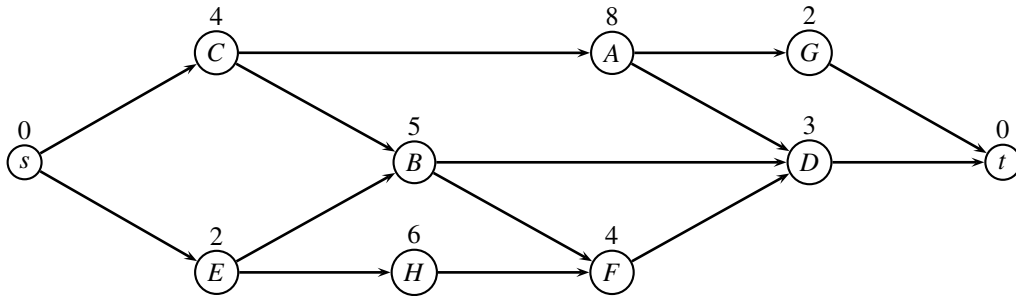
■

Project planning

Consider a critical path problem, in which each task i has a processing time p_i , and a list of predecessors P_i . The predecessors P_i need to finish before task i can begin.

task	predecessor(s)	processing time p_i
A	C	8
B	C, E	5
C		4
D	A, B, F	3
E		2
F	B, H	4
G	A	2
H	E	6

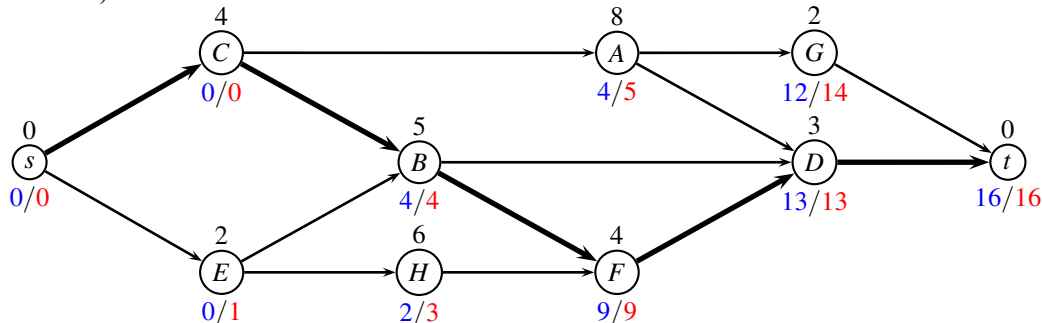
Answer 3 The topological ordering is found by repeatedly selecting a node i with no incoming edges and removing edges leaving i . Moreover, we add activity s before all activities, and t after all activities.



■

Answer 4 Now, we solve the problem

- *Forward procedure:* Starting at time zero $U_s = 0$, calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j) \in E} (U_i + d_i)$. (marked with blue)
- *Makespan:* $T = U_t = 16$.
- *Backward procedure:* Starting at time equal to the makespan $V_t = T$, calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} (V_j - d_i)$ (marked with red)

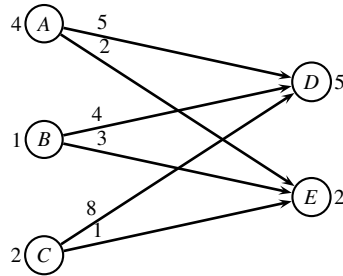


■

Answer 5 The critical path is found where $U_i = V_j$. The fat edges are marked on the above figure. ■

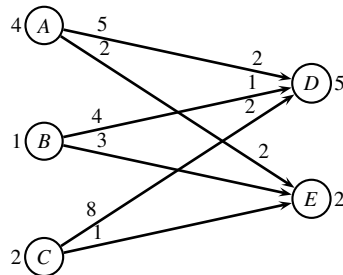
Transportation problem

Three factories A, B, C are transporting items to two customers D, E . The *transportation costs* of sending one item along an edge is indicated on the graph below. The production at factories are respectively $p_A = 4, p_B = 1, p_C = 2$, while the demands at the customers are $d_D = 5, d_E = 2$.



A greedy algorithm considers the factories in the order A, B, C , in each step sending the items to the customers in the cheapest possible way.

Answer 6 Considering the nodes A, B, C we find the greedy solution:

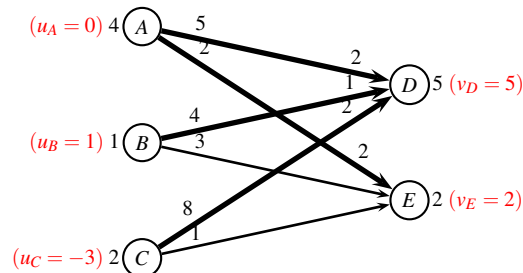


The cost of the greedy solution is

$$z = 5 \cdot 2 + 2 \cdot 2 + 4 \cdot 1 + 8 \cdot 2 = 34$$

■

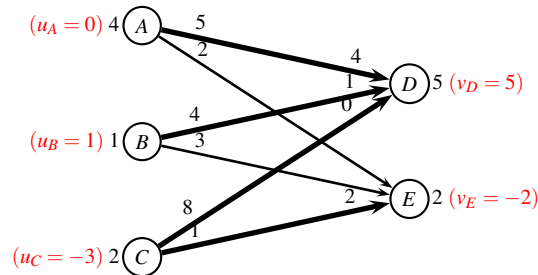
Answer 7 Edges having a flow form a tree (fat edges). We notice that there are $n + m - 1 = 3 + 2 - 1 = 4$ fat edges so the basis has the correct dimension. Calculate the dual variables (red) as the distance from A:



Reduced costs of thin edges is calculated as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, giving

$$\begin{aligned}\bar{c}_{BE} &= 3 + 1 - 2 = 2 \\ \bar{c}_{CE} &= 1 - 3 - 2 = -4\end{aligned}$$

Edge with most negative reduced cost is (C, E) . The edge imposes the cycle C, E, A, D, C . The capacity is $\theta = \min\{2, 2\} = 2$. Sending 2 units along the cycle, we get:



Edge (C, E) enters the tree, while edge (A, E) leaves the tree. We keep edge (C, D) in the tree although the flow is zero.

The dual variables (red) are calculated as distance from A. Reduced costs of thin edges are:

$$\begin{aligned}\bar{c}_{AE} &= 2 + 0 - (-1) = 3 \\ \bar{c}_{BE} &= 3 + 1 - (-1) = 5\end{aligned}$$

No edges have reduced cost, so we have found the optimal solution.

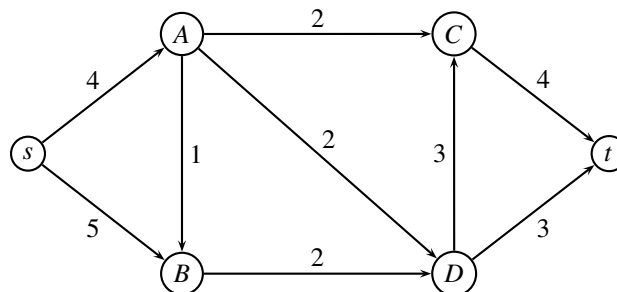
The cost of the optimal solution is

$$z = 5 \cdot 4 + 4 \cdot 1 + 8 \cdot 0 + 1 \cdot 2 = 26$$

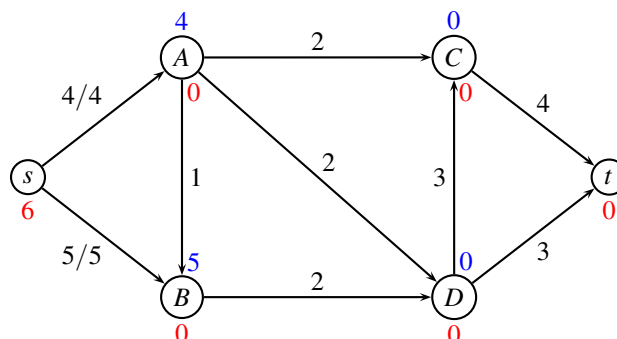
■

Maximum flow problem

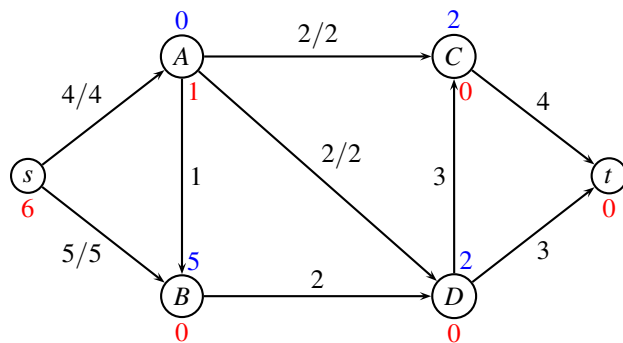
In the following network the edge weights denote the capacity. The problem is to find the maximum flow from s to t .



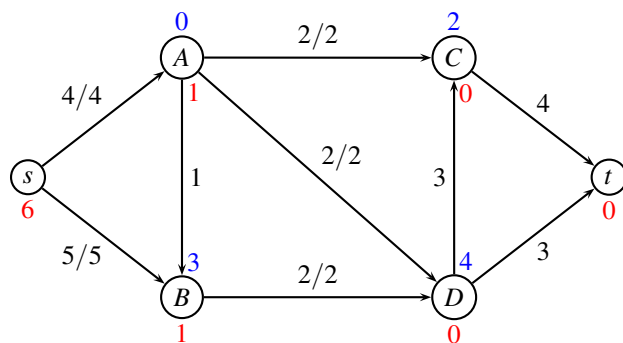
Answer 8 We use preflow-push. Initialize a valid preflow by setting labels $d_s = n = 6$ and remaining $d_i = 0$. (labels d_i are red, surplus $f_x(i)$ are blue)



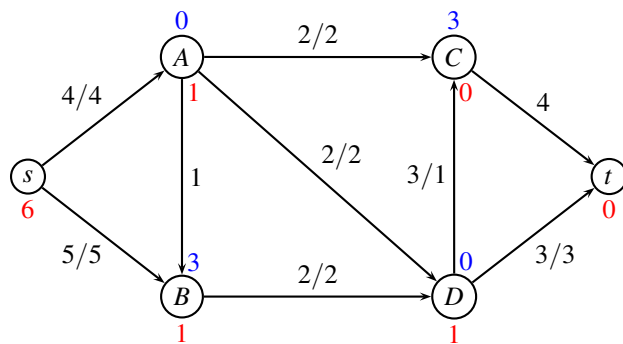
Relabel A to 1, push



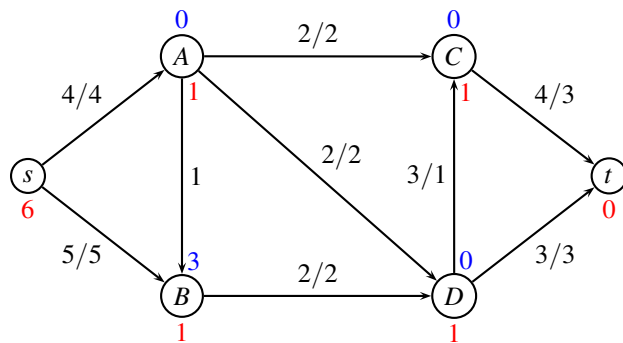
Relabel B to 1, push



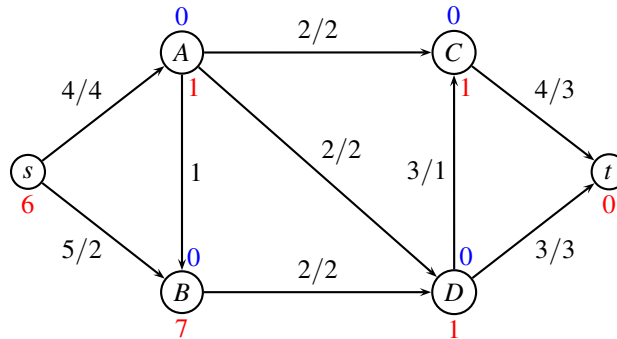
Relabel D to 1, push



Relabel C to 1, push



Relabel B to 7, push

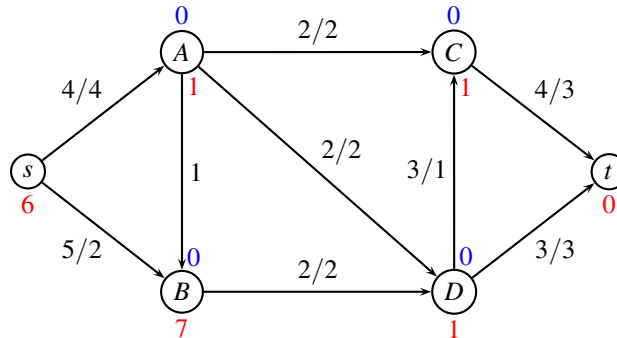


We have a valid flow, and hence the algorithm terminates. The maximum flow is $z = 6$.

If we use augmenting-path we get the following:

- Sending 2 units along path $sACt$
- Sending 2 units along path $sADt$
- Sending 1 unit along path $sBDt$
- Sending 1 unit along path $sBDCt$

This leads to the solution



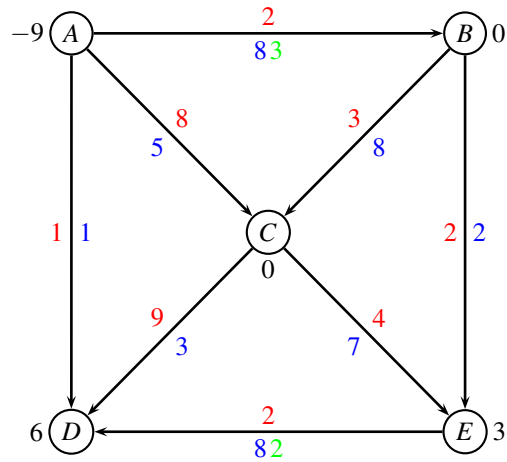
No more augmenting paths exist, so the algorithm stops. ■

Answer 9 If we used the preflow-push method: A minimum cut is found by choosing an unused label, e.g. $k = 2$. We then have $R = \{s, B\}$ and $\bar{R} = \{A, C, D, t\}$. The capacity of the cut is $4 + 2 = 6$.

If we used the augmenting-path method: We mark all nodes red that are reachable from s in the residual graph. These form the cut $R = \{s, B\}$ and $\bar{R} = \{A, C, D, t\}$. The capacity of the cut is $4 + 2 = 6$. ■

Repositioning of containers

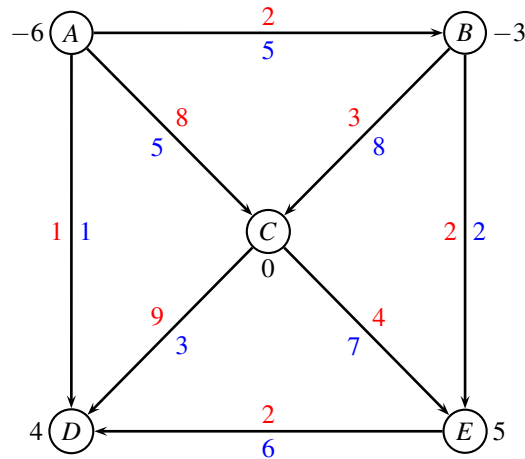
Consider the following network with edge costs c_{ij} (red) and edge capacities u_{ij} (blue). Moreover, we have lower bounds ℓ_{ij} (green) on some of the edges. The demand b_i (black) in every node is positive for a sink, and negative for a source.



Answer 10 Assume that edge (i, j) has upper bound u_{ij} and lower bound ℓ_{ij} .

- Replace the upper bound with $u_{ij} := u_{ij} - \ell_{ij}$
- Set the demand at node i to $b_i := b_i + \ell_{ij}$
- Set the demand at node j to $b_j := b_j - \ell_{ij}$

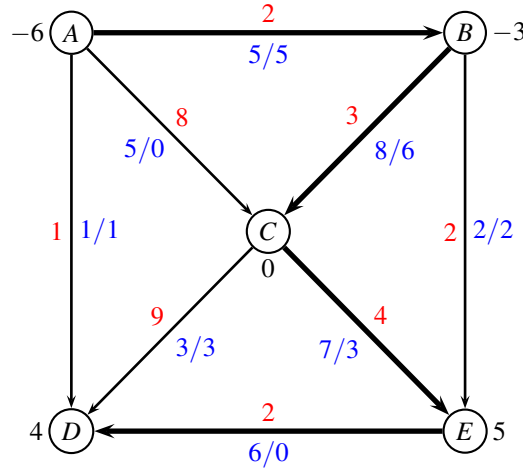
We use the transformation for edges (A, B) and (E, D) getting:



Now, consider the transformed problem. Having solved a maximum flow problem to find a feasible solution we are told that $T = \{(A, B), (B, C), (C, E), (E, D)\}$ and $U = \{(A, D), (B, E), (C, D)\}$ and $L = \{(A, C)\}$. Edges in T form a tree. The flow on edges in U is at the upper bound, the flow on edges in L is at the lower bound (i.e. zero).

Answer 11 Every partitioning of edges in T, U, L defines a unique flow as shown in the notes. In the

specific case we have



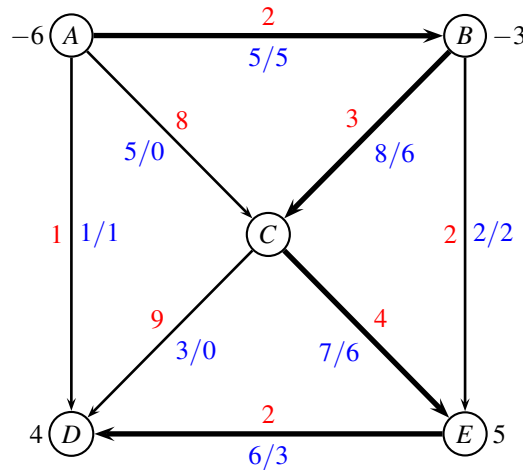
Notice that edges in T are allowed to be at the lower or upper bound as well as any value in-between. The solution value is $z = 2 \cdot 5 + 1 \cdot 1 + 3 \cdot 6 + 2 \cdot 2 + 9 \cdot 3 + 4 \cdot 3 = 72$. ■

Answer 12 We use Network simplex. Choosing A as root node, we find potentials (distances) $y_A = 0$, $y_B = 2$, $y_C = 5$, $y_E = 9$, $y_D = 11$. The reduced costs are found as $\bar{w}_{ij} = w_{ij} + y_i - y_j$, giving

$$\begin{aligned}\bar{w}_{AC} &= 8 + 0 - 5 = 3 && L \text{ edge} \\ \bar{w}_{AD} &= 1 + 0 - 11 = -10 && U \text{ edge} \\ \bar{w}_{BE} &= 2 + 2 - 9 = -5 && U \text{ edge} \\ \bar{w}_{CD} &= 9 + 5 - 11 = 3 && U \text{ edge}\end{aligned}$$

Since edge (C, D) is a U edge and it has positive reduced cost, we can improve the solution. The edge (C, D) imposes the cycle D, C, E, D (we are going in the backward direction of edge (C, D) since it is a U edge).

The capacity we can send along the cycle is $\theta = \min\{3, 4, 6\} = 3$. Sending three items along cycle D, C, E, D means that edge (C, D) becomes an L edge while the tree T is unchanged.

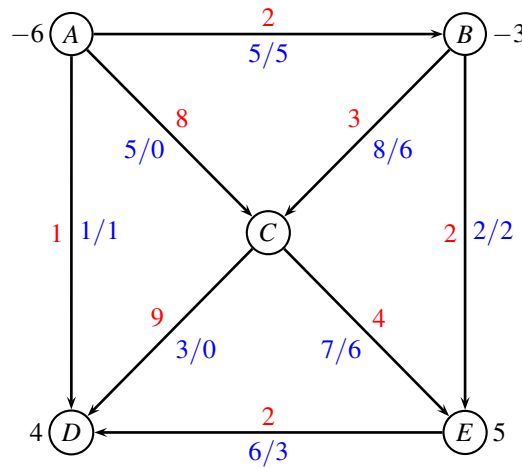


Since T is unchanged we find the same potentials (distances) $y_A = 0$, $y_B = 2$, $y_C = 5$, $y_E = 9$, $y_D = 11$. The reduced costs are also the same (but edge (C, D) is now an L edge)

$$\begin{aligned}
\bar{w}_{AC} &= 8 + 0 - 5 = 3 && L \text{ edge} \\
\bar{w}_{AD} &= 1 + 0 - 11 = -10 && U \text{ edge} \\
\bar{w}_{BE} &= 2 + 2 - 9 = -5 && U \text{ edge} \\
\bar{w}_{CD} &= 9 + 5 - 11 = 3 && L \text{ edge}
\end{aligned}$$

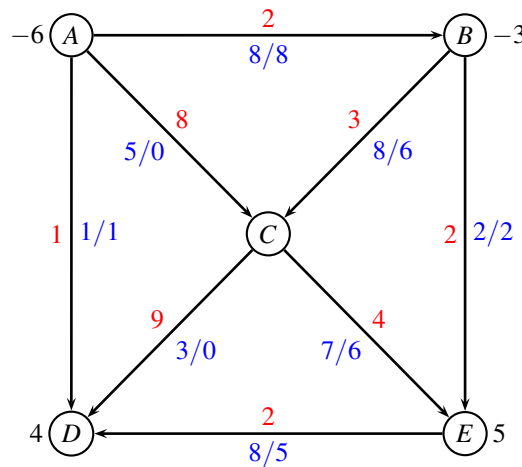
Since all U edges have negative reduced cost, and all L edges have positive reduced cost, we have found the optimal solution. The solution value is $z = 2 \cdot 5 + 1 \cdot 1 + 3 \cdot 6 + 2 \cdot 2 + 4 \cdot 6 + 2 \cdot 3 = 63$.

If we use the augmenting cycle method: Starting from the same initial solution, we find the augmenting cycle C, E, D, C with cost $4 + 2 - 9 = -3$. The capacity we can send along the cycle is $\theta = \min\{4, 6, 3\} = 3$. Sending 3 units along the cycle gives:



No more cycles with negative cost can be found, so the algorithm terminates. ■

Answer 13 We add the lower bounds to all edges $x_{ij} := x_{ij} + \ell_{ij}$ getting the solution



The solution value is $z = 2 \cdot 8 + 1 \cdot 1 + 3 \cdot 6 + 2 \cdot 2 + 4 \cdot 6 + 2 \cdot 5 = 73$.

Checking solution in CPLEX:

```
minimize
2 xAB + 8 xAC + 1 xAD + 3 xBC + 2 xBE + 9 xCD + 4 xCE + 2 xED

subject to
- xAB - xAC - xAD = -9
xAB - xBC - xBE = 0
xAC + xBC - xCD - xCE = 0
xAD + xCD + xED = 6
xBE + xCE - xED = 3

xAB <= 8
xAC <= 5
xAD <= 1
xBC <= 8
xBE <= 2
xCD <= 3
xCE <= 7
xED <= 8

xAB >= 3
xED >= 2

end

Dual simplex - Optimal: Objective = 7.3000000000e+01
Solution time = 0.01 sec. Iterations = 2 (2)
Deterministic time = 0.02 ticks (1.47 ticks/sec)

CPLEX> dis sol var -
Variable Name          Solution Value
xAB                    8.000000
xAD                    1.000000
xBC                    6.000000
xBE                    2.000000
xCE                    6.000000
xED                    5.000000
All other variables in the range 1-8 are 0.
```

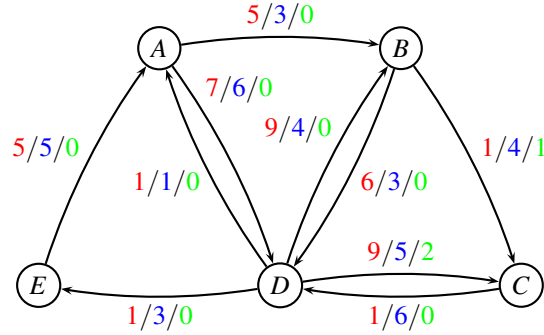
■

Flowing containers in a liner shipping network

The below instance, BALTICLINE, shows a small liner shipping network. Nodes represent ports, and edges represent legs sailed by vessels. The capacities indicate the remaining capacity of the vessels after some mandatory orders have been fulfilled. Therefore, the capacity may vary for each leg. **Note:** The network is the same as in *Project Assignment 2020*, but the (source,terminal) pairs for each commodity is changed.

In mathematical terms, we are given a weighted oriented graph $G = (V, E, c, u, \ell)$ where each edge $(i, j) \in E$ has an associated cost (per container) $c_{ij} \geq 0$, a capacity $u_{ij} \geq 0$, and a lower bound $\ell_{ij} \geq 0$.

BALTICLINE (Route net of a Baltic liner shipping company)



For edge (i, j) the first number (red) is the cost c_{ij} , the second number (blue) is the capacity u_{ij} , and the third number (green) is the lower bound ℓ_{ij} .

We have the demands

k	s_k	t_k	d_k
1	D	C	2
2	D	B	1
3	A	E	3

Commodity $k = 1$ is shipping two containers from D to C , commodity $k = 2$ is shipping one container from D to B , and commodity $k = 3$ is shipping three containers from A to E . The commodities should be transported *unsplittable*, i.e. all d_k units should follow the same path.

Answer 14 We have the routes r outlined in the below table. The cost of sending d_k units along a route r is calculated as $d_k \sum_{(i,j) \in r} c_{ij}$.

route r	commodity k	nodes in route	d_k	$\sum c_{ij}$	cost
1	1	DBC	2	10	20
2	1	DC	2	9	18
3	1	$DEABC$	2	12	24
4	2	DAB	1	6	6
5	2	DB	1	9	9
6	2	$DEAB$	1	11	11
7	3	$ABCDE$	3	8	24
8	3	$ABDE$	3	12	36
9	3	ADE	3	8	24

■

Answer 15

We introduce the binary variable x_i for route number i . Using this notation the path formulation for BALTICLINE becomes:

```

minimize
+20x1+18x2+24x3+6x4+9x5+11x6+24x7+36x8+24x9
subject to
UAB:      +2x3+1x4      +1x6+3x7+3x8      <= 3
UAD:      +3x9 <= 6
UBC: +2x1      +2x3      +3x7      <= 4
UBD:      +3x8 <= 3
UCD:      +3x7 <= 6
UDA:      +1x4 <= 1
UDB: +2x1      +1x5      <= 4
UDC:      +2x2 <= 5
UDE:      +2x3      +1x6+3x7+3x8+3x9 <= 3
UEA:      +2x3      +1x6 <= 5

LBC: +2x1      +2x3      +3x7      >= 1
LDC:      +2x2 >= 2

K1: +x1+x2+x3 = 1
K2: +x4+x5+x6 = 1
K3: +x7+x8+x9 = 1

binary
x1 x2 x3 x4 x5 x6 x7 x8 x9
end

CPLEX> opt
MIP - Integer optimal solution: Objective = 5.1000000000e+01

CPLEX> dis sol var -
Incumbent solution
Variable Name      Solution Value
x2                  1.000000
x5                  1.000000
x7                  1.000000
All other variables in the range 1-9 are 0.

```

■

Now, assume that we want to solve the LP-relaxed version of the path formulation with column generation. After a few iterations of the algorithm (using dummy routes as initial columns) we have the following dual solution:

Dual variables for the upper bound constraints: $y_{AB} = -2$, $y_{DA} = -3$. Dual variables for the lower bound constraints: $y'_{BC} = 7$, $y'_{DC} = 2$. Dual variables for the commodity constraints: $\bar{y}_1 = 8$, $\bar{y}_3 = 2$. All other dual variables are zero.

Answer 16 The reduced cost of a route is calculated as

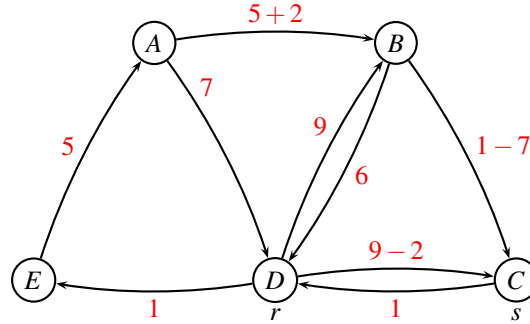
$$\bar{c}_r^k = \sum_{(i,j) \in r} d_k c_{ij} - \sum_{(i,j) \in r} d_k y_{ij} - \sum_{(i,j) \in r} d_k y'_{ij} - \bar{y}_k$$

Which can be written as

$$d_k \sum_{(i,j) \in r} (c_{ij} - y_{ij} - y'_{ij}) - \bar{y}_k$$

We therefore first remove edges with capacity less than d_k , in our case edge (D,A) . Next, we set the

edge costs to $c'_{ij} = c_{ij} - y_{ij} - y'_{ij}$. In this way we get the graph:



Bonus info: The shortest path from D to C has cost $z = 3$. The reduced cost is then

$$\bar{c}_r^k = d_k \cdot z - \bar{y}_k = 2 \cdot 3 - 8 = -2$$

■

Organizing a conference

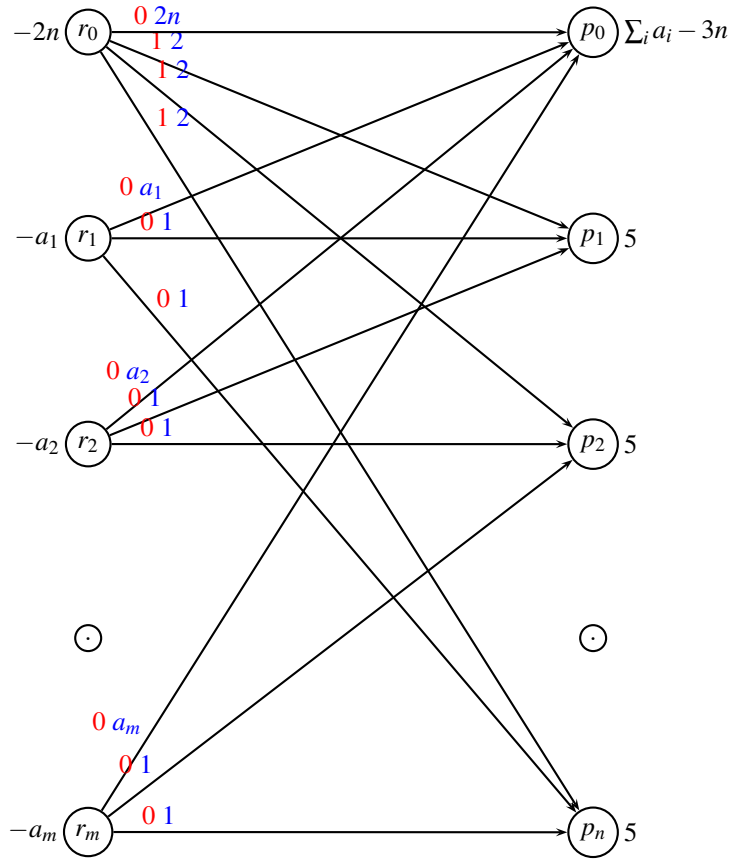
Suppose you are organizing a conference where researchers present articles they have written. Researchers who want to present an article send a paper to the conference organizers. The conference organizers have access to a committee of reviewers $R = \{r_1, \dots, r_m\}$. Each reviewer r_i is willing to read up to a_i articles. There are $P = \{p_1, \dots, p_n\}$ paper submissions, and each paper must be reviewed by 3 to 5 distinct reviewers (more reviewers is better). Moreover, each paper submission has a particular topic and each reviewer has a specialization for a set of topics, so papers on a given topic only get reviewed by those reviewers who are experts on that topic. The conference organizers need to decide which reviewers will review each article.

Answer 17 We formulate the problem as a *minimum cost flow problem* defined on a bi-partite graph consisting of nodes R and P . We add an edge with capacity 1 and cost 0 from reviewer r_i to paper p_j if the reviewer is qualified to read the paper. Each paper p_j has a demand of 5. Each reviewer r_i has a demand of $-a_i$.

A dummy reviewer r_0 is used to fill reviewers up to the 5 needed. We add an edge from dummy reviewer r_0 to each paper p_j having capacity 2 and cost 1. Dummy reviewer r_0 has demand $-2n$.

A dummy paper p_0 is used to take the surplus of reviewers. We add an edge from each reviewer r_i to dummy paper p_0 having capacity a_i and cost 0. Dummy paper p_0 has demand $(\sum_i a_i + 2n) - 5n = \sum_i a_i - 3n$.

Finally, we add an edge from the dummy reviewer r_0 to the dummy paper p_0 with capacity $2n$ and cost 0. This edge takes a possible surplus of dummy reviewers.



After having solved the minimum cost flow problem, the *assignment of reviewers to papers* is given by the flow on edges from R to P . If no solution can be found, it is not possible to satisfy the constraints. In this case, the capacity on edges from the dummy reviewer r_0 to each paper p_j can be increased, and the model re-solved. This will mean that fewer reviewers may be assigned to each paper. ■

end of assignment

Written exam 15 December, 2021

Course title: Network Optimization

Course no: 42115

Aids allowed: All

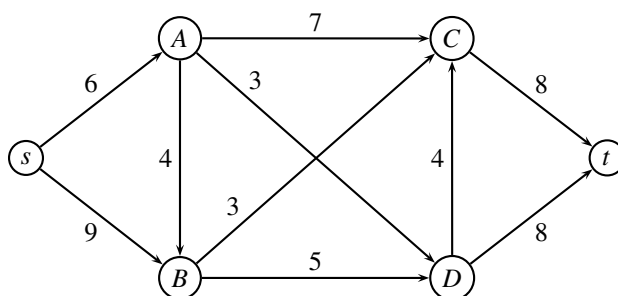
Exam duration: 4 hours

Weighting: The points are indicated at each question

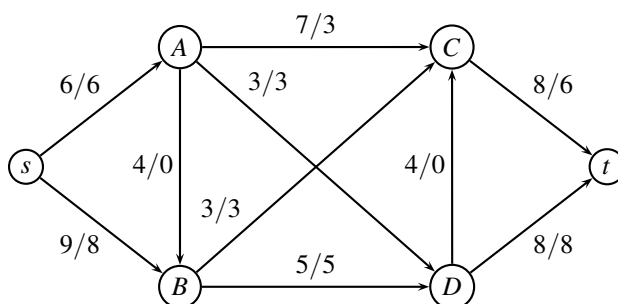
Language: You can write in Danish or English

Maximum flow problem

In the following network the edge weights denote the capacity. The problem is to find the maximum flow from s to t .

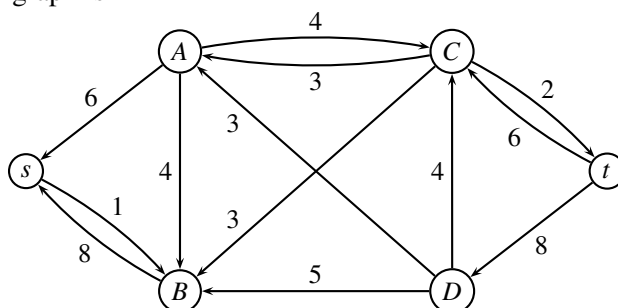


Answer 1 We use the paths $sACt$ with capacity 6, $sBDt$ with capacity 5, and $sBCADt$ with capacity 3. This gives the flow



The flow is $6 + 5 + 3 = 14$. ■

Answer 2 The residual graph is



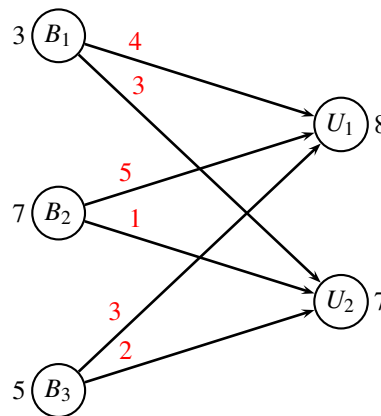
The cut is found by marking all nodes that are reachable from s . This is the set $S = \{s, B\}$. The corresponding cut has capacity $6 + 3 + 5 = 14$. The maximum flow is also $6 + 8 = 14$. ■

Transportation problem

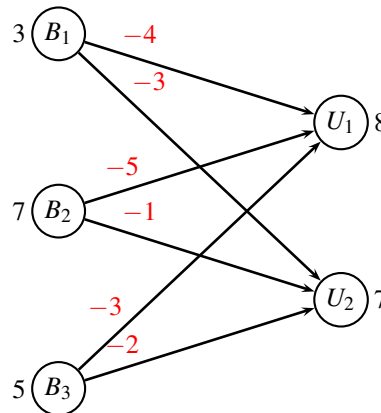
Three bakeries B_1, B_2, B_3 are delivering cakes to two universities U_1 , and U_2 . The bakeries are producing 3, 7 and 5 cakes respectively. University U_1 needs 8 cakes, while university U_2 needs 7 cakes. The profit of the cakes depends on the combination of bakery and university as given in the following matrix:

	B_1	B_2	B_3	demand
U_1	4	5	3	8
U_2	3	1	2	7
production	3	7	5	

We can formulate the problem as a transportation problem in maximization form:



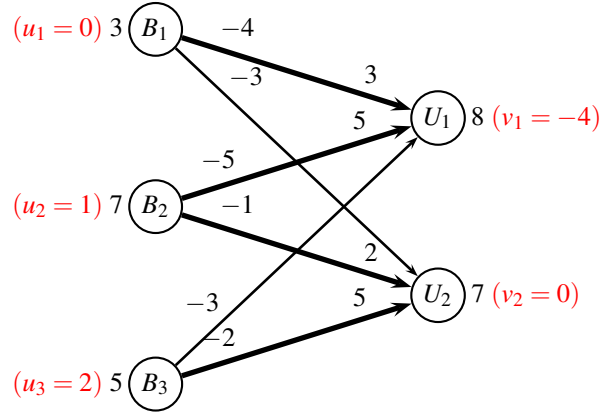
Answer 3 We can just multiply all costs with -1 since maximizing z is equivalent to minimizing $-z$.



Note, that the network simplex algorithm also works with negative costs.

Alternative solution: We can also first change sign, and then add a constant to all costs, to make them positive. Adding a constant will not affect the optimal solution vector (however, the constant should be subtracted from the objective value at the end of the solution algorithm). ■

Answer 4 We find the following greedy solution



The cost of the solution is $(-4) \cdot 3 + (-5) \cdot 5 + (-1) \cdot 2 + (-2) \cdot 5 = -49$.

Bold edges on the graph indicate the basis. Dual values (indicated with red) are found by finding the shortest path from node B_1 , using only the bold edges.

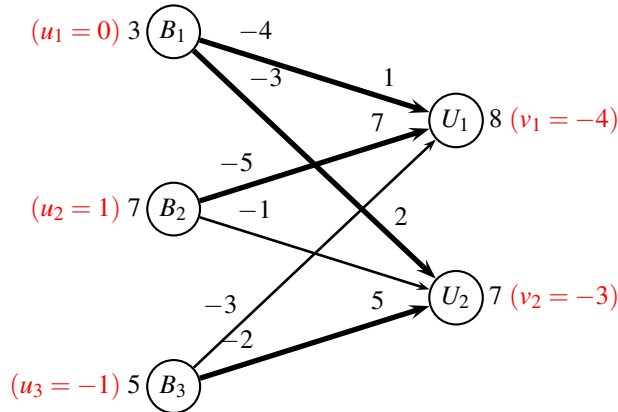
■

Answer 5 We now find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, getting

$$\begin{aligned}\bar{c}_{12} &= -3 + 0 - 0 = -3 \\ \bar{c}_{31} &= -3 + 2 - (-4) = 3\end{aligned}$$

We choose edge (B_1, U_2) corresponding to the cycle (B_1, U_2, B_2, U_1) . We can send at most $\delta = \min\{2, 3\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $(-4) \cdot 1 + (-3) \cdot 2 + (-5) \cdot 7 + (-2) \cdot 5 = -55$.

The reduced costs are

$$\begin{aligned}\bar{c}_{22} &= -1 + 1 - (-3) = 3 \\ \bar{c}_{31} &= -3 + (-1) - (-4) = 0\end{aligned}$$

No negative reduced costs, so we terminate. The optimal solution value is $z_{\min} = -55$. ■

Answer 6 The flow on the edges is the same. The objective is $z_{\max} = -z_{\min} = 55$. ■

Checking solution in CPLEX

```
maximize
  4xB1U1+3xB1U2
+5xB2U1+1xB2U2
+3xB3U1+2xB3U2
subject to
  B1: xB1U1 + xB1U2 = 3
  B2: xB2U1 + xB2U2 = 7
  B3: xB3U1 + xB3U2 = 5
  U1: xB1U1 + xB2U1 + xB3U1 = 8
  U2: xB1U2 + xB2U2 + xB3U2 = 7
end
```

```
CPLEX> read transport
Problem 'transport.lp' read.
Read time = 0.00 sec. (0.00 ticks)
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 2 rows and 3 columns.
Aggregator did 3 substitutions.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal: Objective = 5.5000000000e+01
Solution time = 0.00 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (5.07 ticks/sec)
```

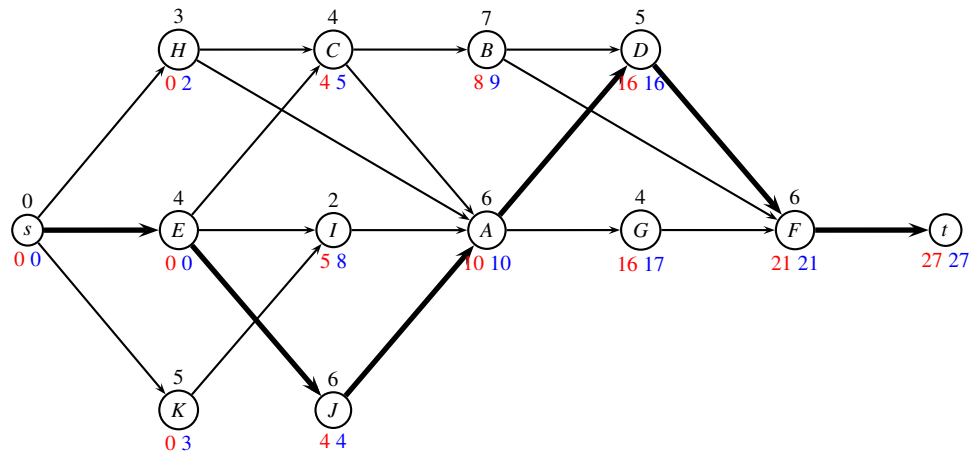
```
CPLEX> dis sol var -
Variable Name          Solution Value
xB1U2                  3.000000
xB2U1                  7.000000
xB3U1                  1.000000
xB3U2                  4.000000
All other variables in the range 1-6 are 0.
```

Critical path

Consider a critical path problem, in which each task i has a duration d_i , and a list of predecessors P_i . The predecessors P_i need to finish before task i can begin.

task i	duration d_i	predecessors P_i
A	6	C,H,I,J
B	7	C
C	4	E,H
D	5	A,B
E	4	
F	6	B,D,G
G	4	A
H	3	
I	2	E,K
J	6	E
K	5	

Answer 7 A topological ordering of the activities must ensure that all edges are pointing forward. This is the case in the following graph:



■

Answer 8 Now, we solve the problem

- *Forward procedure:* Starting at time zero ($U_s = 0$), calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j) \in E} (U_i + d_i)$. (marked with **red**)
- *Makespan:* $T = U_t = 27$.
- *Backward procedure:* Starting at time equal to the makespan ($V_t = T$), calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} (V_j - d_i)$ (marked with **blue**)

■

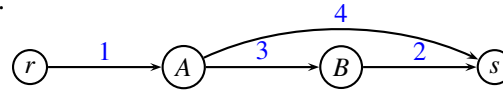
Answer 9 The critical path is found where $U_i = V_j$. The bold edges are marked on the above figure.

■

Max flow and min cut

Answer 10 We have the following two statements

- a) **NO.** The following example shows a case where there are two different optimal solutions: Either 1 unit $rABs$ or 1 unit rAs .



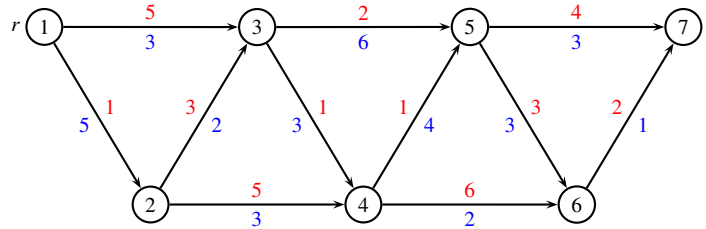
- b) **YES.** If $(R, \bar{R})$ is the smallest among all cuts, then it will also be the smallest among all cuts when multiplying all edge weights by a positive constant.

Alternative solution: Let P be the original problem and P' the problem where all capacities are multiplied by $k > 0$. If (x_{ij}) is an optimal solution to P then the flow on all edges $(i, j) \in R \times \bar{R}$ is equal to the capacity u_{ij} . Now, $(k \cdot x_{ij})$ is also an optimal solution to P' and the flow on all edges $(i, j) \in R \times \bar{R}$ is equal to the capacity $k \cdot u_{ij}$. Hence, $(R, \bar{R})$ is a minimum cut.

■

Resource constrained shortest path (bidirectional method)

Consider the following resource constrained shortest path problem, where costs are **red**, and times are **blue**.



We want to find the shortest (cheapest) path from node $r = 1$ to node $s = 7$, with time constraint $T = 14$.

We first run forward dynamic programming where we stop extending states when their resource is above $T/2$.

node 1	node 2	node 3	node 4	node 5	node 6	node 7																																		
<table><tr><td>cost</td><td>time</td></tr><tr><td>0</td><td>0</td></tr></table>	cost	time	0	0	<table><tr><td>cost</td><td>time</td></tr><tr><td>1</td><td>5</td></tr></table>	cost	time	1	5	<table><tr><td>cost</td><td>time</td></tr><tr><td>4</td><td>7</td></tr><tr><td>5</td><td>3</td></tr></table>	cost	time	4	7	5	3	<table><tr><td>cost</td><td>time</td></tr><tr><td>5</td><td>10</td></tr><tr><td>6</td><td>6</td></tr></table>	cost	time	5	10	6	6	<table><tr><td>cost</td><td>time</td></tr><tr><td>6</td><td>13</td></tr><tr><td>7</td><td>9</td></tr></table>	cost	time	6	13	7	9	<table><tr><td>cost</td><td>time</td></tr><tr><td>12</td><td>8</td></tr></table>	cost	time	12	8	<table><tr><td>cost</td><td>time</td></tr><tr><td></td><td></td></tr></table>	cost	time		
cost	time																																							
0	0																																							
cost	time																																							
1	5																																							
cost	time																																							
4	7																																							
5	3																																							
cost	time																																							
5	10																																							
6	6																																							
cost	time																																							
6	13																																							
7	9																																							
cost	time																																							
12	8																																							
cost	time																																							

We then run backward dynamic programming where we also stop extending states when their resource is above $T/2$.

node 1	node 2	node 3	node 4	node 5	node 6	node 7																																				
<table><tr><td>cost</td><td>time</td></tr><tr><td>14</td><td>9</td></tr></table>	cost	time	14	9	<table><tr><td>cost</td><td>time</td></tr><tr><td>10</td><td>10</td></tr><tr><td>12</td><td>8</td></tr><tr><td>13</td><td>6</td></tr></table>	cost	time	10	10	12	8	13	6	<table><tr><td>cost</td><td>time</td></tr><tr><td>6</td><td>9</td></tr><tr><td>9</td><td>6</td></tr></table>	cost	time	6	9	9	6	<table><tr><td>cost</td><td>time</td></tr><tr><td>5</td><td>7</td></tr><tr><td>8</td><td>3</td></tr></table>	cost	time	5	7	8	3	<table><tr><td>cost</td><td>time</td></tr><tr><td>4</td><td>3</td></tr></table>	cost	time	4	3	<table><tr><td>cost</td><td>time</td></tr><tr><td>2</td><td>1</td></tr></table>	cost	time	2	1	<table><tr><td>cost</td><td>time</td></tr><tr><td>0</td><td>0</td></tr></table>	cost	time	0	0
cost	time																																									
14	9																																									
cost	time																																									
10	10																																									
12	8																																									
13	6																																									
cost	time																																									
6	9																																									
9	6																																									
cost	time																																									
5	7																																									
8	3																																									
cost	time																																									
4	3																																									
cost	time																																									
2	1																																									
cost	time																																									
0	0																																									

Answer 11 We merge forward and backward states in all nodes, deleting states that exceed time constraint T . Infeasible states are marked with brackets. Dominated (but feasible) states are marked with paranthesis.

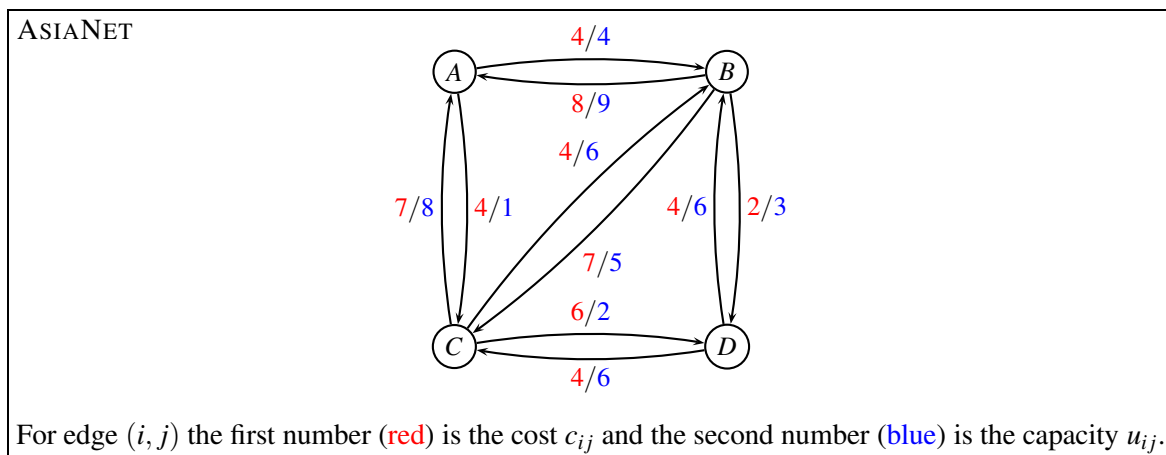
node 1	node 2	node 3	node 4	node 5	node 6	node 7
cost time	cost time	cost time	cost time	cost time	cost time	cost time
14 9	[11 15] 13 13 14 11	[10 16] 11 12 (13 13) 14 9	[10 17] 11 13 (13 13) 14 9	[10 16] 11 12	14 9	

■

Answer 12 The shortest (cheapest) path is found as: (cost,time) = (11,12). ■

Repositioning of empty containers

We are considering the liner shipping network ASIANET from the project assignment. The network, costs, and capacities are **unchanged**.



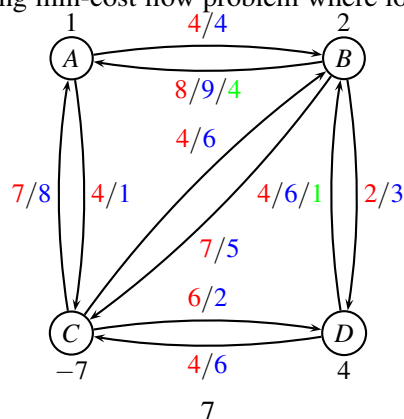
Due to the imbalance in trade, we need to reposition empty containers. For ASIANET, we have the following demand for containers:

node	A	B	C	D
demand	1	2	-7	4

(*)

Negative demands denote a surplus of containers. It is assumed that all empty containers are equal. Furthermore, we have some **new lower bounds** on the edges: Edge (B, A) has a lower bound of $\ell_{BA} = 4$, and edge (D, B) has a lower bound of $\ell_{DB} = 1$.

Answer 13 We have the following min-cost flow problem where lower bounds are green.



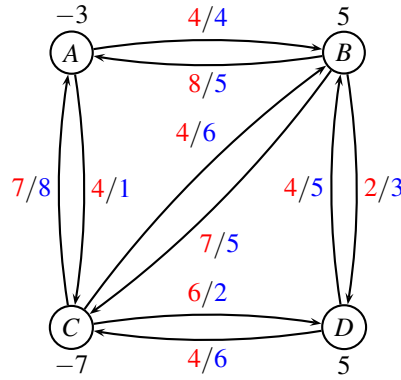
We use the following transformation to handle lower bounds: Assume that edge (i, j) has upper bound u_{ij} and lower bound ℓ_{ij} . Replace the upper bound with $u_{ij} := u_{ij} - \ell_{ij}$. Set the demand at node i to $b_i := b_i + \ell_{ij}$. Set the demand at node j to $b_j := b_j - \ell_{ij}$. In our case, we get

$$u_{BA} := u_{BA} - \ell_{BA} = 9 - 4 = 5, \quad b_B := b_B + \ell_{BA} = 2 + 4 = 6, \quad b_A := b_A - \ell_{BA} = 1 - 4 = -3$$

next we have

$$u_{DB} := u_{DB} - \ell_{DB} = 6 - 1 = 5, \quad b_D := b_D + \ell_{DB} = 4 + 1 = 5, \quad b_B := b_B - \ell_{DB} = 6 - 1 = 5$$

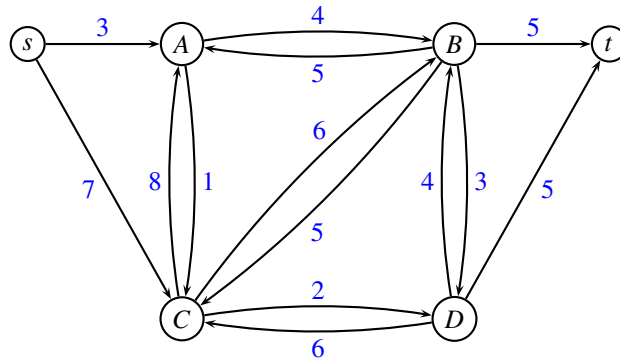
In this way we get the ordinary min-cost flow problem:



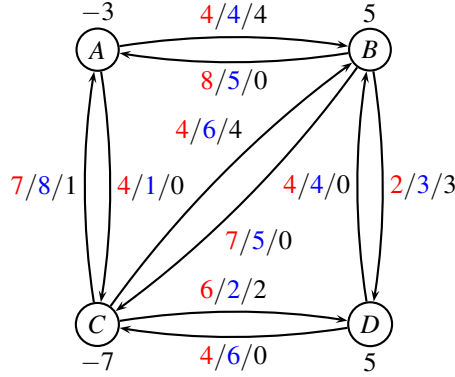
After having solved the problem, we need to add ℓ_{ij} to the flow variables to find the optimal solution.

■

Answer 14 We solve our problem using the *augmenting cycle algorithm*. First, an initial solution is found by adding a source s and terminal t . The source node s is connected to all nodes i having demand $b_i < 0$ with an edge having capacity $-b_i$. All nodes i having demand $b_i > 0$ are connected to the terminal node t with an edge having capacity b_i . Edge costs are ignored.

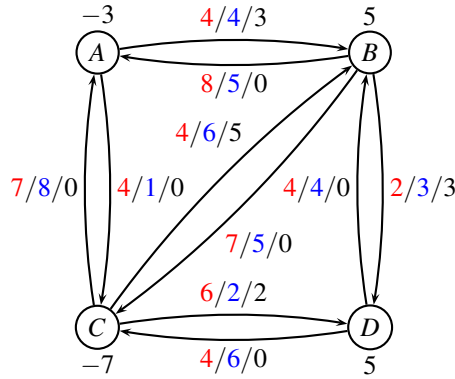


We now solve a maximum-flow problem using the augmented path algorithm. First, we find path $sCABt$ with capacity 4. Next, we find path $sCBDt$ with capacity 3. Then, we find path $sACDt$ with capacity 2. Finally, we find path $sACBt$ with capacity 1. This gives the initial solution to the minimum cost flow problem (black numbers indicate the flow):



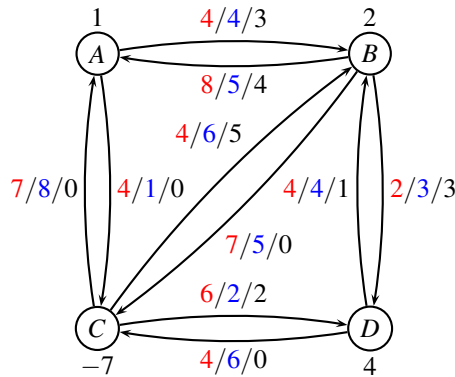
The cost is $z = 4 \cdot 4 + 7 \cdot 1 + 4 \cdot 4 + 2 \cdot 3 + 6 \cdot 2 = 57$.

An augmenting cycle is $CBAC$ with cost $4 - 4 - 7 = -7$ and capacity $\min\{2, 4, 1\} = 1$. We flow 1 unit along the cycle getting:



No more augmenting cycles can be found, so the algorithm terminates. The cost is $z = 4 \cdot 3 + 4 \cdot 5 + 2 \cdot 3 + 6 \cdot 2 = 50$. ■

Answer 15 To find the optimal solution to the original problem (*) we add lower bounds $\ell_{BA} = 4$ and $\ell_{DB} = 1$, getting:



The final cost is $z = 4 \cdot 3 + 4 \cdot 5 + 2 \cdot 2 + 6 \cdot 2 + 8 \cdot 4 + 4 \cdot 1 = 86$. ■

Checking solution in CPLEX

```

minimize
  4xAB+4xAC+8xBA+7xBC+2xBD+7xCA+4xCB+6xCD+4xDB+4xDC
subject to
  A: -xAB-xAC+xBA+xCA = 1

```

```

B: xAB-xBA-xBC-xBD+xCB+xDB = 2
C: xAC+xBC-xCA-xCB-xCD+xDC = -7
D: xBD+xCD-xDB-xDC = 4
UAB: xAB <= 4
UAC: xAC <= 1
UBA: xBA <= 9
LAB: xBA >= 4
UBC: xBC <= 5
UBD: xBD <= 3
UCA: xCA <= 8
UCB: xCB <= 6
UCD: xCD <= 2
UDB: xDB <= 6
LDB: xDB >= 1
UDC: xDC <= 6
end

CPLEX> opt
Dual simplex - Optimal: Objective = 8.6000000000e+01
Solution time = 0.04 sec. Iterations = 0 (0)
Deterministic time = 0.01 ticks (0.37 ticks/sec)

```

```

CPLEX> dis sol var -
Variable Name      Solution Value
xAB                3.000000
xBA                4.000000
xBD                3.000000
xCB                5.000000
xCD                2.000000
xDB                1.000000
All other variables in the range 1-10 are 0.

```

Multicommodity-flow

We consider again the route net ASIANET where we have **new demands** as follows:

k	s	t	d
1	C	B	4
2	A	D	2
3	D	A	6

Thus, commodity $k = 1$ is shipping four containers from C to B , commodity $k = 2$ is shipping two containers from A to D , and commodity $k = 3$ is shipping six containers from D to A .

Answer 16 We have the routes r outlined in the below table. The cost of sending d_k units along a route r is calculated as $d_k \sum_{(i,j) \in r} c_{ij}$.

number	commodity k	route r	d_k	$\sum c_{ij}$	cost
1	1	CAB	4	11	44
2	1	CB	4	4	16
3	2	ABCD	2	17	34
4	2	ABD	2	6	12
5	3	DBA	6	12	72
6	3	DCA	6	11	66
7	3	DCBA	6	16	96

This gives the path formulation:

```

minimize
+44x1+16x2+34x3+12x4+72x5+66x6+96x7
subject to
UAB: +4x1      +2x3+2x4      <= 4
UAC:                                     <= 1
UBA:                                     +6x5      +6x7 <= 9
UBC:      +2x3                                     <= 5
UBD:      +2x4                                     <= 3
UCA: +4x1      +6x6      <= 8
UCB:      +4x2      +6x7 <= 6
UCD:      +2x3      <= 2
UDB:      +6x5      <= 6
UDC:      +6x6+6x7 <= 6

K1: +x1+x2 = 1
K2: +x3+x4 = 1
K3: +x5+x6+x7 = 1
end

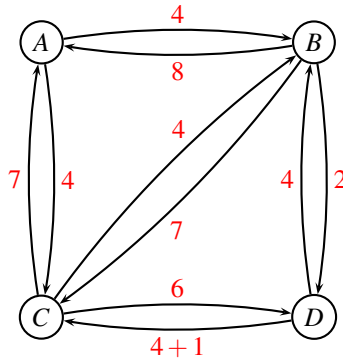
```

■

We solve the problem through column generation. After a few iterations we have the dual variables: $y_{DC} = -1$ (all other $y_{ij} = 0$), while $\bar{y}_1 = 16$, $\bar{y}_2 = 34$, $\bar{y}_3 = 72$.

Here, y_{ij} denote the dual variable corresponding to capacity constraints, and $\bar{y}_k$ denote the dual variable corresponding to commodity constraints.

Answer 17 We update the edge costs for edge (D,C) getting the graph:



We find

$k = 1$: (demand $d_1 = 4$). We remove edges AC, BD, CD (due to capacity). Shortest path CB (route 1) with cost $z = 4$. Reduced cost is $d_1 \cdot z - \bar{y}_1 = 4 \cdot 4 - 16 = 0$.

$k = 2$: (demand $d_2 = 2$) We remove edge AC (due to capacity). Shortest path ABD (route 4) with cost $z = 6$. Reduced cost is $d_2 \cdot z - \bar{y}_2 = 2 \cdot 6 - 34 = -22$.

$k = 3$: (demand $d_3 = 6$) We remove edges AB, AC, BC, BD and CD (due to capacity). Shortest path DCA (route 6) with cost $z = 12$. Reduced cost is $d_3 \cdot z - \bar{y}_3 = 6 \cdot 12 - 72 = 0$. (**Alternative solution**: Shortest path DBA (route 5) with cost $z = 12$.)

The most negative reduced cost is obtained with path ABD (route 4) with reduced cost -22.

Checking solution in CPLEX

\ Example 4068b

```

minimize
+44x1+16x2+34x3+12x4+72x5+66x6+96x7
subject to
UAB: +4x1      +2x3+2x4          <= 4
UAC:                                     <= 1
UBA:                                     +6x5      +6x7 <= 9
UBC:          +2x3                <= 5
UBD:          +2x4                <= 3
UCA: +4x1          +6x6          <= 8
UCB:      +4x2          +6x7 <= 6
UCD:          +2x3                <= 2
UDB:          +6x5                <= 6
UDC:          +6x6+6x7 <= 6

```

```

K1: +x1+x2 = 1
K2: +x3+x4 = 1
K3: +x5+x6+x7 = 1

```

```

R1: x1 <= 0
\\ R2: x2 <= 0
\\ R3: x3 <= 0
R4: x4 <= 0
\\ R5: x5 <= 0
\\ R6: x6 <= 0
R7: x7 <= 0
end

```

```

CPLEX> read iter0
Problem 'iter0.lp' read.
Read time = 0.00 sec. (0.00 ticks)
CPLEX> opt
Tried aggregator 1 time.

```

LP Presolve eliminated 16 rows and 7 columns.

All rows and columns eliminated.

Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal: Objective = 1.1600000000e+02

Solution time = 0.00 sec. Iterations = 0 (0)

Deterministic time = 0.01 ticks (9.26 ticks/sec)

CPLEX> dis sol var -

Variable Name	Solution Value
x2	1.000000
x3	1.000000
x6	1.000000

All other variables in the range 1-7 are 0.

CPLEX> dis sol dua -

Constraint Name	Dual Price
UDC	-1.000000
K1	16.000000
K2	34.000000
K3	72.000000
R4	-22.000000

All other dual prices in the range 1-16 are 0.

■

Research planning

A research company has to handle 3 projects, P_1, P_2, P_3 , during the next 5 months.

project	start month	end month	man-months needed
P_1	1	4	14
P_2	1	5	16
P_3	3	5	8

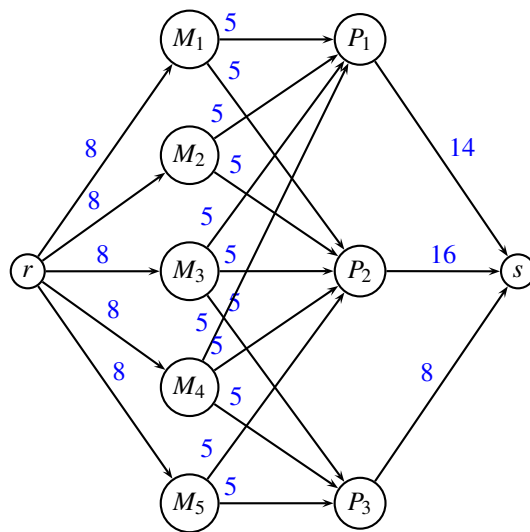
Project P_1 runs from month 1 to 4, P_2 runs from month 1 to 5, and P_3 runs from month 3 to 5. The projects require, respectively, 14, 16, and 8 man-months. Each month, 8 researchers are available, however not more than 5 researchers can work on the same project each month.

Answer 18 We create a node for each month $M_1 \dots, M_5$ and a node for each project $P_1, \dots, P_3$. If project P_j is running in month M_i we connect the two nodes with an edge of capacity 5.

A root node r and sink node s is added. The root node r is connected to each month M_i with edges having capacity 8. All project nodes P_j are connected to the sink with an edge of capacity equal to the man-months needed on the project.

Now the problem is a maximum-flow problem. If we can find a solution that satisfies all demand for man-months (i.e. a flow of value $14 + 16 + 8$) then a feasible solution exists.

The flow on edges (M_i, P_j) denotes how many researchers should be assigned to project P_j in month M_i .



The above figure illustrates the formulation as a maximum flow problem. ■

Solution 16 December, 2022

Course title: Network Optimization

Course no: 42115

Aids allowed: All

Exam duration: 4 hours

Weighting: The points are indicated at each question

Language: You can write in Danish or English

Figures: Figures can be found on the last pages of this assignment. You can use them in your answer.

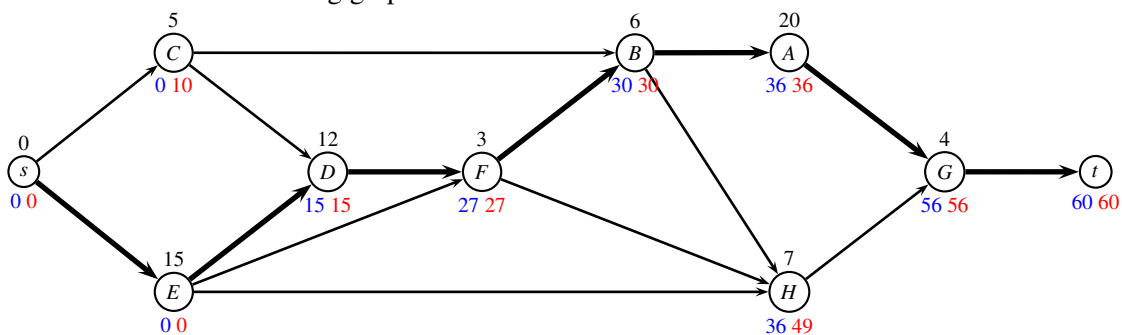
The Pisinger cake

The Pisinger cake was created by Oscar Pischinger, a famous baker from Vienna, in the 19th century.

The duration of each task is given in the following table together with predecessors that need to be finished before the task can start

task	time (min)	predecessors
A	20	B
B	6	C, F
C	5	
D	12	C, E
E	15	
F	3	E, D
G	4	A, H
H	7	E, F, B

Answer 1 A topological ordering of the activities must ensure that all edges are pointing forward. This is the case in the following graph:



Answer 2 Let U_i be the earliest starting time of task i (marked with blue), and let V_i be the latest starting time (marked with red).

- *Forward procedure:* Starting with $U_s = 0$, we use recursion $U_j = \max_{(i,j) \in E} (U_i + d_i)$.

- *Makespan*: $T = U_t = 60$.
- *Backward procedure*: Starting with $V_t = T$, we use recursion $V_i = \min_{(i,j) \in E} (V_j - d_i)$.

■

Answer 3 The critical path is found where $U_i = V_j$ (fat edges on above figure). ■

The following recipe is taken from Rosa Cooking
<https://www.rosacooking.com/en/pisinger-cake/>

Ingredients

- *Wafers*:
1 pack of round wafers (you can use 8–10 home-made ice-wafers — they should be thin and flat)
- *Cream*:
4 whole eggs
2 egg yolks
300 g sugar
200 g cooking chocolate
300 g butter
200 g almonds
juice of 1 lemon
- *Chocolate glazing*:
100 g cooking chocolate
3 tablespoons of gloss oil
a little milk

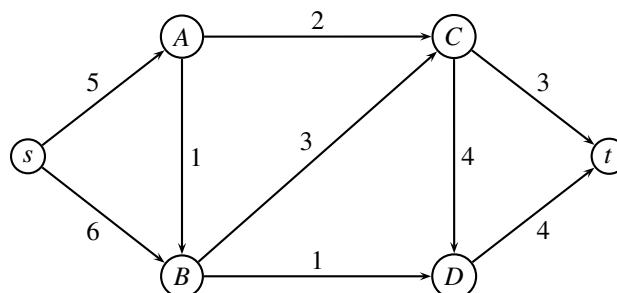
Preparation

- Using a round bowl, cut the wafers to get 10 round sheets.
- Cream: Beat eggs, chocolate, egg yolks, sugar in steam, when the mixture starts to thicken, cool it a bit, then mix in the frothy butter, lemon juice and ground almonds.
- Peel and fry almonds. Chop almonds and mix them into the cream.
- Coat the wafer-sheets with the obtained cream.
- Next leave them to cool under load.
- Pour the chocolate icing over the cooled cake and keep it cool.

Bon appetite!

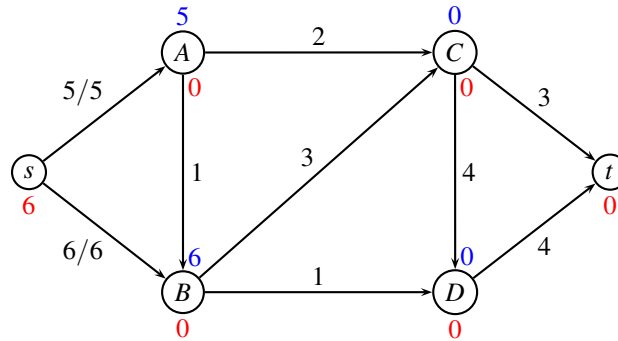
Maximum flow problem

In the following network the edge weights denote the capacity. The problem is to find the maximum flow from s to t .

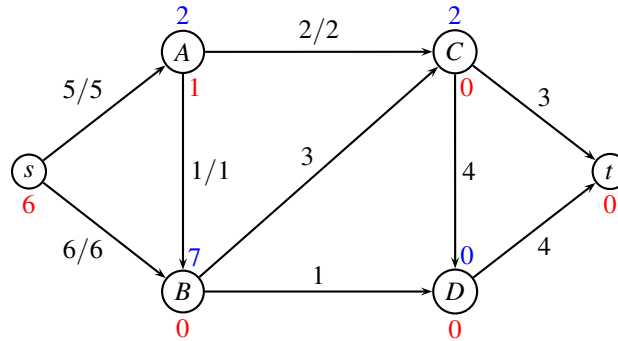


Answer 4 We use *preflow-push*. (label d_i is **red**, surplus $f_x(i)$ is **blue**)

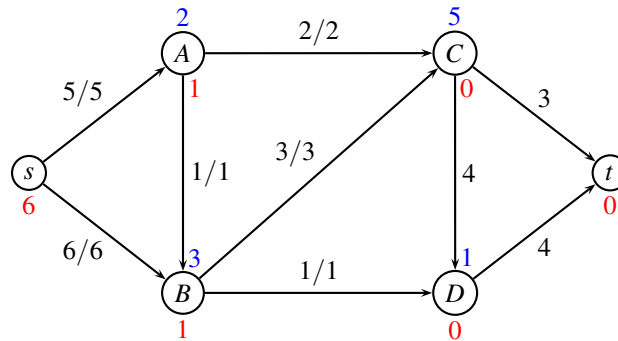
Initialize a valid preflow by setting labels $d_s = n = 6$ and remaining $d_i = 0$. Flow on edges leaving s is set to capacity of edges.



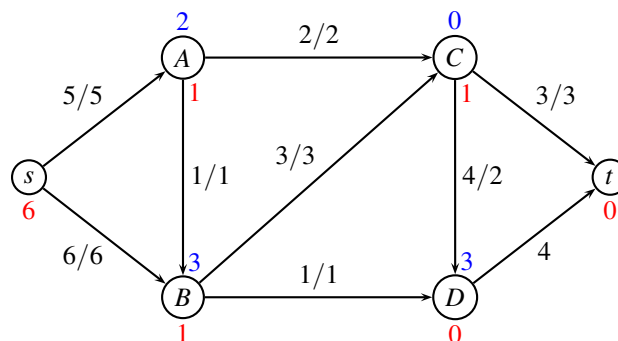
Relabel A to 1, push



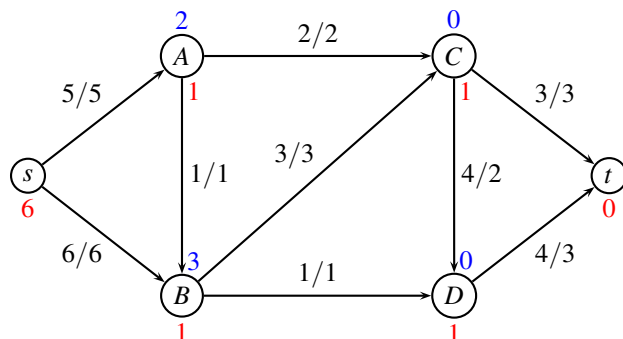
Relabel B to 1, push



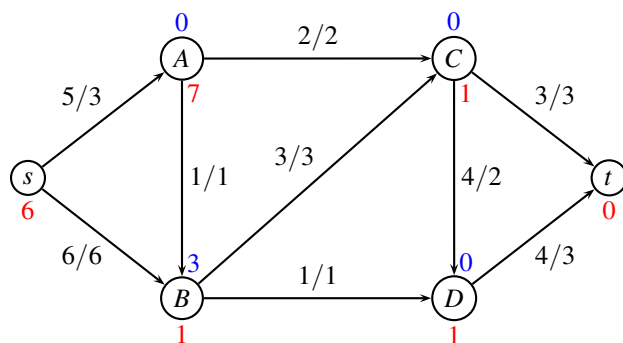
Relabel C to 1, push



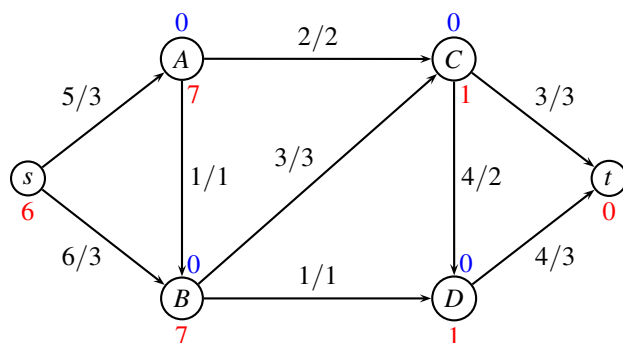
Relabel D to 1, push



Relabel A to 7 (since only outgoing edge in residual graph is As), push



Relabel B to 7 (outgoing edges in residual graph are BA and Bs), push



Since no nodes have a surplus, we have a valid flow, and hence the algorithm terminates. The maximum flow is $z = 6$.

Note: Many alternative iterations exist depending on in which order you choose the nodes. They all find the same optimal solution. **Fewest iterations:** Relabel A to $d_A = 1$, push 2 to C , push 1 to B . Relabel A to $d_A = 7$, push 2 to s . Relabel B to $d_B = 1$, push 3 to C , push 1 to B . Relabel B to $d_B = 7$, push 3 to s . Relabel C to $d_C = 1$, push 3 to t , push 2 to D . Relabel D to $d_D = 1$, push 3 to t . ■

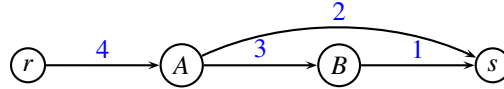
Answer 5 We use the *preflow-push* method: A minimum cut is found by choosing an unused label, e.g. $k = 2$. We then have $R = \{s, A, B\}$ and $\bar{R} = \{C, D, t\}$. The capacity of the cut is $2 + 3 + 1 = 6$. ■

Max flow min cut

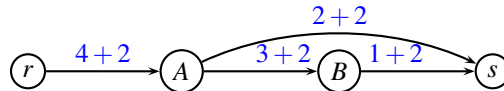
We are considering a maximum flow problem defined on a directed graph $G = (V, E)$ and edge capacities $u_{ij} \geq 0$. The solution vector is denoted (x_{ij}) , and a cut is denoted $(R, \bar{R})$.

Answer 6 We have the two statements

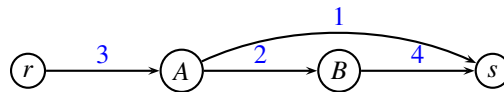
A) **NO**. The following example shows a case where the minimum cut is $R = \{rAB\}$.



Adding a constant 2 to all edges gives a minimum cut $R = \{r\}$.



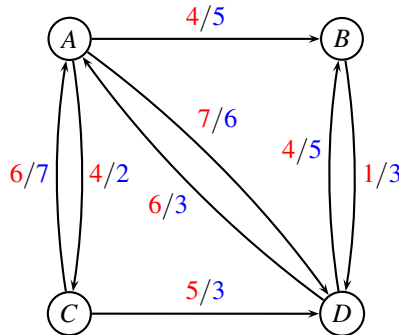
B) **NO**. The following example shows a case where there are two different cuts of capacity 3: Either $R = \{r\}$ or $R = \{rA\}$. The maximum flow is also 3, so both cuts are minimum cuts.



■

Repositioning of empty containers

Consider the following network, where costs are **red**, and capacities are **blue**.

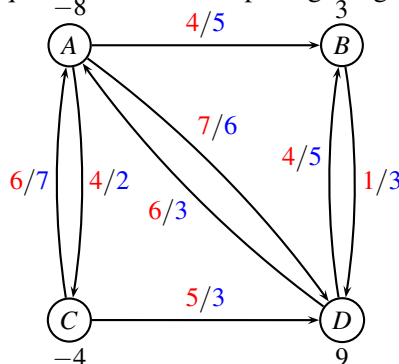


Due to the imbalance in trade, empty containers tend to accumulate at various ports. For the above network, one could have the following surplus of containers:

node i	A	B	C	D
surplus σ_i	8	-3	4	-9

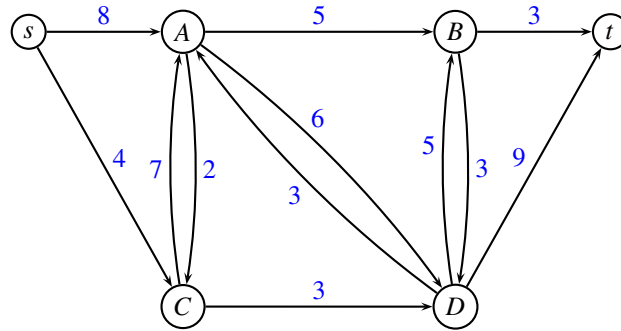
We would like to reposition the containers in the cheapest possible way. It is assumed that all empty containers are equal.

Answer 7 We set the demands equal to minus the surplus, getting:



■

Answer 8 We add a source s and a sink t . All nodes with negative demand are connected to s with an edge having capacity equal to minus the demand. All nodes with positive demand are connected to t with an edge having capacity equal to the demand.



We should use augmenting paths of largest capacity, so we get:

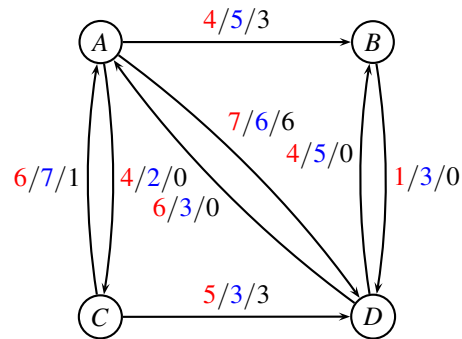
Path $sADt$ flow 6.

Path $sCDt$ flow 3.

Path $sABt$ flow 2.

Path $sCABt$ flow 1.

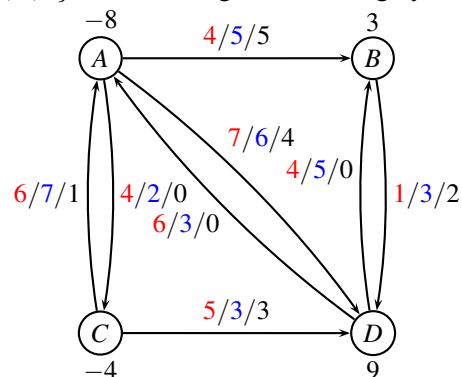
We now have the initial solution



Objective value $z = 4 \cdot 3 + 6 \cdot 1 + 7 \cdot 6 + 5 \cdot 3 = 75$.

Note Alternative paths exist which may lead to a different start solution. ■

Answer 9 We use *augmenting cycle* method. An augmenting cycle is $ABDA$, cost $4 + 1 - 7 = -2$. Capacity of cycle is $\min\{5 - 3, 3, 6\} = 2$. Sending 2 units along cycle:



Since no more negative cycles exist, terminate. Objective value $z = 4 \cdot 5 + 6 \cdot 1 + 7 \cdot 4 + 1 \cdot 2 + 5 \cdot 3 = 71$.

Using *Network simplex* we get: $T = \{(A,B), (C,A), (A,D)\}$ and $L = \{(A,C), (D,A), (B,D), (D,B)\}$ and $U = \{(C,D)\}$.

Using A as root we find dual variables (distances using edges in T) $y_A = 0, y_B = 4, y_C = -6, y_D = 7$. Reduced costs are found as $\bar{c}_{ij} = c_{ij} + y_i - y_j$

$$\begin{aligned}\bar{c}_{CD} &= 5 + (-6) - 7 = -8 & (U) \\ \bar{c}_{AC} &= 4 + 0 - (-6) = 10 & (L) \\ \bar{c}_{DA} &= 6 + 7 - 0 = 13 & (L) \\ \bar{c}_{BD} &= 1 + 4 - 7 = -2 & (L) \\ \bar{c}_{DB} &= 4 + 7 - 4 = 7 & (L)\end{aligned}$$

Edge (B,D) is in L , and has most negative reduced cost. Corresponding cycle is $\{A, B, D, A\}$. We can send $\min\{2, 5, 6\} = 2$ units, getting the same flow as on top of page.

Now we have $T = \{(B,D), (C,A), (A,D)\}$ and $U = \{(A,B), (C,D)\}$ and $L = \{(A,C), (D,A), (D,B)\}$. Using A as root we find dual variables $y_A = 0, y_B = 6, y_C = -6, y_D = 7$. Reduced costs are

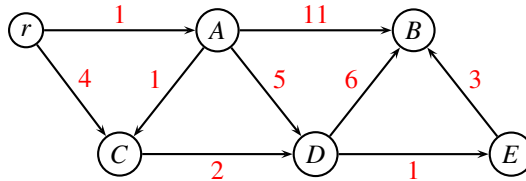
$$\begin{aligned}\bar{c}_{AB} &= 4 + 0 - 6 = -2 & (U) \\ \bar{c}_{CD} &= 5 + (-6) - 7 = -8 & (U) \\ \bar{c}_{AC} &= 4 + 0 - (-6) = 10 & (L) \\ \bar{c}_{DA} &= 6 + 7 - 0 = 13 & (L) \\ \bar{c}_{DB} &= 4 + 7 - 6 = 5 & (L)\end{aligned}$$

Since no edges in L have negative reduced cost, and no edges in U have positive reduced cost, we terminate. ■

Shortest Path Problem in a dynamic network

In network communication there are several protocols to ensure that packages are sent to the correct destination. One of these, the *Open Shortest Path First* (OSPF), makes use of Dijkstra's algorithm to route packages in a dynamic network, where connections are added/removed.

Consider the following directed graph where distances w_{ij} are marked with red.

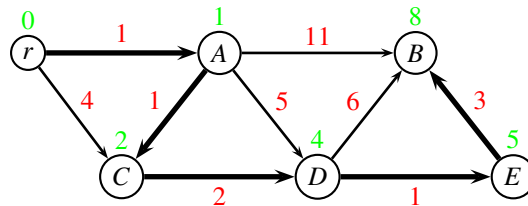


Assume that you should find the shortest path from r to all other nodes.

Answer 10 The following table indicates the nodes and their distance

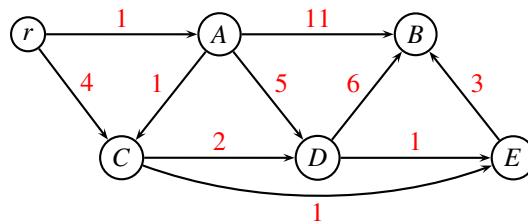
considered node	r	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
r	0	1	∞	4	∞	∞
A	0	1	12	2	6	∞
C	0	1	12	2	4	∞
D	0	1	10	2	4	5
E	0	1	8	2	4	5
B	0	1	8	2	4	5

Shortest path tree is:



■

Now, assume that an edge (C, E) with distance 1 is added to the graph:

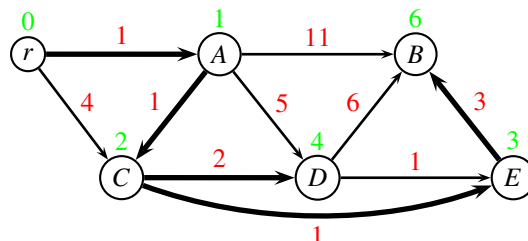


Answer 11 All distances are still valid, so we can initialize Dijkstra with the distances found in previous question. Furthermore, nodes that were considered before node C in previous question, do not need to be considered. ■

Answer 12 We initialize with the distances found in previous question, and skip nodes r and A since they were considered before node C

	r	A	B	C	D	E
init	0	1	8	2	4	5
C	0	1	8	2	4	3
E	0	1	6	2	4	3
D	0	1	6	2	4	3
B	0	1	6	2	4	3

Shortest path tree is:



■

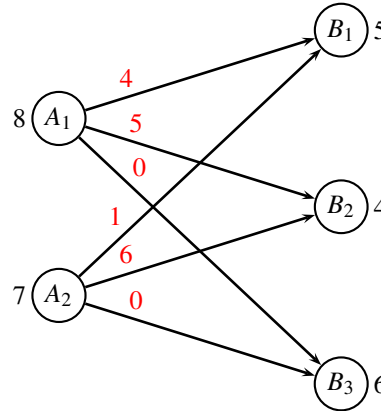
Transportation problem

Two producers A_1 and A_2 are delivering products to two customers B_1 and B_2 . The producers can produce at most 8 and 7 items respectively. Customer B_1 has a demand of 5 items, while customer B_2 has a demand of 4 items. The transportation costs are given by the following table

	B_1	B_2	max production
A_1	4	5	8
A_2	1	6	7
demand	5	4	

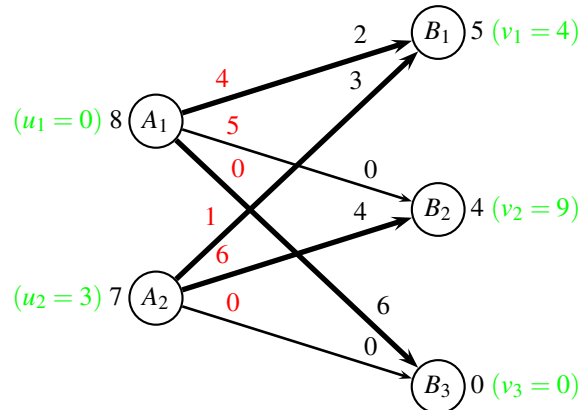
Note, that the problem is not balanced.

Answer 13 We add a dummy node B_3 to take the excess production. Producers are connected to the dummy node with edges having cost 0.



■

Answer 14 We first consider A_1 , and send 6 units to B_3 , and then 2 units to B_1 . Next, we consider A_2 , and send 3 units to B_1 , and then 4 units to B_2 . This gives the following greedy solution:



The cost of the solution is $z = 4 \cdot 2 + 0 \cdot 6 + 1 \cdot 3 + 6 \cdot 4 = 35$.

Fat edges on the graph indicate the basis. Dual values (indicated with green) are found by finding the shortest path from node A_1 , using only the fat edges. ■

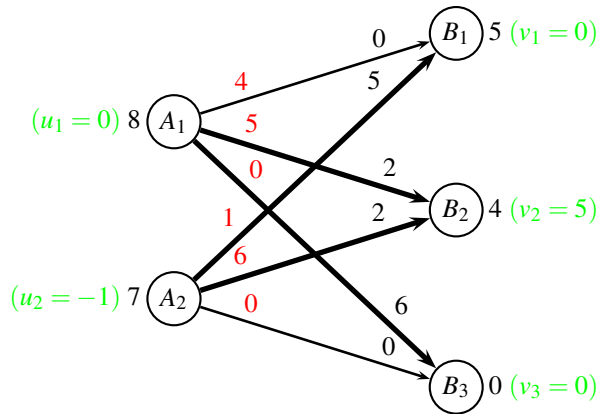
Answer 15 We now find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, getting

$$\bar{c}_{12} = 5 + 0 - 9 = -4$$

$$\bar{c}_{23} = 0 + 3 - 0 = 3$$

We choose edge (A_1, B_2) corresponding to the cycle (A_1, B_2, A_2, B_1) . We can send at most $\delta = \min\{4, 2\} = 2$ units along this cycle.

After the pivot operation we get the solution



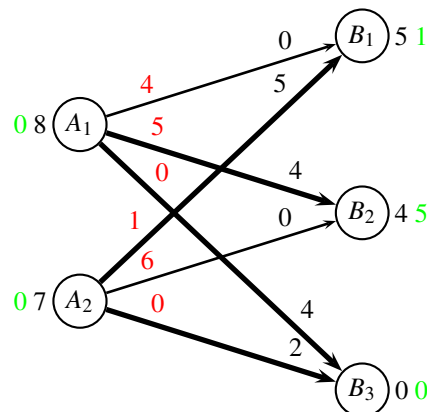
The cost of the solution is $z = 5 \cdot 2 + 0 \cdot 6 + 1 \cdot 5 + 6 \cdot 2 = 27$. Dual variables are calculated as before. The reduced costs are

$$\bar{c}_{11} = 4 + 0 - 0 = 4$$

$$\bar{c}_{23} = 0 + (-1) - 0 = -1$$

We choose edge (A_2, B_3) corresponding to the cycle (A_2, B_3, A_1, B_2) . We can send at most $\delta = \min\{6, 2\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $z = 5 \cdot 4 + 0 \cdot 4 + 1 \cdot 5 + 0 \cdot 2 = 25$. Dual variables are calculated as before. The reduced costs are

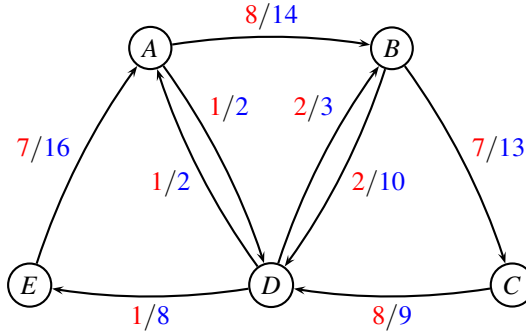
$$\bar{c}_{11} = 4 + 0 - 1 = 3$$

$$\bar{c}_{22} = 6 + 0 - 5 = 1$$

No negative reduced costs, so we terminate. The optimal solution value is $z = 25$. ■

Flowing containers

MEDITLINE (Route net of a Mediterranean liner shipping company)



For edge (i, j) the first number is the **cost** c_{ij} and the second number is the **capacity** u_{ij} (in TEU).

We consider again the route net MEDITLINE from project assignment 2022. The network is *identical* to the one used in the project assignment, but we have *new demands* as follows:

k	s_k	t_k	d_k	w_k
1	D	C	3	2
2	A	D	2	4

Commodity $k = 1$ is shipping three 40 foot containers (size $w_k = 2$ TEU) from D to C , and commodity $k = 2$ is shipping two 40 foot high-cube containers (size $w_k = 4$ TEU) from A to D .

We are considering a *weighted multi-commodity flow problem*: For each commodity, a container of size w_k TEU has to follow the same path, but the d_k different containers can follow individual paths.

Answer 16 We have the routes r outlined in the below table. The cost of sending w_k units along a route r is calculated as $w_k \sum_{(i,j) \in r} c_{ij}$.

number r	commodity k	route r	w_k	$\sum c_{ij}$	cost
1	1	$DABC$	2	16	32
2	1	DBC	2	9	18
3	1	$DEABC$	2	23	46
4	2	$ABCD$	4	23	92
5	2	ABD	4	10	40

■

We solve the LP-relaxed problem through *column generation*. After a few iterations we have the

restricted master problem on the form

$$\begin{array}{llllll}
\min & c_1x_1 & + & c_2x_2 & + & c_3x_3 & \\
\text{s.t.} & & & 2x_2 & + & 4x_3 & \leq 14 & (y_{AB}) \\
& & & & & & \leq 2 & (y_{AD}) \\
& 2x_1 & + & 2x_2 & & & \leq 13 & (y_{BC}) \\
& & & & & 4x_3 & \leq 10 & (y_{BD}) \\
& & & & & & \leq 9 & (y_{CD}) \\
& & & & & & \leq 2 & (y_{DA}) \\
& 2x_1 & & & & & \leq 3 & (y_{DB}) \\
& & & 2x_2 & & & \leq 8 & (y_{DE}) \\
& & & & & 2x_2 & \leq 16 & (y_{EA}) \\
& x_1 & + & x_2 & & & \geq 3 & (\bar{y}_1) \\
& & & & & x_3 & \geq 2 & (\bar{y}_2) \\
& & & & & & & x_1, x_2, x_3 \geq 0
\end{array} \tag{1}$$

Let y_{ij} denote the dual variable corresponding to the capacity constraint for edge (i, j) , and let $\bar{y}_k$ denote the dual variable corresponding to the demand constraint for commodity k .

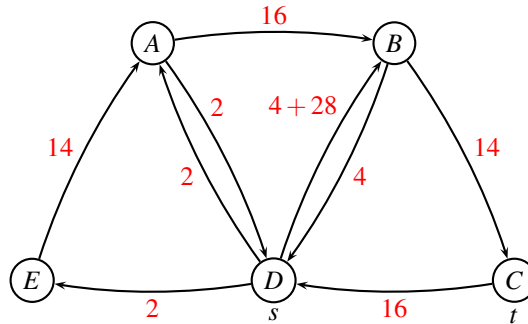
The dual variables for the above model are: $y_{DB} = -14$, while all other are $y_{ij} = 0$. Furthermore, $\bar{y}_1 = 46$, $\bar{y}_2 = 40$.

Answer 17 The reduced cost of a route r is calculated as

$$\bar{c}_r^k = \sum_{(i,j) \in r} w_k c_{ij} - \sum_{(i,j) \in r} w_k y_{ij} - \bar{y}_k = \sum_{(i,j) \in r} w_k (c_{ij} - y_{ij}) - \bar{y}_k$$

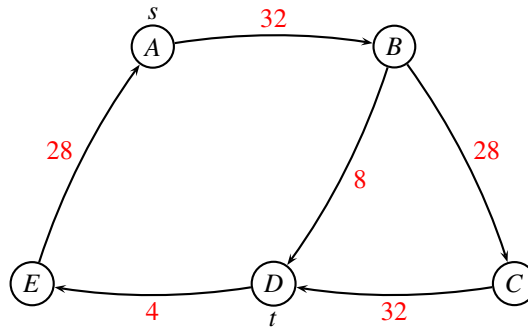
In order to solve the pricing-problem as a shortest path problem we set the cost of edge (i, j) to $\tilde{c}_{ij} = w_k(c_{ij} - y_{ij})$. For commodity k we first remove all edges with capacity $u_{ij} < w_k$. as they cannot accommodate the requested number of TEU. Next, we solve the shortest path problem using edge costs $\tilde{c}_{ij}$, with root s_k and destination t_k . let z_k be the length of the shortest path for commodity k . After solving the shortest path problem for each k , we find the reduced cost as $\bar{c}^k = z_k - \bar{y}_k$.

$k = 1$: We do not remove any edges. Costs are updated as $\tilde{c}_{ij} = w_k(c_{ij} - y_{ij})$



Shortest path is $DABC$ with cost $z_1 = 32$. The reduced cost becomes $\bar{c}^1 = z_1 - \bar{y}_1 = 32 - 46 = -14$.

$k = 2$: We remove edges $(A, D), (D, A), (D, B)$. Costs are updated as $\tilde{c}_{ij} = w_k(c_{ij} - y_{ij})$



Shortest path is ABD with cost $z_2 = 40$. The reduced cost becomes $\bar{c}^2 = z_2 - \bar{y}_2 = 40 - 40 = 0$.

Note: Instead of multiplying all edge weights by w_k one can also use the original edge weights, and calculate the reduced cost as $\bar{c}^k = w_k z_k - \bar{y}_k$. ■

Answer 18 The most negative reduced cost is obtained for $k = 1$ with path $DABC$ (route 1) having reduced cost -14. ■

Chocolate factories

Two chocolate factories A and B are producing at most 500 chocolate-bars per month, each. Customer C demands 350 chocolate-bars, while customer D demands 400 chocolate-bars per month. Production costs are \$5 at factory A , while production costs are \$6 at factory B .

The factories can either deliver directly to the customers, or they can deliver through a warehouse W . Shipping costs per chocolate-bar are as follows:

From/to	W	C	D
A	1	5	6
B	2	4	5
W	-	3	2

The capacity of the shipping company is given by the following table:

From/to	W	C	D
A	400	500	350
B	200	600	350
W	-	200	200

The warehouse has a limited capacity, so at most 300 chocolate-bars can be shipped through the warehouse. However, at least 80 chocolate-bars need to be shipped through the warehouse.

Answer 19 We construct a network as follows: Nodes correspond to; producers A, B , customers C, D and warehouse W . The demand for producers is -500 , and the demand for customers is 350 and 400.

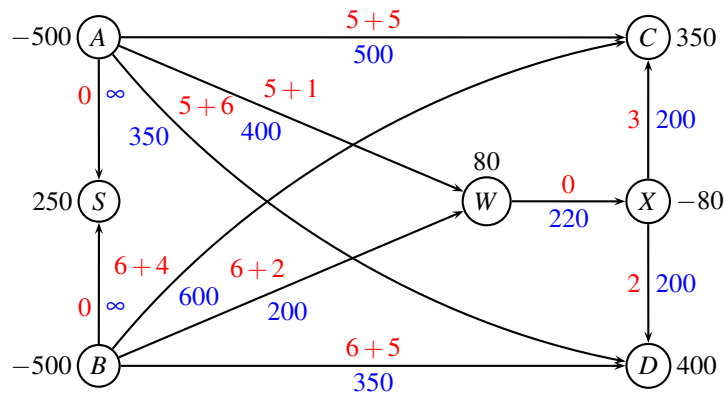
We connect each producer to the three possible nodes C, D and W . Edge costs are set to transportation cost plus production cost. Capacity corresponds to the capacity of the shipping company.

As the production of $500+500$ exceeds the demand of $350+400$, we add a sink node S to absorb the excess production, with demand $b_S = 500 + 500 - (350 + 400) = 250$. The producers are connected to S with edges having cost 0 and infinite capacity.

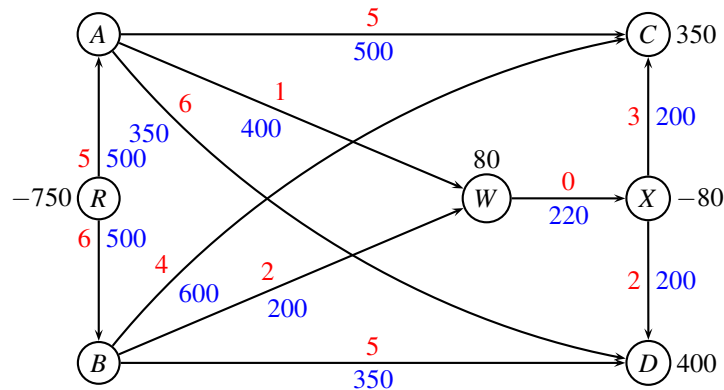
Since the capacity of the warehouse is 300, we split it into two nodes W and X , connected by an edge with cost 0 and capacity 300. There is also a lower bound ℓ on the edge, so we use the following transformation for edge (W, X)

$$u_{WX} := u_{WX} - \ell_{WX} = 300 - 80 = 220 \quad b_W := b_W + \ell_{WX} = 80 \quad b_X := b_X - \ell_{WX} = -80$$

Note, that since the edge (W, X) has cost 0, the objective function is not affected by the transformation, and we do not need to do any calculations after the problem has been solved.



Alternative solution Add a root node R with capacity equal to all demand, i.e. $b_R = -(350 + 400) = -750$. Production costs/capacities are assigned to edges (R, A) and (R, B) .



Alternative solution Some students have understood the formulation, such that A and B together can produce 500 units. In this case the root node R above will have demand $b_R = -500$, and we need an extra node S with demand $b_S = -250$ to supply nodes C and D with the remaining demand, at no cost.

■

end of assignment

Solution 15 December, 2023

Course title: Network Optimization

Course no: 42115

Aids allowed: All

Exam duration: 4 hours

Critical path with earliest start times

Thristur (pristur) is the favorite mini-candy in Iceland. Production consists of the following six steps:

activity	description	duration	predecessor	earliest starting time
<i>A</i>	Cut toffee into squares	2	<i>F</i>	8
<i>B</i>	Cut licorice into pieces	1		
<i>C</i>	Cover with chocolate	3	<i>A, E</i>	
<i>D</i>	Mix caramel cream	3		
<i>E</i>	Melt chocolate	4	<i>B, F</i>	7
<i>F</i>	Mix caramel cream and licorice	4	<i>D, B</i>	

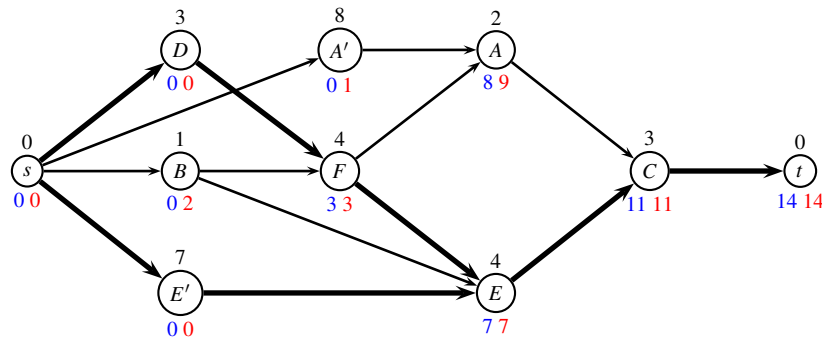
Since some of the ingredients are made in remote factories, we have earliest starting times on some of the activities.

Answer 1 We add a node A' with duration 8 and no predecessors. We include A' as predecessor to A . We add a node E' with duration 7 and no predecessors. We include E' as predecessor to E . In this way we get:

activity	description	duration	predecessor
A'	Earliest time of A	8	
A	Cut toffee into squares	2	F, A'
B	Cut licorice into pieces	1	
C	Cover with chocolate	3	A, E
D	Mix caramel cream	3	
E'	Earliest time of E	7	
E	Melt chocolate	4	B, F, E'
F	Mix caramel cream and licorice	4	D, B

Tasks without a predecessor have start-node s as predecessor. ■

Answer 2 A topological ordering of the activities must ensure that all edges are pointing forward. This is the case in the following graph:



■

Answer 3 Let U_i be the earliest starting time of task i (marked with blue), and let V_i be the latest starting time (marked with red).

- *Forward procedure:* Starting with $U_s = 0$, we use recursion $U_j = \max_{(i,j) \in E} (U_i + d_i)$.
- *Makespan:* $T = U_t = 14$.
- *Backward procedure:* Starting with $V_t = T$, we use recursion $V_i = \min_{(i,j) \in E} (V_j - d_i)$.

Note: If you solved the original problem then earliest/latest times: D 0 0, B 0 2, F 3 3, A 7 9, E 7 7, C 11 11. ■

Answer 4 The critical path is found where $U_i = V_j$ (fat edges on above figure). Notice that we have two critical paths.

Note: If you solved the original problem then the critical path is $sDFECt$. ■

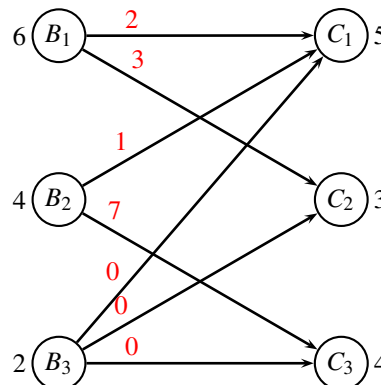
Transportation problem

Two bakeries B_1 and B_2 are delivering cakes to three courses C_1 (42115), C_2 (42114) and C_3 (42142). The bakeries are producing 6 and 4 cakes respectively. Course C_1 can eat at most 5 cakes, course C_2 can eat at most 3 cakes, and course C_3 can eat at most 4 cakes. All cakes produced need to be eaten. The transportation costs are given by the following table

	C_1	C_2	C_3	production
B_1	2	3	-	6
B_2	1	-	7	4
max demand	5	3	4	

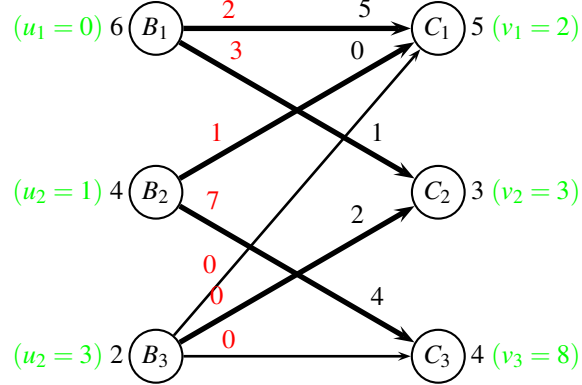
A “-” indicates that transportation is not possible. The objective is to minimize overall transportation costs. **Note,** that the problem is not balanced.

Answer 5 We add a dummy node B_3 to model unmet demand. The dummy node has production $(5 + 3 + 4) - (6 + 4) = 2$. The dummy node is connected to all courses C_j with edges having cost 0.



■

Answer 6 We first consider B_1 , and send 5 units to C_1 , and then 1 units to C_2 . Next, we consider B_2 , and send 4 units to C_3 . Finally, we consider B_3 , and send 2 units to C_2 . This gives the following greedy solution:



The cost of the solution is $z = 2 \cdot 5 + 3 \cdot 1 + 7 \cdot 4 + 0 \cdot 2 = 41$.

Fat edges on the graph indicate the basis. Since we have only 4 basis variables, we add an additional fat edge (B_2, C_1) .

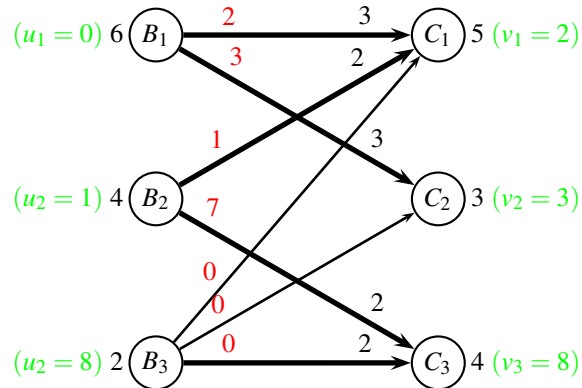
Dual values (indicated with green) are calculated by finding the shortest path from node B_1 , using only the fat edges. ■

Answer 7 We now find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - v_j$, getting

$$\begin{aligned}\bar{c}_{31} &= 0 + 3 - 2 = 1 \\ \bar{c}_{33} &= 0 + 3 - 8 = -5\end{aligned}$$

We choose edge (B_3, C_3) corresponding to the cycle $(B_3, C_3, B_2, C_1, B_1, C_2)$. We can send at most $\delta = \min\{4, 5, 2\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $z = 2 \cdot 3 + 3 \cdot 3 + 1 \cdot 2 + 7 \cdot 2 + 0 \cdot 2 = 31$. Dual variables are calculated as before. The reduced costs are

$$\begin{aligned}\bar{c}_{31} &= 0 + 8 - 2 = 6 \\ \bar{c}_{32} &= 0 + 8 - 3 = 5\end{aligned}$$

No negative reduced costs, so we terminate. The optimal solution value is $z = 31$. ■

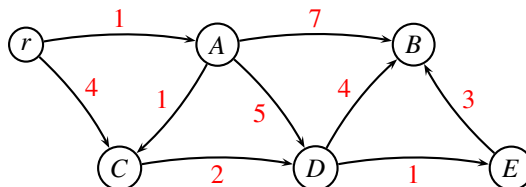
Checking:

```
minimize 2 x11 + 3 x12 + 1 x21 + 7 x23 + 0 x31 + 0 x32 + 0 x33
subject to
  x11 + x12 = 6
  x21 + x23 = 4
  x31 + x32 + x33 = 2
  x11 + x21 + x31 = 5
  x12 + x32 = 3
  x23 + x33 = 4
end
```

```
CPLEX> opt
Dual simplex - Optimal:  Objective = 3.1000000000e+01
CPLEX> dis sol var -
Variable Name      Solution Value
x11                 3.000000
x12                 3.000000
x21                 2.000000
x23                 2.000000
x33                 2.000000
All other variables in the range 1-7 are 0.
```

Shortest Path Problem

Consider the following directed graph where distances $w_{ij} \geq 1$ are indicated with red.



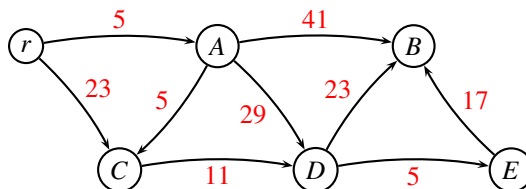
We want to find the shortest path from r to all nodes. If several paths have the same length, we are interested in the shortest path containing *most* edges.

Answer 8 Let $N = 6$ be the number of nodes. This is an upper bound on the longest path. Now, set the edge weights to

$$w'_{ij} = Nw_{ij} - 1$$

This will mean that we still find the shortest path with respect to the original weights w_{ij} but get a reward of 1 for each edge included. Furthermore, notice that since all $w_{ij} \geq 1$ then the new weights w'_{ij} will be non-negative.

This results in the following graph:



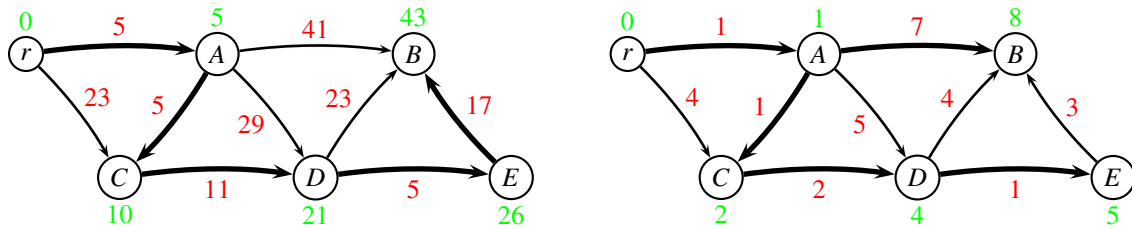
Note: Alternative formulation is: $w'_{ij} = w_{ij} - \epsilon$ where ϵ is a small number, e.g. 0.1. ■

Answer 9 In the following table we report the development of the shortest distances y_i for each node i . The first column indicates which node is considered in Dijkstra's algorithm. (**Note:** right table for original problem)

	r	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
r		5		23		
A			46	10	34	
C					21	
D			44			26
E			43			
B						

	r	A	B	C	D	E
init	0	∞	∞	∞	∞	∞
r		1		4		
A			8	2	6	
C					4	
D						5
E						
B						

The shortest path tree is (**Note:** right figure for original problem)



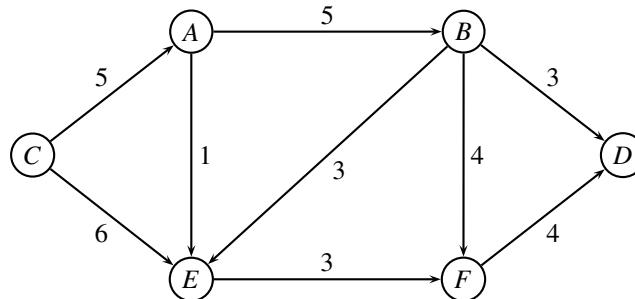
We can next use the shortest path tree to find the original costs and edges:

	r	A	B	C	D	E
shortest path	0	1	8	2	4	5
edges	0	1	5	2	3	4

■

Circulation problem / Maximum flow with demands

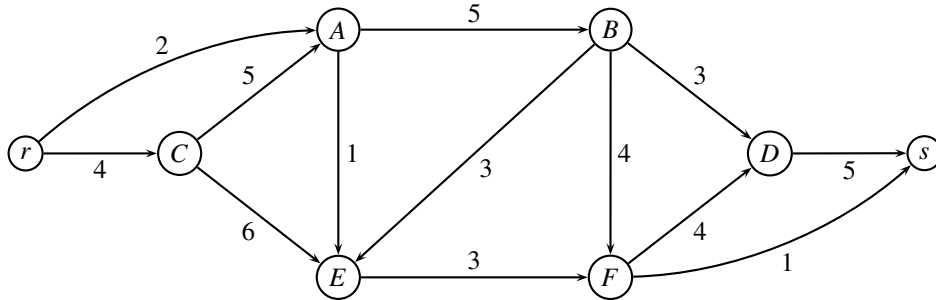
Given a directed graph $G = (V, E)$ and associated edge capacities u_{ij} for each edge (i, j) . In the *Circulation Problem* we have multiple sources and multiple sinks. If a node has positive demand $d_i > 0$ it is called a sink. If a node has negative demand $d_i < 0$ it is called a source. Obviously, we need to have $\sum_{i \in V} d_i = 0$ to match supply and demand.



Assume that we have demands

node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

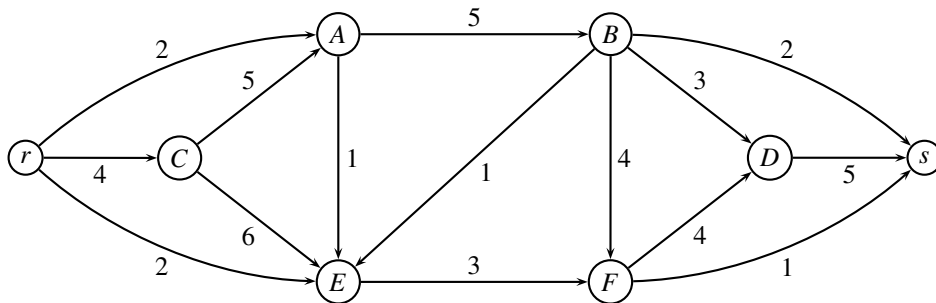
Answer 10 We introduce a root node r and connect it to all nodes having a negative demand. The capacity of the edge is equal to $-d_i$. We introduce a sink node s and connect it to all nodes having positive demand. The capacity of the edge is equal to d_i .



■

Assume that we have a lower bound on edge (B, E) of value $\ell_{BE} = 2$. This means that in a given solution, there should be at least 2 units flowing on the edge.

Answer 11 We use a similar transformation as used in min-cost flow problems. Reduce the capacity of edge (B, E) from 3 to 1. Increase demand at node B from 0 to 2. Decrease demand at node E from 0 to -2. This leads to the model:

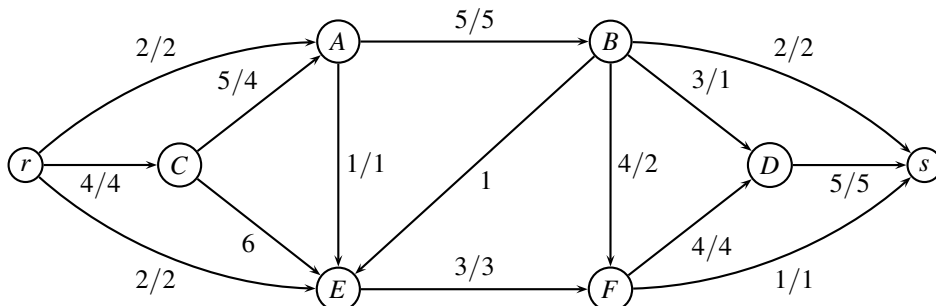


■

Answer 12 We use the augmenting path method, with the following paths:

- 4 items on $rCABFDs$
- 2 items on $rEFBs$
- 1 item on $rABDs$
- 1 item on $rAEFs$

This results in the following flow



■

Answer 13 The maximum flow can be found by summing flow from r , which is $z = 2 + 4 + 2 = 8$.

We find the minimum cut by marking all nodes that are reachable from r . This becomes $R = \{r\}$ and $\bar{R} = \{A, B, C, D, E, F, s\}$. The capacity of the cut is $c(R) = 2 + 4 + 2 = 8$. So, the solution is optimal.

Notice that for a circulation problem (if feasible), a minimum cut will be across all edges going out from r , or across all edges going into s . ■

Hungarian Algorithm

In the *Assignment problem* we are asked to assign a number of resources to an equal number of activities, so as to minimize total cost of allocation. Each resource should be assigned to exactly one activity, and each activity should have exactly one resource assigned.

The assignment problem is frequently solved with the *Hungarian Algorithm*: Given a cost matrix $C = \{c_{ij}\}$ saying how much it costs to assign resource i to activity j . The first step of the algorithm is to subtract the smallest number from each column, then subtract the smallest number from each row.

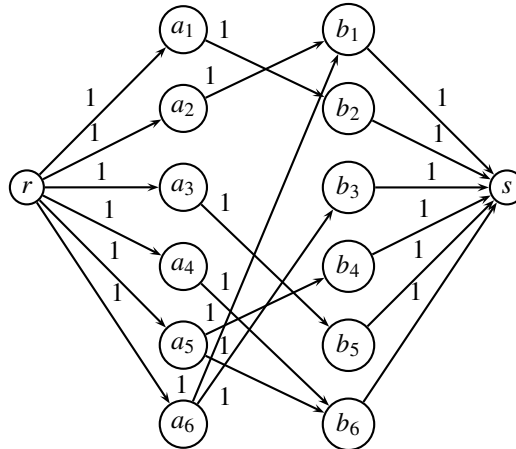
Next, we need to find the largest set of *independent* 0-values. Two 0-values are *independent* if they do not appear in the same row or column. For instances, in the first matrix below the bold-face 0-values are independent, but not in the second matrix

$$\begin{pmatrix} . & \mathbf{0} & . & . & . & . \\ \mathbf{0} & . & . & . & . & . \\ . & . & . & . & \mathbf{0} & . \\ . & . & . & . & . & \mathbf{0} \\ . & . & . & 0 & . & 0 \\ 0 & . & \mathbf{0} & . & . & . \end{pmatrix} \qquad \begin{pmatrix} . & \mathbf{0} & . & . & . & . \\ 0 & . & . & . & . & . \\ . & . & . & . & 0 & . \\ . & . & . & . & . & \mathbf{0} \\ . & . & . & 0 & . & \mathbf{0} \\ \mathbf{0} & . & \mathbf{0} & . & . & . \end{pmatrix} \quad (1)$$

Answer 14 We formulate it as a *maximum flow problem* with root r and sink s .

Introduce a node a_i for each row i in matrix C , and introduce a node b_j for each column j in matrix C . Connect the root r to a_i with edge capacity 1. Connect node b_j to the sink s with edge capacity 1.

For each zero value in the matrix $c_{ij} = 0$ connect node a_i and b_j using an edge with capacity 1.



If a maximum flow solution has flow on edge (a_i, b_j) then the corresponding zero-value in the matrix ($c_{ij} = 0$) should be selected. ■

Repositioning of empty containers

Consider the MEDITLINE network from the project assignment. In a given month, we have the following inventory of empty containers:

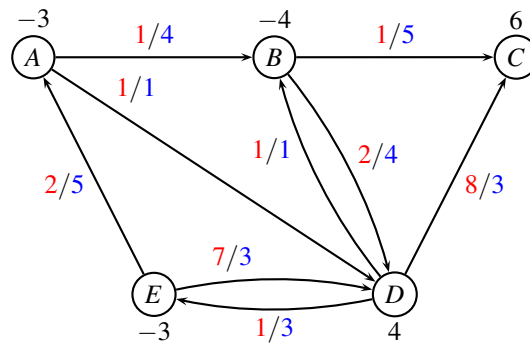
node	A	B	C	D	E
inventory	3	4	2	4	3

All empty containers are equal. We want to reposition the empty containers such that all containers are moved to C and D . Furthermore, the two nodes C and D should have the same amount of containers after repositioning.

Answer 15 We have 16 containers in total, so C and D should have 8 each after repositioning. This results in the following demands:

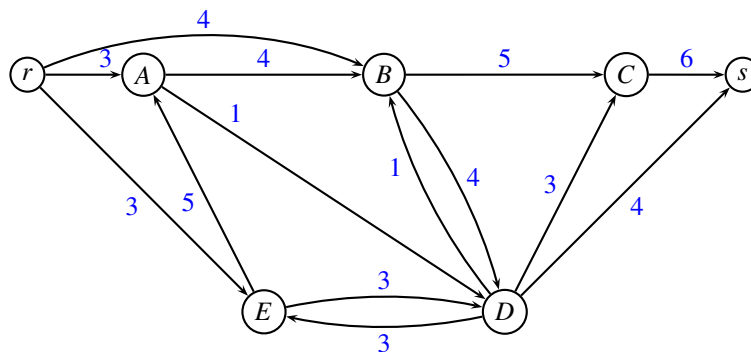
node	A	B	C	D	E
demand	-3	-4	6	4	-3

Adding demands to the graph gives the min-cost-flow problem:



Answer 16 We use the *augmenting cycle* algorithm to solve the min-cost flow problem.

To find an initial flow we solve a maximum flow problem. We first add a root r and a sink s . The root r is connect to all nodes with negative demand, with edge capacity equal to the demand. All nodes with positive demand are connected to the sink s , with edge capacity equal to the demand. In this way we get:

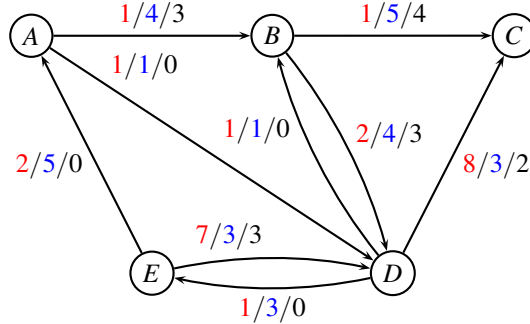


We solve the maximum flow problem with the *augmenting path* algorithm:

- 4 units $rBCs$
- 3 units $rABDs$

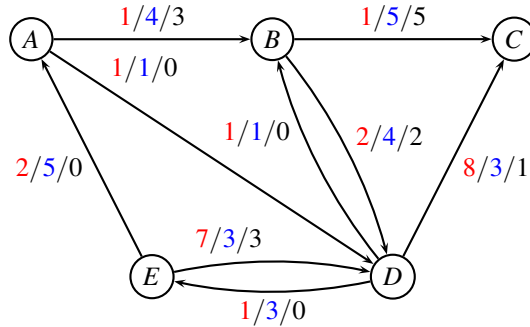
- 2 units $rEDCs$
- 1 units $rEDs$

Now, we have the flow:

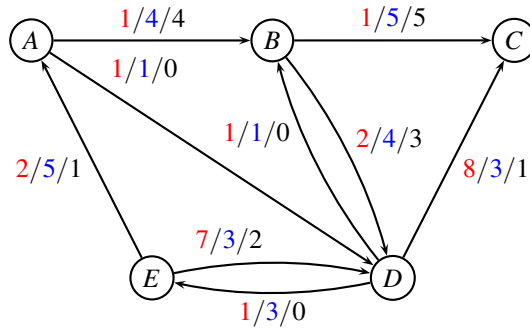


The cost of the flow is $z = 1 \cdot 3 + 1 \cdot 4 + 2 \cdot 3 + 8 \cdot 2 + 7 \cdot 3 = 50$.

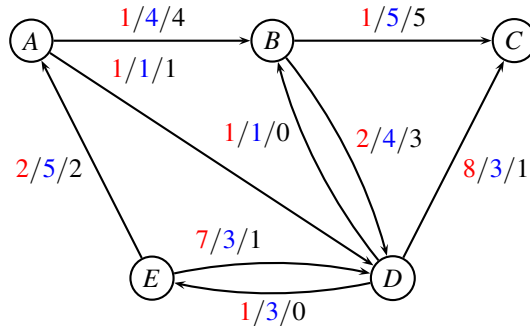
An augmenting cycle is BCD with cost $1 - 8 - 2 = -9$. The bottleneck is $\delta = \min\{1, 2, 3\} = 1$. We send 1 unit along the cycle getting:



An augmenting cycle is $ABDE$ with cost $1 + 2 - 7 + 2 = -2$. The bottleneck is $\delta = \min\{1, 2, 3, 5\} = 1$. We send 1 unit along the cycle getting:



An augmenting cycle is $ADEA$ with cost $1 - 7 + 2 = -4$. The bottleneck is $\delta = \min\{1, 2, 4\} = 1$. We send 1 unit along the cycle getting:



No more negative cycles can be found, so we terminate. The cost of the flow is $z = 1 \cdot 4 + 1 \cdot 5 + 2 \cdot 2 + 1 \cdot 1 + 2 \cdot 3 + 8 \cdot 1 + 7 \cdot 1 = 35$ ■

Checking:

```

minimize 1 xAB + 1 xAD + 1 xBC + 2 xBD + 1 xDB + 8 xDC + 1 xDE + 2 xEA + 7 xED
subject to
    xEA - xAB - xAD = -3
    xAB + xDB - xBC - xBD = -4
    xBC + xDC = 6
    xAD + xBD + xED - xDB - xDC - xDE = 4
    xDE - xEA - xED = -3

    xAB <= 4
    xAD <= 1
    xBD <= 4
    xBC <= 5
    xDB <= 1
    xDC <= 3
    xDE <= 3
    xEA <= 5
    xED <= 3
end

```

```

CPLEX> opt
Dual simplex - Optimal: Objective = 3.5000000000e+01
CPLEX> dis sol var -
Variable Name      Solution Value
xAB                4.000000
xAD                1.000000
xBC                5.000000
xBD                3.000000
xDC                1.000000
xEA                2.000000
xED                1.000000
All other variables in the range 1-9 are 0.

```

Flowing containers

We consider again the route net MEDITLINE from project assignment 2023, in which we have *new demands* as follows:

k	s_k	t_k	d_k
1	E	D	3
2	D	C	1
3	E	C	2

Commodity $k = 1$ is shipping three containers from E to D , commodity $k = 2$ is shipping one container from D to C , and commodity $k = 3$ is shipping two containers from E to C .

We are considering a *unsplittable multi-commodity flow problem* in which all the d_k containers must follow the same path.

Answer 17 We have the routes r outlined in the below table. The cost of sending d_k units along a route r is calculated as $d_k \sum_{(i,j) \in r} c_{ij}$.

number r	commodity k	route r	d_k	$\sum c_{ij}$	cost
1	1	$EABD$	3	5	15
2	1	ED	3	7	21
3	2	DBC	1	2	2
4	2	DC	1	8	8
5	2	$DEABC$	1	5	5
6	3	$EABC$	2	4	8
7	3	$EABDC$	2	13	26
8	3	EDC	2	15	30

■

We solve the LP-relaxed problem through *column generation*. Let y_{ij} denote the dual variable corresponding to the capacity constraint for edge (i, j) , and let $\bar{y}_k$ denote the dual variable corresponding to the demand constraint for commodity k .

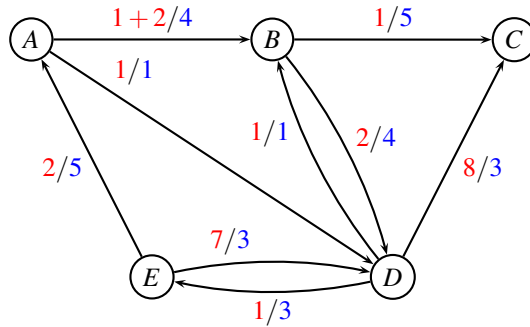
The dual variables for the above model are: $y_{AB} = -2$, while all other are $y_{ij} = 0$. Furthermore, $\bar{y}_1 = 21$, $\bar{y}_2 = 8$, $\bar{y}_3 = 30$.

Answer 18 The reduced cost of a route r is calculated as

$$\bar{c}_r^k = \sum_{(i,j) \in r} d_k c_{ij} - \sum_{(i,j) \in r} d_k y_{ij} - \bar{y}_k = \sum_{(i,j) \in r} d_k (c_{ij} - y_{ij}) - \bar{y}_k$$

In order to solve the pricing-problem as a shortest path problem we set the cost of edge (i, j) to $\tilde{c}_{ij} = d_k(c_{ij} - y_{ij})$. For commodity k we first remove all edges with capacity $u_{ij} < d_k$, as they cannot accommodate the demand. Next, we solve the shortest path problem using edge costs $\tilde{c}_{ij}$, with root s_k and destination t_k . Let z_k be the length of the shortest path for commodity k .

Since $y_{AB} = -2$ and remaining $y_{ij} = 0$, we get the graph:



Edges with capacity less than d_k are removed when solving the shortest path problem for commodity k . In this way we get:

$k = 1$: We remove edges $(AD), (DB)$. The shortest path is ED (route 2) with cost $z = 7$. Reduced cost is $d_1 \cdot z - \bar{y}_1 = 3 \cdot 7 - 21 = 0$.

$k = 2$: No edges are removed. The shortest path is DBC (route 3) with cost $z = 2$. Reduced cost is $d_2 \cdot z - \bar{y}_2 = 1 \cdot 2 - 8 = -6$.

$k = 3$: We remove edges $(AD), (DB)$. The shortest path is $EABC$ (route 6) with cost $z = 6$. Reduced cost is $d_3 \cdot z - \bar{y}_3 = 2 \cdot 6 - 30 = -18$.

■

Answer 19 The path with most negative reduced cost is added to the problem, which in our case is EABC (route 6). ■

Checking:

```

minimize
+15x1+21x2+2x3+8x4+11x5+8x6+26x7+30x8
subject to
UAB: +3x1          +1x5+2x6+2x7      <= 4
UAD:              <= 1
UBC:          +1x3      +1x5+2x6      <= 5
UBD: +3x1          +2x7      <= 4
UDB:          +1x3          <= 1
UDC:              +1x4      +2x7+2x8 <= 3
UDE:              +1x5          <= 3
UEA: +3x1          +1x5+2x6+2x7      <= 5
UED:      +3x2          +2x8 <= 3

K1: +x1+x2 = 1
K2: +x3+x4+x5 = 1
K3: +x6+x7+x8 = 1

\\ R1: x1 <= 0
\\ R2: x2 <= 0
R3: x3 <= 0
\\ R4: x4 <= 0
R5: x5 <= 0
R6: x6 <= 0
\\ R7: x7 <= 0
R8: x8 <= 0
end

```

```

CPLEX> opt
Dual simplex - Optimal: Objective = 5.1000000000e+01

CPLEX> dis sol var -
Variable Name      Solution Value
x1                  0.666667
x2                  0.333333
x4                  1.000000
x7                  1.000000
All other variables in the range 1-8 are 0.
CPLEX> dis sol dua -
Constraint Name      Dual Price
UAB                  -2.000000
K1                   21.000000
K2                    8.000000
K3                   30.000000
R3                   -6.000000
R6                  -18.000000
All other dual prices in the range 1-16 are 0.

```

_____end of solution_____

Solution 17 December, 2024

Course title: Network Optimization

Course no: 42115

Aids allowed: All

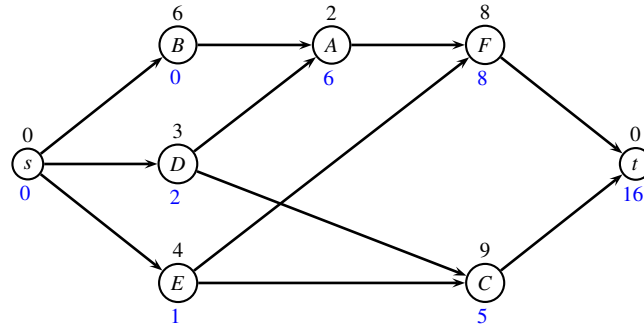
Exam duration: 4 hours

Critical path with time windows

Assume that each task j has a corresponding time windows $[e_j, \ell_j]$ denoting the earliest and latest time when the task may start.

activity i	description	duration d_i	predecessor	e_j	ℓ_j
A	Boil apples	2	B, D	0	∞
B	Peel apples and cut into cubes	6		0	∞
C	whip cream	9	D, E	0	6
D	slice vanilla bean, scrape out seeds	3		2	3
E	fry bread-crumbs with butter	4		1	∞
F	mix apples and bread-crumbs	8	A, E	0	∞

Answer 1 Introduce start and end nodes s, t . Add edges (s, i) to nodes i with no predecessor, and edges (i, t) to nodes i with no successor. Then we have the topological ordering



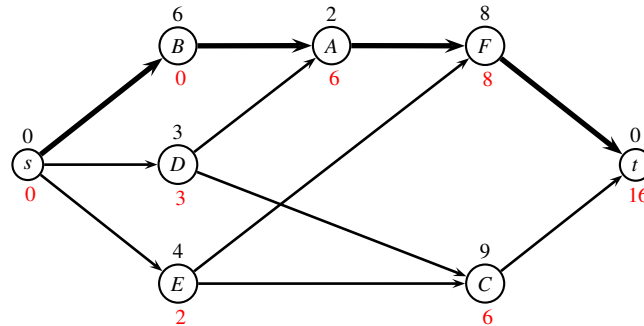
Let U_i be the earliest starting time of task i (marked with blue). *Forward procedure:* Consider the nodes j in topological order. Starting with $U_s = 0$, calculate the earliest time U_j each activity j can be started, using the recursion $U_j = \max \{e_j, \max_{(i,j) \in E} \{U_i + p_i\}\}$. The *makespan* is $T = U_t = 16$. ■

Note: An alternative method is to add predecessor tasks to handle e_j in time windows. For each task j with $e_j > 0$ add a dummy task between s and j with duration e_j .

Note: At this stage it is not possible to add successor tasks of duration ℓ_j after each node j .

Answer 2 Let V_i be the latest starting time (marked with red). *Backward procedure:* Consider the nodes j in backward topological order. Starting with $V_t = T$, calculate the latest time V_i each activity

i can be started using the recursion $V_i = \min \{ \ell_i, \min_{(i,j) \in E} \{ V_j - p_i \} \}$.



■

Note: An alternative method is to add successor tasks to handle ℓ_j in time windows. For each task j with $\ell_j < \infty$ add a dummy task between j and t with duration $T - \ell_j - p_j$.

Answer 3 The critical path is found where nodes have $U_i = V_i$. It is indicated with fat edges above, as $sBAFt$. ■

Note: When time windows are included in the recursion, the critical path is not guaranteed to be a full path from s to t (but it is in our case).

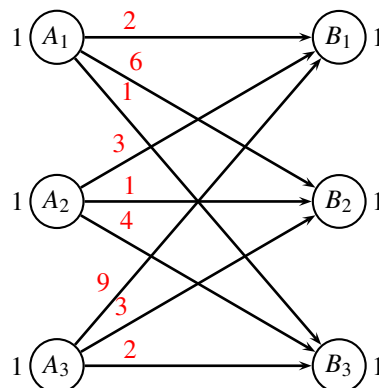
Assignment problem

In the *Assignment problem* we are asked to assign a number of resources A_i to an equal number of activities B_j , so as to minimize total cost of allocation. Each resource should be assigned to exactly one activity, and each activity should have exactly one resource assigned.

The below cost matrix $C = \{c_{ij}\}$ says how much it costs to assign resource i to activity j .

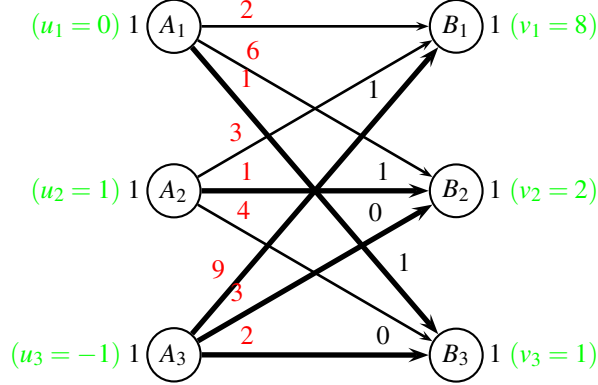
	B_1	B_2	B_3
A_1	2	6	1
A_2	3	1	4
A_3	9	3	2

Answer 4 Construct a bipartite graph with nodes A_i and B_j . All nodes A_i and B_j have a production/demand of 1.



■

Answer 5 We first consider A_1 , and send 1 unit to B_3 . Next, we consider A_2 , and send 1 unit to B_2 . Finally, we consider A_3 , and send 1 unit to B_1 . This gives the following greedy solution:



The cost of the solution is $z = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 9 = 11$.

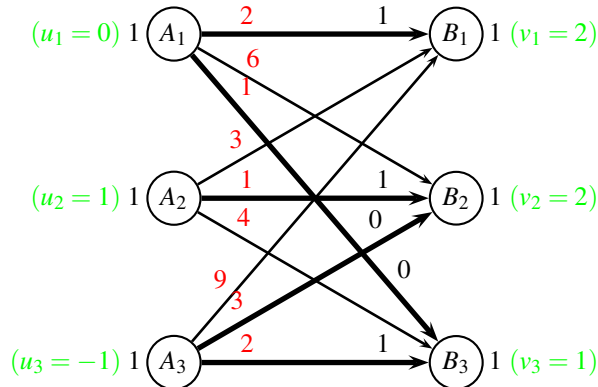
Fat edges on the graph indicate the basis. Since we have only 3 basis variables, we add two additional fat edge (A_3, B_2) . and (A_3, B_3) . (These can be arbitrarily chosen as long as the fat edges are a tree).

Dual values (indicated with green) are calculated by finding the shortest path from node A_1 , using only the fat edges. ■

Answer 6 The reduced costs are calculated as $\bar{c}_{ij} = c_{ij} + u_i - v_j$. This gives

$$\begin{aligned}\bar{c}_{11} &= 2 + 0 - 8 = -6 \\ \bar{c}_{12} &= 6 + 0 - 2 = 4 \\ \bar{c}_{21} &= 3 + 1 - 8 = -4 \\ \bar{c}_{23} &= 4 + 1 - 1 = 4\end{aligned}$$

The most negative reduced cost is for edge (A_1, B_1) giving the cycle $(A_1, B_1, A_3, B_3, A_1)$. Sending $\theta = \min\{1, 1\} = 1$ units along the cycle gives:



Edge A_1B_1 becomes fat while bottleneck edge A_3B_1 becomes thin. The cost of the solution is $z = 1 \cdot 2 + 1 \cdot 1 + 1 \cdot 2 = 5$.

The reduced costs are calculated as $\bar{c}_{ij} = c_{ij} + u_i - v_j$. This gives

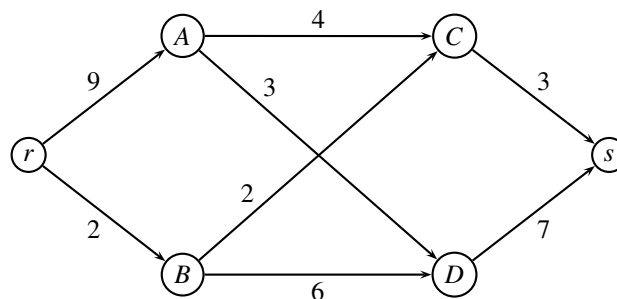
$$\begin{aligned}\bar{c}_{12} &= 6 + 0 - 2 = 4 \\ \bar{c}_{21} &= 3 + 1 - 2 = 2 \\ \bar{c}_{23} &= 4 + 1 - 1 = 4 \\ \bar{c}_{31} &= 9 + (-1) - 2 = 6\end{aligned}$$

No more negative reduced costs, so we finish. The optimal solution is $z = 5$. ■

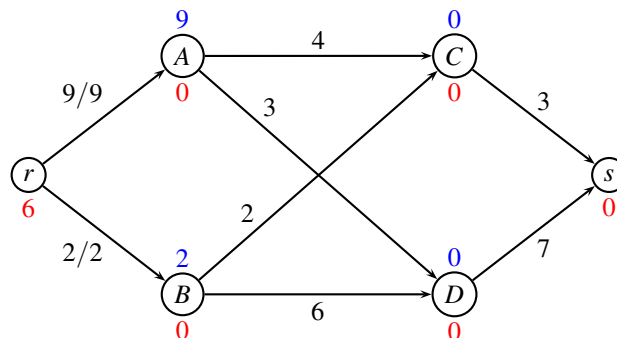
Note: We could also have chosen A_1B_3 as bottleneck edge, since they both have flow 1. In the next iteration we will find a cycle of capacity $\theta = 0$, giving us a new basis. Then we can show that all reduced costs are positive.

Maximum flow problem

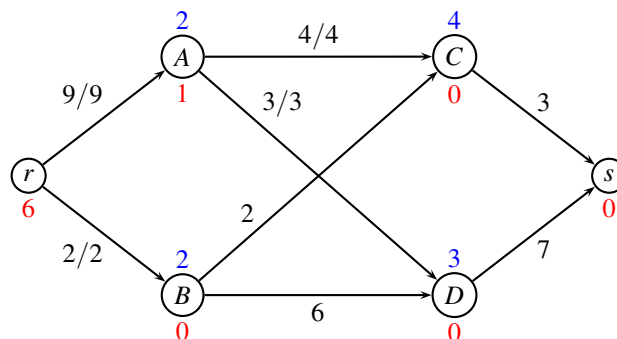
Consider the following maximum flow problem where the capacity u_{ij} is indicated on edge $(i, j) \in E$.



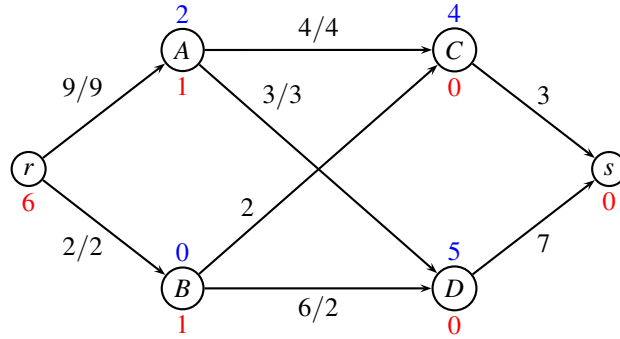
Answer 7 For node i , red value is label d_i , blue value is flow excess $f_x(i)$. Initialize: Set $d_r = n$ and all other $d_i = 0$. Push as much as possible from r .



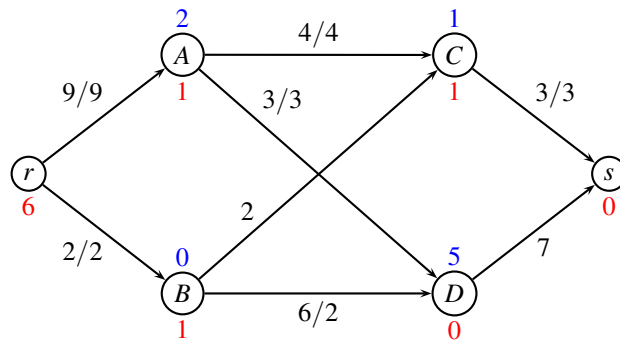
Iteration 1: relabel A to 1, push 4 on AC , and 3 on AD



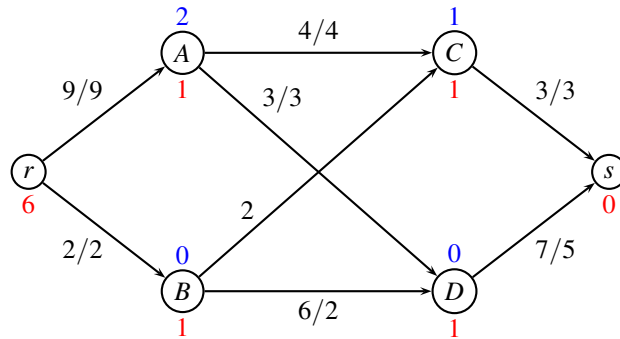
Iteration 2: relabel B to 1, push 2 on BD



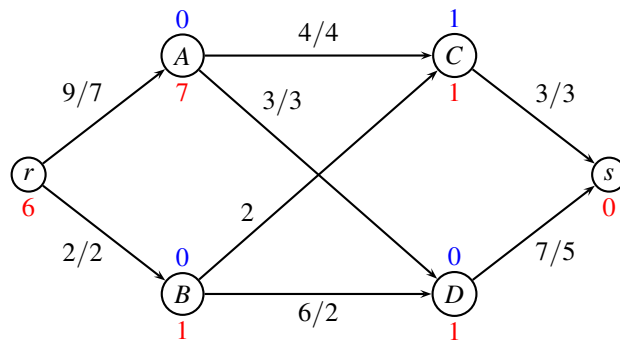
Iteration 3: relabel C to 1 , push 3 on Cs



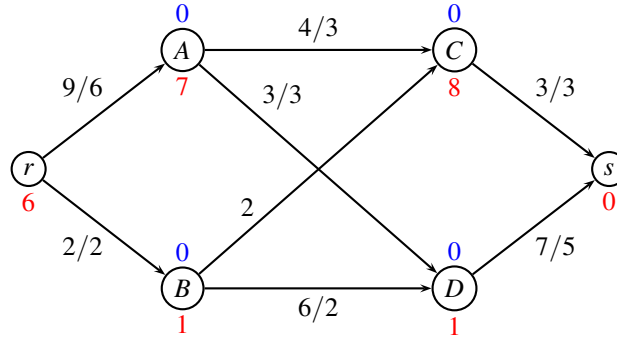
Iteration 4: relabel D to 1 , push 5 on Ds



Iteration 5: relabel A to 7 (the only outgoing edge in residual graph E_x is Ar), push 2 on Ar



Iteration 6: relabel C to 8 (the only outgoing edge in E_x is CA), push 1 on CA , push 1 on Ar

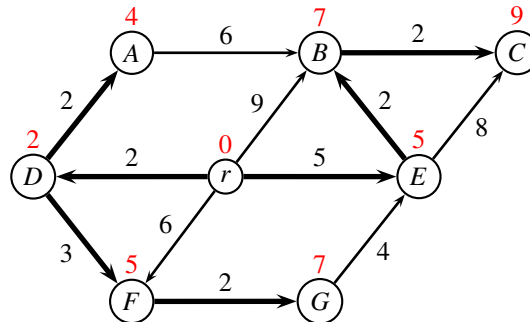


Flow is feasible, hence we stop. The optimal solution is $|f| = 6 + 2 = 8$. ■

Note: In iteration 5, if we relabel C instead of A , then we may end up with several iterations back and forth between A and C .

Answer 8 A minimum cut is found by selecting an unused label value, e.g. $k = 2$. Then we have $R = \{i \mid d_i > k\} = \{r, A, C\}$ and $\bar{R} = \{i \mid d_i < k\} = \{B, D, s\}$. The capacity of the cut is $u(R) = 2 + 3 + 3 = 8$. ■

A* algorithm for shortest path

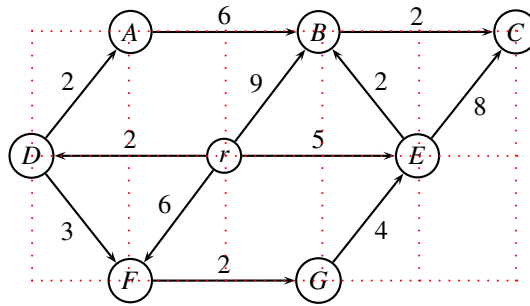


Answer 9 Running Dijkstra's algorithm. Each row indicates the node we are considering. We get the distances y_i as follows

node	r	A	B	C	D	E	F	G
init	0	∞	∞	∞	∞	∞	∞	∞
r			9		2	5	6	
D		4					5	
A								
F								7
E			7	13				
G								
B				9				
C								

For each node, the optimal distance is marked with bold in the above table. The distances are indicated at the above graph.

The shortest path tree is found by backtracking through the predecessors for each node. For instance, node C (distance 9) was updated when we considered node B , hence we mark edge BC fat. The resulting shortest path tree is shown above. ■



If we only are interested in solving the shortest distance to a specific terminal node $t = C$, we can use the A^* algorithm which makes use of heuristic distances to guide the search.

The Manhattan distance between two nodes (x_1, y_1) and (x_2, y_2) is measured as $h = |x_1 - x_2| + |y_1 - y_2|$. For instance, the Manhattan distance from r to C is $3 + 1 = 4$, while the distance from F to C is $4 + 2 = 6$.

Answer 10 The Manhattan distances are:

i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

■

The following A^* algorithm finds the shortest path from r to t . It is identical to Dijkstra's algorithm, except that in every iteration we select the node i with lowest value of $y_i + h_i$. (The normal Dijkstra algorithm selects the node with lowest value of y_i). Furthermore, the algorithm stops when node t is added to T (i.e. marked "red").

```

1  $T \leftarrow \emptyset$ ;
2  $y_r \leftarrow 0$ ;  $y_i \leftarrow \infty$  for all other  $i$ ;
3 while  $V \setminus T \neq \emptyset$  do
4   select an  $i \in V \setminus T$  minimizing  $y_i + h_i$ ;
5   for  $\{(i, j) \in E : j \notin T\}$  do
6     if  $y_i + w_{ij} < y_j$  then
7        $y_j \leftarrow y_i + w_{ij}$ ;
8    $T \leftarrow T \cup \{i\}$ ;
9   if  $i = t$  then
10    stop

```

Answer 11 Running the A^* algorithm, we get the distances y_i and values $y_i + h_i$ as follows

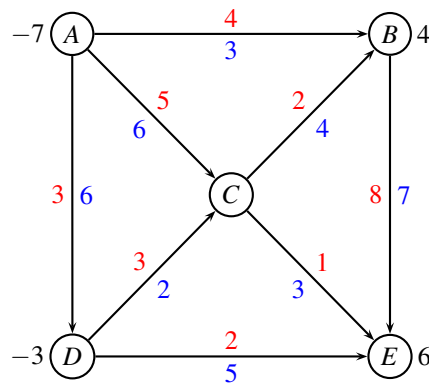
node	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4
init	0	∞	∞	∞	∞	∞	∞	∞
r			9 11		2 8	5 7	6 12	
E			7 9	13 13				
D		4 8					5 11	
A								
B				9 9				
C								

We stop as soon as $t = C$ is reached. As before, the shortest distance to node C is 9. The shortest path is found by following the predecessors (marked with boldface): The predecessor of C is B . The predecessor of B is E . The predecessor of E is r . Hence, the shortest path is $r \rightarrow E \rightarrow B \rightarrow C$, with cost $5 + 2 + 2 = 9$.

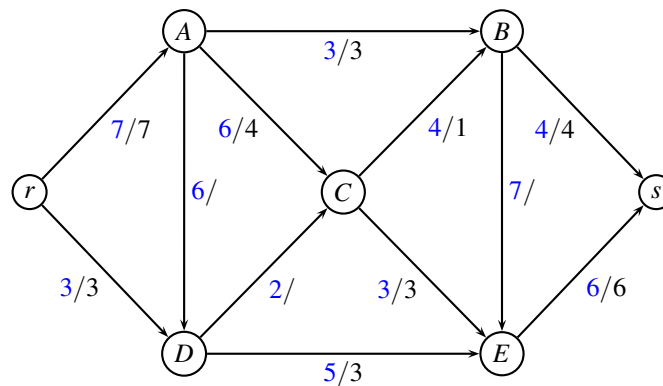
Compared to the previous question, fewer updates are needed. However, we do not have a shortest path tree (only the path $r \rightarrow C$), since we terminated early. ■

Minimum cost flow problem

Consider the following minimum cost flow problem costs w_{ij} are indicated with **red**, capacities u_{ij} are indicated with **blue**, and demands d_i are indicated in black.



Answer 12 We add a root node r and connect it to all nodes i having a negative demand d_i using an edge (r, i) with capacity $-d_i$. We add a sink node s and connect it to all nodes i having a positive demand d_i using an edge (i, s) with capacity d_i . All costs are removed. In this way we get the following maximum flow problem: In this way we get the following solution

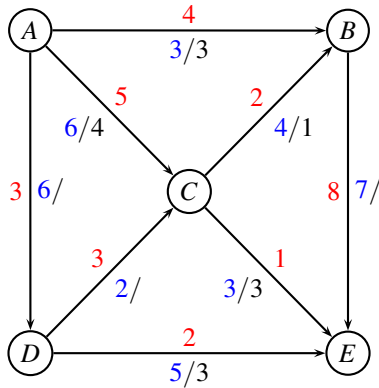


In order to solve the problem we use the following augmenting paths

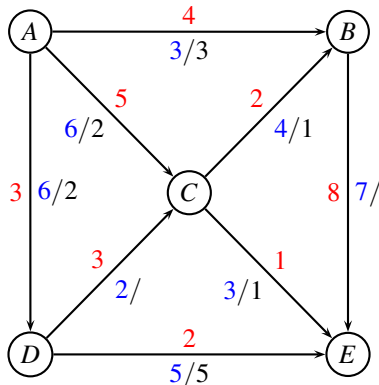
- send 3 units along path $rABs$
- send 1 units along path $rACBs$
- send 3 units along path $rACEs$
- send 3 units along path $rDEs$

This results in a solution of cost 43. ■

Answer 13 We use the **augmenting cycle** algorithm. We have the initial solution:

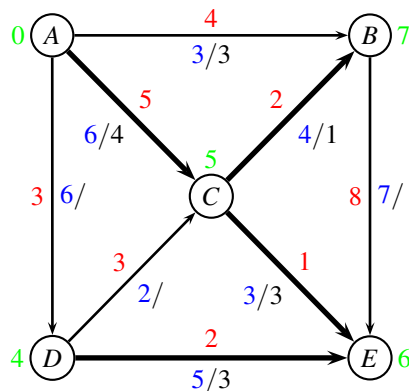


We find a negative cycle $ADECA$ with cost $3 + 2 - 1 - 5 = -1$. We can send $\theta = \min\{6, 2, 3, 4\} = 2$ units.



The cost of the solution is 41. No more negative cycles exist, so we terminate.

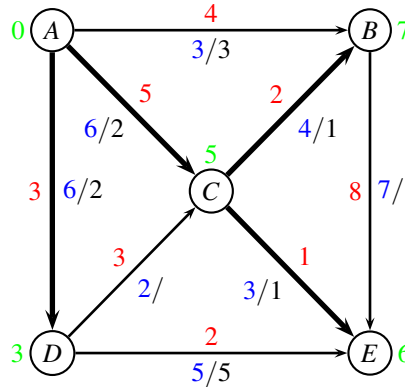
We can also use the **network simplex** algorithm. We have the initial solution:



The tree is $T = \{AC, CB, CE, DE\}$, lower bound edges are $L = \{AD, BE, DC\}$, upper bound edges are $U = \{AB\}$. Dual costs are calculated as distance from A (marked with green above). We find the reduced costs as $\bar{c}_{ij} = c_{ij} + u_i - u_j$, getting

$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - 7 = -1 & (U \text{ edge}) \\ \bar{c}_{AD} &= 3 + 0 - 4 = -1 & (L \text{ edge}) \\ \bar{c}_{BE} &= 8 + 7 - 6 = 9 & (L \text{ edge}) \\ \bar{c}_{DC} &= 3 + 4 - 5 = 2 & (L \text{ edge})\end{aligned}$$

Edge AD is lower bound edge, and has negative reduced cost. The corresponding cycle is $ADECA$ with bottleneck capacity $\theta = \min\{6, 2, 3, 4\} = 2$. Sending 2 units along the cycle. AD becomes fat, and bottleneck edge DE becomes thin.



The tree is $T = \{AC, AD, CB, CE\}$, lower bound edges are $L = \{BE, DC\}$, upper bound edges are $U = \{AB, DE\}$.

Dual costs are calculated as distance from A. We find the reduced costs:

$$\begin{aligned}\bar{c}_{AB} &= 6 + 0 - 7 = -1 && (U \text{ edge}) \\ \bar{c}_{BE} &= 8 + 7 - 6 = 9 && (L \text{ edge}) \\ \bar{c}_{DC} &= 3 + 4 - 5 = 2 && (L \text{ edge}) \\ \bar{c}_{DE} &= 2 + 4 - 6 = 0 && (L \text{ edge})\end{aligned}$$

No lower bound edges have negative reduced cost, and no upper bound edges have positive reduced cost. So we terminate. The cost of the solution is 41. ■

Note: Depending on the start solution, it may be necessary with more iterations. If you find an optimal solution as start solution, try to modify it, so you at least have one iteration.

```

minimize
  4 xAB + 5 xAC + 3 xAD + 8 xBE + 2 xCB + 1 xCE + 3 xDC + 2 xDE
subject to
A: -xAB - xAC - xAD          = -7
B:  xAB + xCB - xBE          = 4
C:  xAC + xDC - xCB - xCE    = 0
D:  xAD - xDC - xDE          = -3
E:  xBE + xCE + xDE          = 6

bounds
  xAB <= 3
  xAC <= 6
  xAD <= 6
  xBE <= 7
  xCB <= 4
  xCE <= 3
  xDC <= 2
  xDE <= 5
end

```

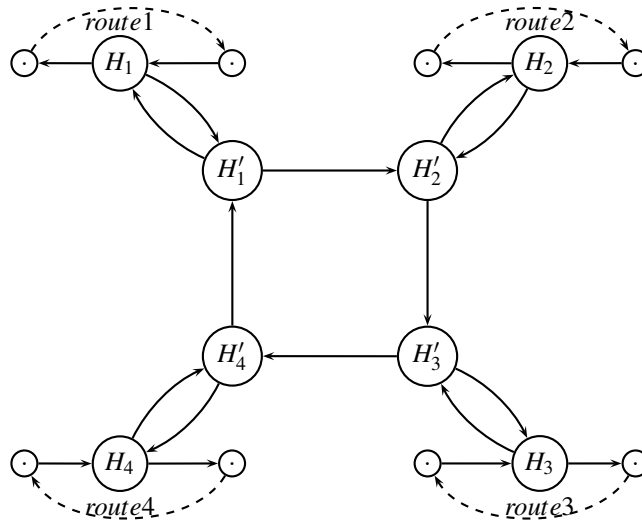


```

CPLEX> opt
Dual simplex - Optimal: Objective = 4.1000000000e+01
CPLEX> dis sol var -
Variable Name          Solution Value
xAB                    3.000000
xAC                    2.000000
xAD                    2.000000
xCB                    1.000000
xCE                    1.000000
xDE                    5.000000
All other variables in the range 1-8 are 0.

```

Modeling a hub



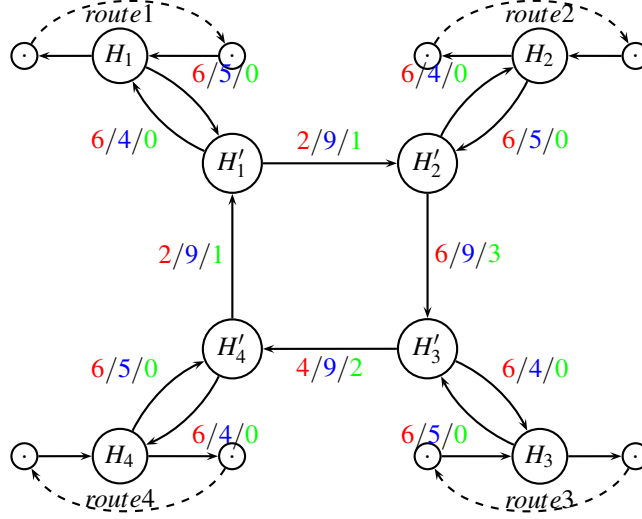
We study how the hub port H in a liner shipping network, HUBLINE, can be formulated in graph terms as part of a multi-commodity flow network. We have four routes arriving to the hub as follows:

route	1	2	3	4
weekday	Monday	Tuesday	Friday	Sunday

We assume that each vessel is only one day in the port, and it is assumed that all loading and unloading is done during this day.

Crane capacity: The crane capacity is $u = 5$ for unloading and $u = 4$ for loading a vessel. *Transshipment cost:* The cost of loading/unloading is 6 euro per container. *Transshipment time:* The time for transferring a container from one vessel to another can be calculated from the time table. *Storage capacity:* The port has a limited storage capacity of $\bar{u} = 9$ containers on shore. *Storage cost:* The shipping company must pay $\bar{c} = 2$ euro for each container each night it is stored in the port.

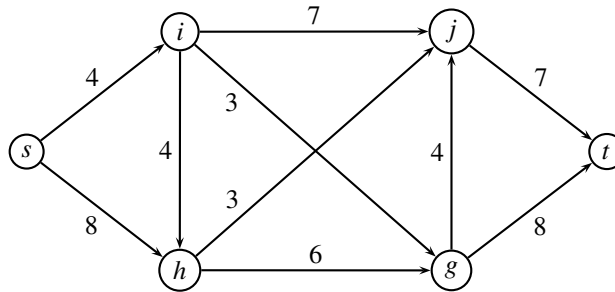
Answer 14 All values can be modeled as follows:



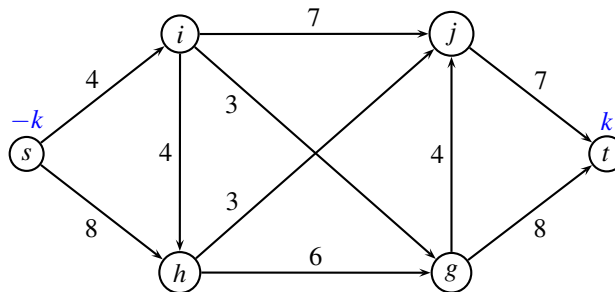
■

Maximum flow with lower bound on edges

Given a directed graph $G = (V, E, u)$, root/sink (s, t) and a target for the flow k . We assume that all capacities u are positive integers.

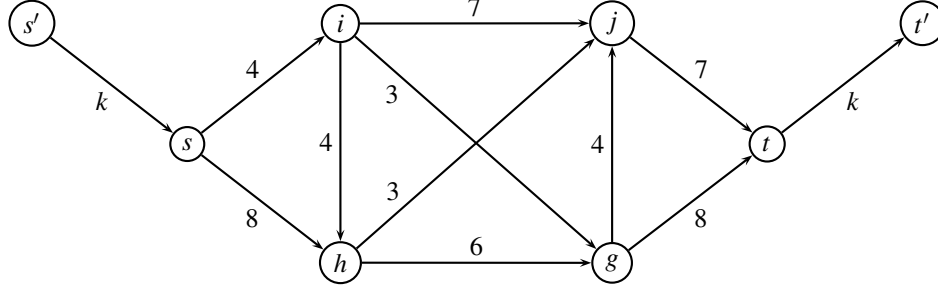


Answer 15 We formulate it as a circulation problem, with demand $d_s = -k$ and $d_t = k$.



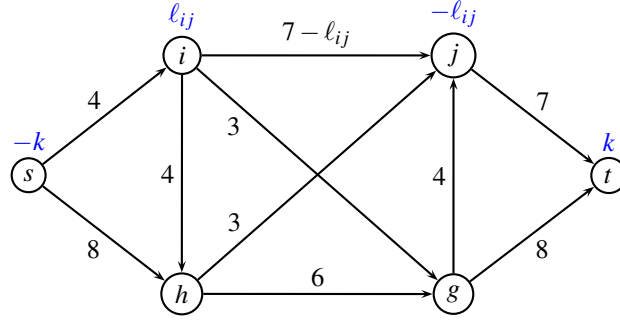
To modify the circulation problem to a normal maximum-flow problem a super-source s' and super-sink t' is added. Furthermore we add edge (s', s) with capacity k , and edge (t, t') with capacity k . If we can find a flow of value $|f| = k$ (i.e. all edges from s' are saturated), the circulation problem is

feasible.



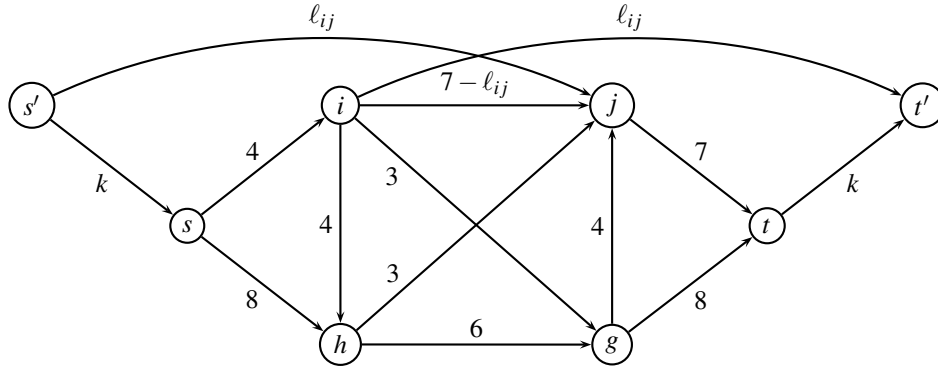
■

Answer 16 We formulate it as a circulation problem, where node i gets demand ℓ_{ij} and node j gets demand $-\ell_{ij}$. Furthermore, we have demand $d_s = -k$ and $d_t = k$.



To modify the circulation problem to a normal maximum-flow problem a super-source s' and super-sink t' is added. We add edge (s', s) with capacity k and (s', j) with capacity ℓ_{ij} . We add edge (t, t') with capacity k and (i, t') with capacity ℓ_{ij} . If we can find a flow of value $|f| = k + \ell_{ij}$ (i.e. all edges from s' are saturated), the circulation problem is feasible.

Notice that the method works for multiple edges with lower bound, by repeating the transformation for each edge.



■

Answer 17 We first find an upper bound U on the maximum flow $|f|$ as sum of the capacities for edges leaving s , or entering t .

$$U = \min \left\{ \sum_{(s,i) \in E} u_{si}, \sum_{(i,t) \in E} u_{it} \right\}$$

Now, use binary search to find a maximum flow. I.e. first choose $k = \lfloor U/2 \rfloor$, and solve the above circulation problem. If the problem is feasible we know that z is between k and U . Otherwise z is between 0 and $k - 1$. Repeat until the optimal value z^* has been found. This means that we will solve $\log U$ maximum-flow problems, which is a polynomial algorithm. ■